

ECE 201 Spring 2026

Instructors: Sanjeev Kulkarni, Pramod Viswanath

Contents

1	Introduction	1
1.1	Scope of Electrical Engineering	1
1.2	A Signals and Systems Approach	1
1.3	Unreasonable Effectiveness and Breadth of Electrical Engineering	3
2	Basics of Signals	5
2.1	What are Signals?	5
2.2	Analog and Digital Signals	7
2.3	Some Basic Signal Operations	10
2.4	Noise	12
2.5	Some Common Signals	13
2.6	Delta Functions	15
2.7	Separable Signals in 2-D	20
3	Basics of Systems	23
3.1	What are Systems?	23
3.2	Linearity	24
3.3	Time-Invariance	25
3.4	Impulse Response and Output of LTI Systems	25
3.5	Convolution	28
3.6	Frequency Response of LTI Systems	33
4	Frequency Domain and Fourier Transforms	35
4.1	Frequency Content: Combining Sinusoids	35
4.2	Some Motivating Examples	36
4.3	A Trivial Frequency Decomposition	39
4.4	Summary of the Four Cases	40
4.5	FT and DFT	43
4.6	Some Examples of Transform Pairs	45
4.7	Some Transform Properties	46
4.8	Convolution Property and Another Look at Impulse and Frequency Response	50
4.9	2-D Transforms*	51
5	Sampling and Quantization	55
5.1	Sampling a Signal	55
5.2	Reconstructing a Signal	56
5.3	The Sampling Theorem	58
5.4	Quantization	62

5.5	Nonuniform Quantization	64
5.6	Dithering and Halftoning	66
6	The Human Auditory and Visual Systems	69
6.1	The Human Ear	69
6.2	Properties of the Auditory System	70
6.3	The Human Eye	71
6.4	Luminance Versus Brightness	72
6.5	Spatial and Temporal Properties	73
6.6	Color Perception and Representation	74
7	Filtering and Enhancement	77
7.1	Point/Histogram Methods	78
7.1.1	Point Transformations	78
7.1.2	Histogram Modification	80
7.2	Time/Spatial Domain Methods	82
7.2.1	Noise Reduction	82
7.2.2	Sharpening	83
7.3	Frequency Domain Methods	84
7.4	Geometric Transformations	85
7.5	Modulation and Demodulation	86
8	Information, Entropy, and Coding	91
8.1	The Need for Data Compression	91
8.2	Basic Idea of Compression	92
8.3	The Coding Problem	92
8.4	Variable-Length Coding	93
8.5	Issues in Variable-Length Coding	94
8.6	Information and Entropy	95
8.7	Huffman Coding	96
8.8	Block Coding	99
8.9	Comments on Huffman Coding	100
9	Some Data Compression Algorithms	103
9.1	Compression for Specific Data Types	103
9.2	Run-Length Coding	104
9.3	Lempel-Ziv Coding	106
9.4	Transform Coding of Images (JPEG)	107
9.5	Video Coding (MPEG)	109
10	Information Protection	113
10.1	Expansion for Protection	114
10.2	Noise and Level of Protection	114
10.3	Erasur Correction	115
10.4	Single Error Detection	117
10.5	Single Error Correction: Preliminaries	118
10.6	Rectangular and Triangular Codes	119
10.7	Higher Dimensional Codes	122

10.8 Hamming Codes	123
10.9 Valid Codewords and Invalid Strings	125
10.10 Hamming Distance	126
10.11 Channel Coding Theorem	128
11 The Laplace Transform	131
11.1 The Bilateral Laplace Transform	131
11.2 Region of Convergence	132
11.3 Some Bilateral Laplace Transform Pairs and Properties	134
11.4 Inverse Laplace Transform	136
11.5 The (Unilateral) Laplace Transform	137

Chapter 1

Introduction

1.1 Scope of Electrical Engineering

In today's world, it's hard to go through a day without encountering some aspect of technology developed by electrical engineers. The impact has been so pervasive and ingrained in society that many revolutionary developments are taken for granted. For example, power transmission, electric lighting, and incandescent bulbs seem mundane, but were highly influential, pioneering developments. Radio, telephony, and television now also seem mundane but again are among the great inventions of modern times. However, in addition to these dated examples, the field of electrical engineering continues to be an exciting, rapidly developing, and influential area of technology. Computers and electronics have witnessed incredible advances over many decades and continue to do so. The internet, wireless communications, and multimedia are further examples of technologies that have had tremendous impact on society. Without a doubt, these and other aspects of information technology will continue to play vital roles for the foreseeable future.

So what is “electrical engineering”? The study of electricity and electric circuits have often been equated with “electrical engineering”. However, these areas form a just small part of this broad field. In addition to studying the basic physical phenomena of electricity and magnetism, electrical engineers design, build, and analyze systems that deal with two basic entities: electric power and information.

1.2 A Signals and Systems Approach

In describing electrical engineering we used the word “system” which is difficult to define. The word “system” is used both in a general, non-technical sense such as that given in a standard dictionary, as well as in a rather precise and more technical sense as we will discuss in Chapter 3. For now, we will consider only the more general use of the word “system”.

In the most general sense, we will think of a system as simply being some portion of our physical world on which we wish to focus our attention. In engineering, we are interested in constructing systems to carry out various tasks. That is, the system is immersed in some environment and through interaction with the environment the system performs certain functions (see Figure 1.1). Usually, for a system to perform some useful function, something needs to be known about the environment. This information is provided by *sensors*. The sensors can be thought of as input

*©2023 by Sanjeev R. Kulkarni. All rights reserved.

[†]Lecture Notes for ECE201 Information Signals.

[‡]Thanks to Richard Radke for producing the figures.

devices that measure certain physical variables and pass on these *signals* (or data, information) for further processing. To carry out its task, the system must often alter or in some way interact with the environment. It does this through *actuators*, which can be thought of as output devices.

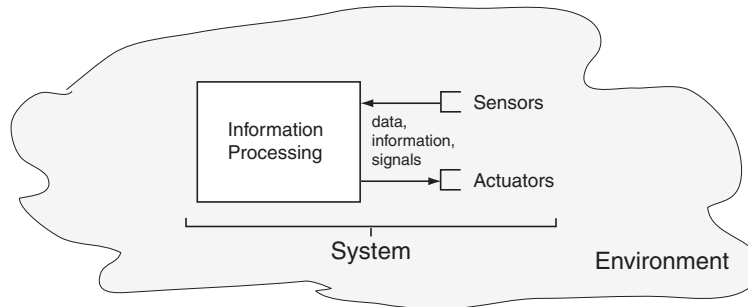


Figure 1.1: A general system immersed in its environment.

To get a sense of the field of electrical engineering, it is helpful to try to give a breakdown of the field based on certain criteria. One approach is to consider a breakdown by areas of application of systems designed by electrical engineers. However, there are so many applications in such a wide range of areas that this approach is difficult.

A second approach is to break down the field by the level of description of the generic system shown in Figure 1.1. One such breakdown might be into layers such as physical, device, component, algorithmic, and conceptual as in Figure 1.2. At the core is the physical layer that describes a system at the most detailed level in terms of the underlying physical phenomena and processes. At the next level, the description would be in terms of devices whose function can in turn be explained directly in terms of the underlying physics. The devices can be considered as building blocks of larger components of the system. These components can be explained in terms of the algorithms (or methods or high-level functions) that they use to accomplish their task. Finally, the algorithms can be considered as particular solutions or implementations of the underlying problems they are designed to solve. A description at the level of the underlying problems themselves constitutes the conceptual layer. This type of breakdown provides a useful categorization. Most advanced topics in electrical engineering can be placed in the context of these layers. For example, the study of quantum effects falls into the physical layer, circuits fall into the device layer, computer architecture falls into the component layer, etc.

A third approach is to give a breakdown in terms of internal tasks that the system carries out. That is, internal to the system, there are a number of different processes or tasks that are often performed, and we can give a breakdown in terms of these processes (as opposed to the external task of the whole system itself, which would be like a breakdown by application). One of the first tasks is sensing: that is, converting physical variables into signals usable by the system. The signals often need to be stored for later use, and depending on the amount of data, the signals may need to be compressed for more efficient storage. Prior to storage, the signals may need to be filtered to eliminate noise or other degradations (in the signals). Once stored, a variety of general purpose computations on the data commonly need to be done. If the system consists of separated

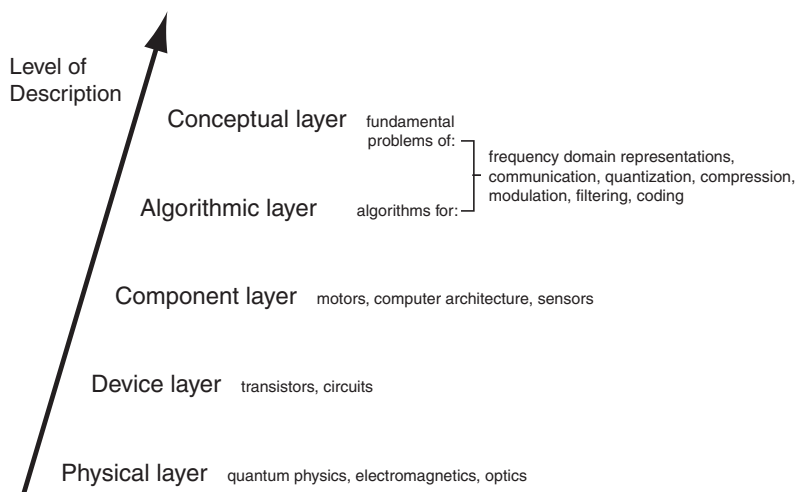


Figure 1.2: A breakdown of electrical engineering by level of description.

sub-systems, these sub-systems may need to communicate with one another. In certain cases there may be a need to protect the information to keep data private or to prevent either intentional or accidental degradation of the data. Sometimes the data needs to be analyzed or higher-level tasks such as object recognition need to be performed. Learning and adaptation are often useful in improving system performance over time or providing robustness to unanticipated changes. Issues of control and actuators are also important since the systems are generally designed to control some aspect of the environment or to provide displays or other outputs for users. Finally, there are some systems that are highly distributed. That is, there are a very large number of distinct units (sometimes mobile) that interact with each other. For these distributed and perhaps mobile systems, a number of new issues arise.

This breakdown by internal task is the approach we will follow, focused on some specific internal tasks from the conceptual and algorithmic layers.

1.3 Unreasonable Effectiveness and Breadth of Electrical Engineering

The Nobel Prize-winning physicist Eugene Wigner commented on the “unreasonable effectiveness of mathematics in the natural sciences.” In a similar way, one might ponder the unreasonable effectiveness of electrical engineering in the applied sciences and technology. The tools and methods of electrical engineering have been far more effective than one might have prior reason to believe. Advances in electrical engineering have led to devices and components that are extremely small, fast, flexible, reliable, and economically feasible. This success has led to a paradigm in engineering systems of converting whatever physical phenomenon the system is dealing with into electrical signals (via sensors), performing any necessary manipulations on the electrical signals, and finally converting electrical signals back to physical actions (via actuators). Thus, even for many non-electrical applications, the core of the system may be largely based on electrical engineering with only the sensors and actuators serving as input and output interfaces with the environment. An interesting consequence of this wildly successful paradigm is that the studies of many information processing tasks in themselves have become parts of the field of electrical engineering. For example,

in addition to just communication based on electrical principles, electrical engineering includes the study of the fundamental problem of communication in and of itself (independent of any particular implementation). This has greatly contributed to making electrical engineering an extremely broad field spanning a range of distinct subfields.

One way to get a sense of the breadth and range of electrical engineering activities is to consider its relationships with other fields of study. Among all traditional disciplines at a typical university, electrical engineering is perhaps the broadest in the sense that various subareas within electrical engineering are closely related to and/or overlap with subareas in many other traditional disciplines. For example,

- computer science: computer architecture, software, networks, algorithms
- physics: optics, quantum physics, semiconductors, electromagnetics
- material science: semiconductors
- mechanical engineering: systems theory, control, robotics
- statistics: detection, estimation, information theory, machine learning
- biomedical engineering: signal processing, medical imaging, prosthetic devices
- mathematics: differential equations, functional analysis, probability and random processes, algebra
- philosophy: inductive inference, learning theory, epistemology
- cognitive science: pattern recognition, learning, neural networks
- economics: time series analysis, game theory
- operations research: decision-making, optimization, stochastic modeling

Chapter 2

Basics of Signals

2.1 What are Signals?

As mentioned in Chapter 1, a system designed to perform a particular task often uses measurements obtained from the environment and/or inputs from a user. These in turn may be converted into other forms. The physical variables of interest are generally called *signals*. In an electrical system, the physical variables of interest might be a voltage, current, amount of charge, etc. In a mechanical system, the variables of interest might be the position, velocity, mass, volume, etc. of various objects. Financial examples might include the price of a stock, commodity, or option, an interest rate, or an exchange rate. In performing its tasks, the system may need to manipulate or combine various signals, extract information, or otherwise process the signals. These actions are called *signal processing* or *signal analysis*.

A convenient abstraction is to model the value of a physical variable of interest by a number. We are usually interested in the physical variable not at just a single time, but rather at a set of times. In this case, the signal is a *function* of time, say $x(t)$, where x denotes the physical variable of interest and t denotes time. For example, $x(t)$ might denote a voltage level, or the velocity of an object, or the price of a stock at time t .

In some cases, we might be interested in measuring the quantity as a function of some variable other than time. For example, suppose we are interested in measuring the water temperature in the ocean as a function of depth. In this case, the signal is a function of a spatial variable, with $x(u)$ denoting temperature at depth u .

A signal need not be a function of just a single variable. To continue the example above, suppose we are interested in the temperature at particular points in the ocean, not simply as a function of depth. In this case, we might let $x(u, v, w)$ denote the temperature at the point (u, v, w) , so the signal is a function of three variables. Now, if we are also interested in how the temperature evolves in time, the signal $x(u, v, w, t)$ would be a function of four variables.

Examples of signals that we will encounter frequently are audio signals, images, and video. An audio signal is created by changes in air pressure, and therefore can be represented by a function of time $x(t)$ with x representing the air pressure due to the sound at time t . An example of an audio signal of someone saying “Matlab” is shown in Figure 2.1. A black and white image can be represented as a function $x(u, v)$ of two variables. Here (u, v) denotes a particular point on the image, and the value $x(u, v)$ denotes the brightness (or gray level) of the image at that point.

*©2023 by Sanjeev R. Kulkarni. All rights reserved.

[†]Lecture Notes for ECE201 Information Signals.

[‡]Thanks to Richard Radke for producing the figures.

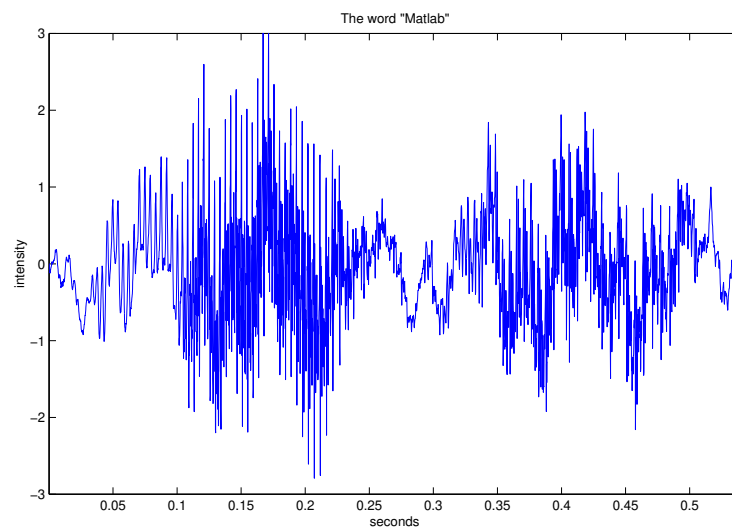


Figure 2.1: Someone saying the word “Matlab”



Figure 2.2: A gray-scale image.

An example of a black and white image is shown in Figure 5.15. A video can be thought of as a sequence of images. Hence, a black and white video signal can be represented by a function $x(u, v, t)$ of three variables (two spatial variables and time). In this case, for a fixed t , $x(\cdot, \cdot, t)$ represents the still image/frame at time t , while for a fixed (u, v) , $x(u, v, \cdot)$ denotes how the brightness at the point (u, v) changes as a function of time.



Figure 2.3: Video frames from a commercial.

Three frames of a video of a commercial are shown in Figure 2.3. It turns out that color images (or video) can be represented by a combination of three intensity images (or video, respectively), as will be discussed later in Chapter 7.

2.2 Analog and Digital Signals

Often the domain and the range of a signal $x(t)$ are modeled as continuous. That is, the time (or spatial) coordinate t is allowed to take on arbitrary values (perhaps within some interval) and the value of the signal itself is allowed to take on arbitrary values (again within some interval). Such signals are called *analog* signals. A continuous model is convenient for some situations, but in other situations it is more convenient to work with *digital* signals — i.e., signals that have a discrete (often finite) domain and range. Two other related words that are often used to describe signals are *continuous-time* and *discrete-time*, referring to signals where the independent variable denotes time and takes on either a continuous or discrete set of values, respectively.

Sometimes a signal that starts out as an analog signal needs to be *digitized* (i.e., converted to a digital signal). The process of digitizing the domain is called *sampling*. For example, if $x(t)$ denotes temperature as a function of time, and we are interested only in the temperature at 1 second intervals, we can sample x at the times of interest as shown in Figure 5.1.

Another example of sampling is shown in Figure 2.5. An original image $x(u, v)$ is shown together with sampled versions of the image. In the sampled versions of the image, the blocks of constant intensity are called *pixels*, and the gray level is constant within the pixel. The gray level value is associated with the intensity at the center of the pixel. But rather than simply showing a small dot in the center of the pixel, the whole pixel is colored with the same gray level for a more natural appearance of the image. The effect of more coarse sampling can be seen in the various images. Actually, the so-called “original” image in Figure 2.5a is also sampled, but the sampling is fine enough that we don’t notice any graininess.

The process of digitizing the range is called *quantization*. In quantizing a signal, the value $x(\cdot)$ is only allowed to take on some discrete set of values (as opposed to the variable t taking on discrete values as in sampling).

Figure 5.11 shows the original temperature signal $x(t)$ (shown previously in Figure 5.1) as well

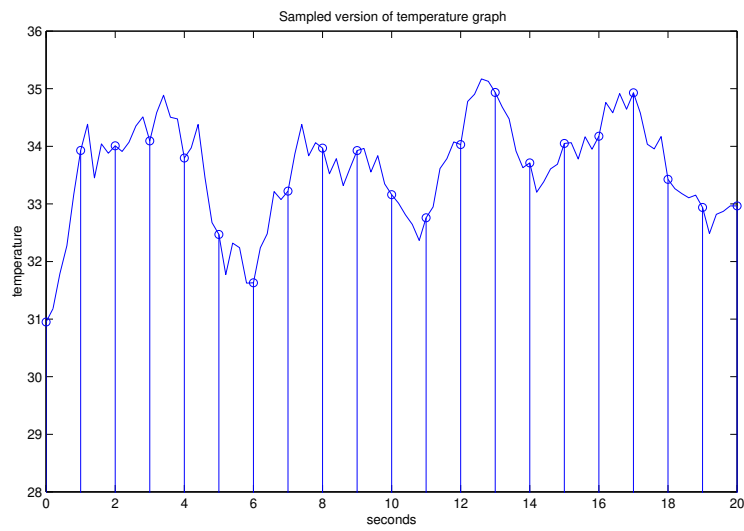


Figure 2.4: Sampling an analog signal.

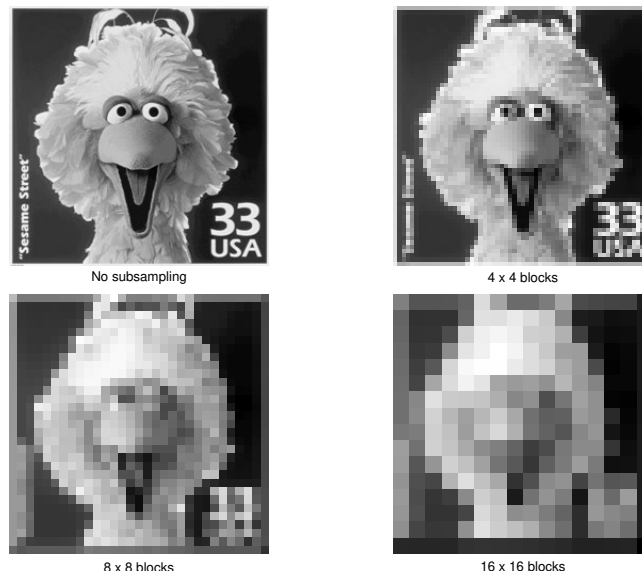


Figure 2.5: Sampling an image.

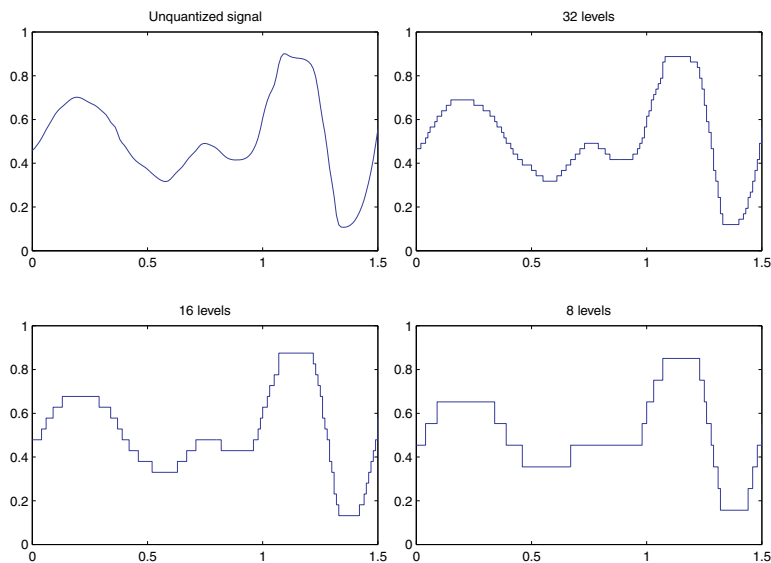


Figure 2.6: Quantized versions of an analog signal.

various quantized versions of x . Figure 5.14 shows the image from Figure 5.15 and various quantized versions. In the quantized versions of the images, the gray levels can take on only some discrete set of values. Actually, the so-called “original” image is also quantized, but because of the resolution of the printer and limitations of the human visual system, a technique known as halftoning (discussed in Chapter 6) can be used so that we don’t notice any artifacts due to quantization. It is typical in images to let the gray level take on 256 integer values with 255 being the brightest gray level and 0 the darkest. In Figures 5.14d-f there are only 8, 4, and 2 gray levels respectively, and quantization artifacts become quite noticeable.

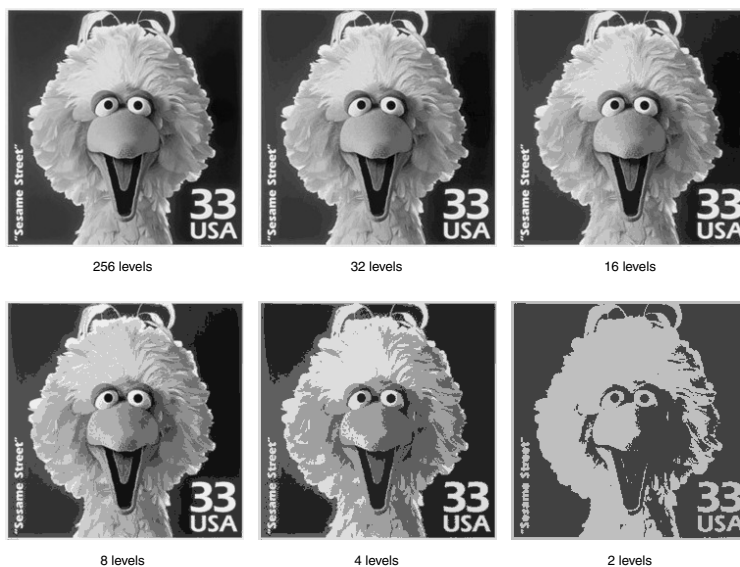


Figure 2.7: Quantized versions of a gray-scale image.

Sampling and quantization to digitize a signal seem to throw away much information about a signal, and one might wonder why this is ever done. The main reason is that digital signals are easy to store and process with digital computers. Digital signals also have certain nice properties in terms of robustness to noise, as we'll discuss in Section 2.4. However, there are also situations in which analog signals are more appropriate. As a result there is often a need for *analog-to-digital conversion* and *digital-to-analog conversion* (also written A/D and D/A conversion). In digitizing signals, one would also like to know how much information is lost by sampling and quantization, and how best to do these operations. The theory for sampling is clean and elegant, while the theory for quantization is more difficult. It turns out that choices for sampling rates and number of quantization levels also depend to a large extent on system and user requirements. For example, in black-and-white images, 256 gray levels is adequate for human viewing – much more than 256 would be overkill, while much less would lead to objectionable artifacts. We defer a more detailed consideration of sampling and quantization until Chapter 5 after we have covered some additional background material.

Before discussing some basic operations on signals, we describe a fairly common notational convention which we will also follow. Continuous-time signals will be denoted using parentheses, such as $x(t)$, while discrete-time signals will use brackets such as $x[n]$. This convention also applies even if the independent variable represents something other than time. That is, $y(u)$ denotes a signal where the domain is continuous, while $y[k]$ indicates a discrete domain, whether or not the independent variables u and k refer to time. Often the letters i, j, k, l, m, n are used to denote a discrete independent variable.

2.3 Some Basic Signal Operations

In addition to the obvious operations of adding or multiplying two signals, and differentiating or integrating a signal, certain other simple operations are quite common in signal processing. We give a brief description of some of these here. The original signal is denoted by $x(t)$.

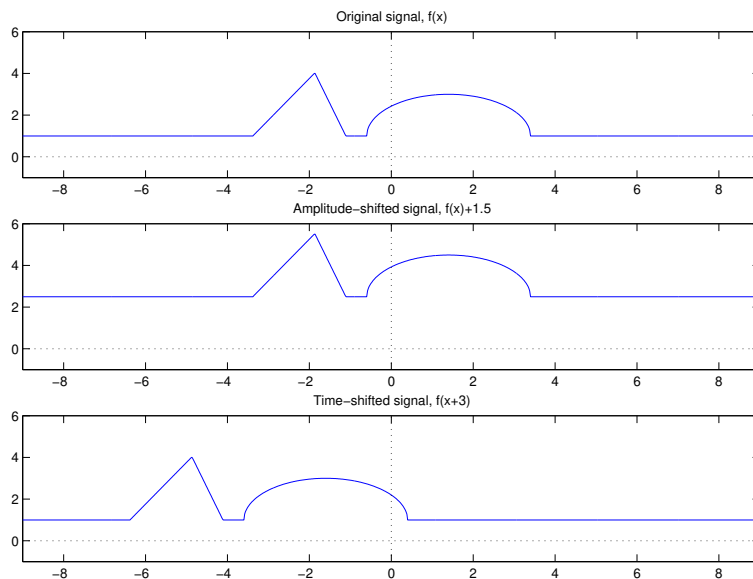


Figure 2.8: Amplitude- and time-shifted versions of a signal.

The signal $a + x(t)$ where a is some number is just adding a constant signal to $x(t)$ and simply shifts the range (or amplitude) of the signal by the amount a . A somewhat different operation is obtained when one shifts the domain of the signal. Namely, the signal $x(t - t_0)$ is a *time-shift* of the original signal $x(t)$ by the amount t_0 . It's like a delayed version of the original signal. Figure 2.8 shows amplitude and time-shifted versions of a signal.

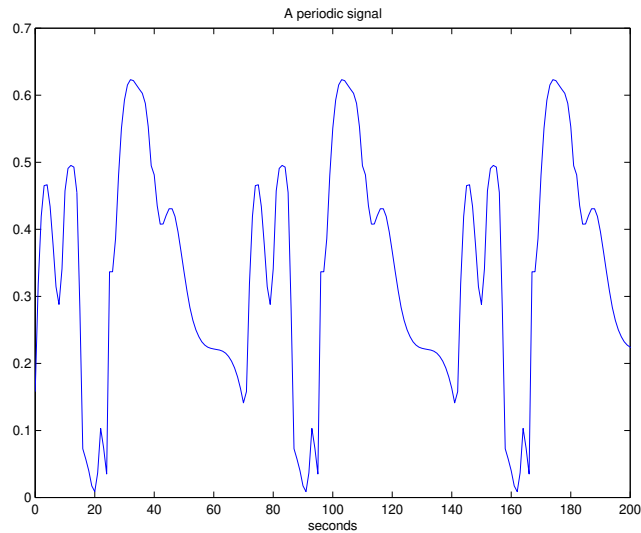


Figure 2.9: A periodic signal.

For some signals, appropriate time shifts can leave the signal unchanged. Formally, a signal is said to be *periodic* with period P if $x(t - P) = x(t)$ for all t . That is, the signal simply repeats itself every P seconds. Figure 2.9 shows an example of a periodic signal.

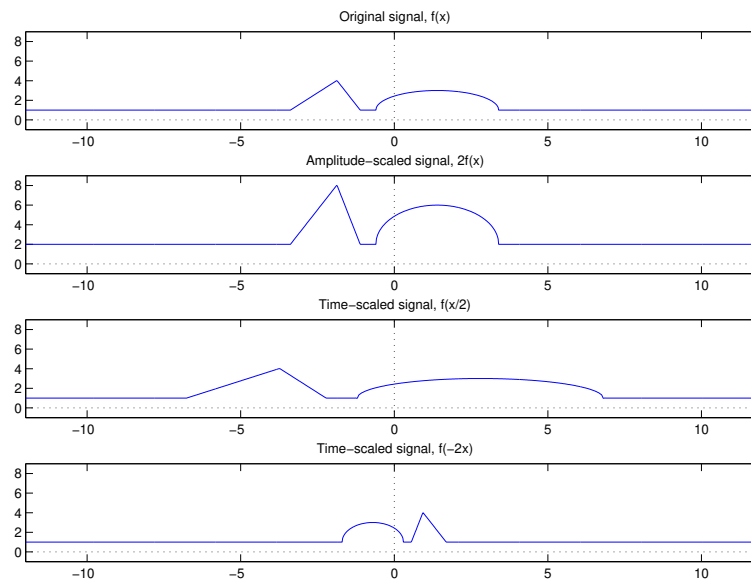


Figure 2.10: Amplitude- and time-scaled versions of a signal.

Amplitude scaling a signal to get $ax(t)$ is simply multiplying $x(t)$ with a constant signal a . However, a rather different operation is obtained when one scales the time domain. Namely, the signal $x(at)$ is like the original signal, but with the time axis compressed or stretched (depending on whether $a > 1$ or $a < 1$). Of course, if $a = 1$ the signal is unchanged. Figure 2.10 shows the effects of amplitude and time scaling. For negative values of a , the signal is “flipped” (or “reflected”) about the range axis, in addition to any compression or stretching. In particular, if $a = -1$, the signal is reflected about the range axis, but there is no stretching or compression. For some functions, the reflection about the range axis leaves the function unchanged, that is, the signal is symmetric about the range axis. Formally, the property required for this is $x(-t) = x(t)$ for all t . Such functions are called *even*. A related notion is that of an *odd* function, for which $x(-t) = -x(t)$. These functions are said to be symmetric about the origin, meaning that they remain unchanged if they are first reflected about the range axis and then reflected about the domain axis. Figure 2.11 shows examples of an even function and an odd function.

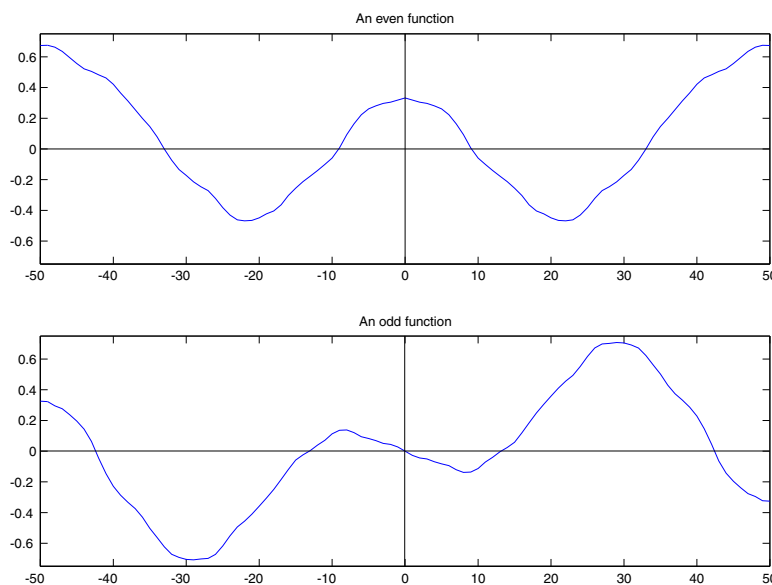


Figure 2.11: Examples of even and odd functions.

The signal $x(y(t))$ is called the *composition* of the two functions $x(\cdot)$ and $y(\cdot)$. For each t , it denotes the operation of taking the value $y(t)$ and evaluating $x(\cdot)$ at the time $y(t)$. Of course, we can get a very different result if we reverse the order and consider $y(x(t))$.

One other operation that is extremely useful is known as *convolution*. We will defer a description of this operation until Section 3.5.

2.4 Noise

In many applications desired signals are subject to various types of degradations. These degradations can arise from a variety of sources such as limitations of the sensing device, random and/or unmodeled fluctuations of underlying physical processes, or environmental conditions during sensing, transmission, reception, or storage of the data. The term *noise* is typically used to describe a wide range of degradations.

It is often useful to try and model certain properties of the noise. One widely used model is to

assume that the original (desired) signal is corrupted by *additive noise*, that is, by adding another unwanted signal. Of course, if we knew the noise signal that was added, we could simply subtract it off to get back the original signal, and the noise would no longer be an issue. Unfortunately, we usually do not have such detailed knowledge of the noise signal. More realistically, we might know (or assume) that the noise satisfies certain properties without knowing the exact values of the noise signal itself. It is very common to model the noise as random, and assume that we know something about the distribution of the noise. For example, we might assume that the noise is randomly (uniformly) distributed over some interval, or that it has a Gaussian (normal) distribution with a known mean and variance. Even this minimal type of knowledge can be extremely useful as we will see later.

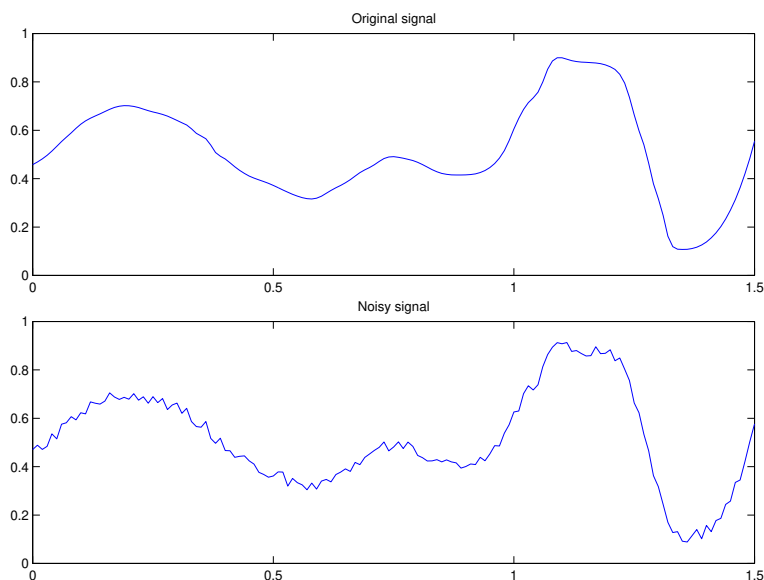


Figure 2.12: Adding noise to an analog signal.

Robustness to effects of noise can be a major design consideration for certain systems. This can be one reason why for many applications a digital system might be preferred over an analog one. Of course, the power of digital computing is also a key reason for the prevalence of digital systems, and robustness to noise is one factor that makes digital computing so reliable. Figures 2.12 and 2.13 illustrate the effect of adding noise on an analog signal and a quantized (although still continuous-time) signal. Without further knowledge of signal and noise characteristics, the noise cannot be removed from an analog signal since any possible value could be a valid value for the signal. On the other hand, if we know the original signal is quantized (so it takes on only a discrete set of values), then depending on the noise level, it may be possible to remove much of the noise by simply re-quantizing the noisy signal. This process simply maps the observed signal values to one of the possible original levels (for example, by selecting the closest level).

2.5 Some Common Signals

Here we briefly define some signals that we will commonly encounter. Perhaps the most basic and frequently used signal is a sinusoid defined by

$$x(t) = A \sin(\omega t)$$

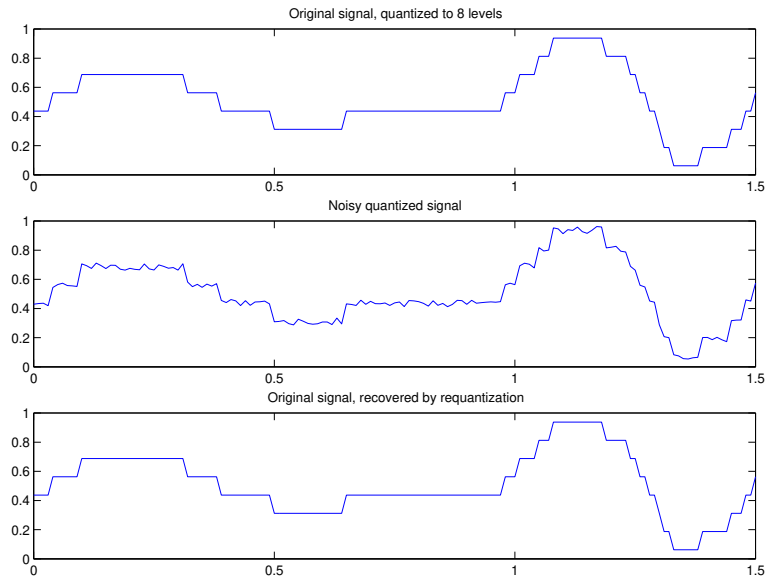
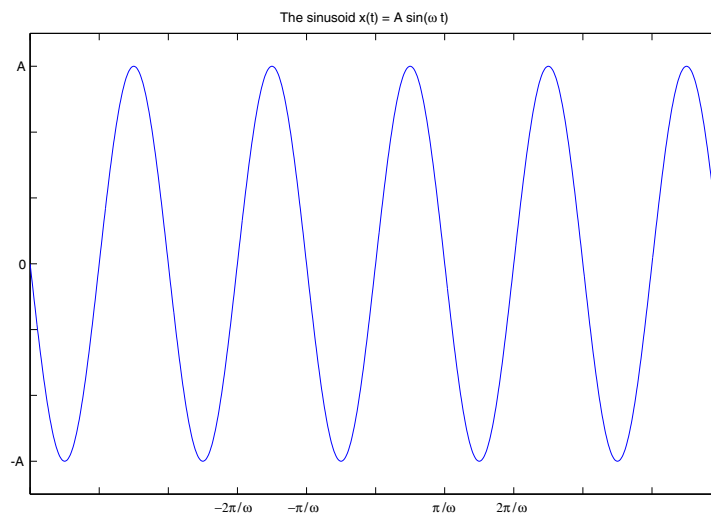


Figure 2.13: Adding noise to a quantized signal.

and shown in Figure 2.14.

Figure 2.14: The sinusoid $x(t) = A \sin(\omega t)$.

Here A is the amplitude, and ω is the radian frequency. The units of ω are radians/sec so that when multiplied by time t (in sec) we get radians. An equivalent form for the sinusoid that is often used is

$$x(t) = A \sin(2\pi f t).$$

The frequency f is in units of *Hertz* (abbreviated *Hz*) which is sec^{-1} , or often called cycles per second. Of course, f and ω are related by $\omega = 2\pi f$. Also, it's clear that since $\sin(\theta + \pi/2) = \cos \theta$,

we could have equivalently written the sinusoid as

$$x(t) = A \cos(2\pi ft - \pi/2).$$

Up to this point, we have only considered real-valued signals. Although physical quantities can generally be represented in terms of real-valued signals, it turns out to be extremely useful to consider signals taking on complex values. The most basic complex-valued signal we will use is the complex exponential $e^{j\omega t}$. (Note that here we have used the symbol j instead of i to denote the imaginary number $\sqrt{-1}$. This is common in electrical engineering since the symbol i has traditionally been used to represent an electrical current.) The well-known Euler identity can be used to write the complex exponential in terms of standard sinusoids. Namely,

$$e^{j\omega t} = \cos(\omega t) + j \sin(\omega t).$$

As with sinusoids, the complex exponential can also be written in terms of frequency in Hertz rather than radian frequency.

There are some basic signals commonly used in signal processing and that are given special notation. One is the step function (also called the Heaviside step function or the unit step function), often denoted as $u(t)$. Some others are the ramp function, rectangular function (or rectangular pulse), triangle function/pulse, and the sinc function (pronounced like “sink”). These signals are defined as follows:

$$\begin{aligned} u(t) &= \begin{cases} 0 & \text{if } t < 0 \\ 1 & \text{if } t \geq 0 \end{cases} \\ \text{ramp}(t) &= \begin{cases} 0 & \text{if } t < 0 \\ t & \text{if } t \geq 0 \end{cases} \\ \text{rect}(t) &= \begin{cases} 1 & \text{if } -1/2 \leq t < 1/2 \\ 0 & \text{otherwise} \end{cases} \\ \text{tri}(t) &= \begin{cases} 1 - |t| & \text{if } -1 \leq t \leq 1 \\ 0 & \text{otherwise} \end{cases} \end{aligned}$$

and

$$\text{sinc}(t) = \frac{\sin(\pi t)}{\pi t}$$

and are shown in Figure 2.15.

2.6 Delta Functions

The notion of a delta function is extremely useful in the analysis of signals and systems, although it may feel unnatural on first exposure. Although the concept of the delta function can be made completely rigorous, rather than get side-tracked with too much mathematical detail and sophistication, our aim here is to provide some intuition and ability to work with the delta function. On the other hand, it is important to have enough rigor so that this important tool is used properly.

The delta function in continuous-time is also called the Dirac delta function, unit impulse function, or sometimes just the impulse function. It is defined implicitly through its behavior under integration as follows:

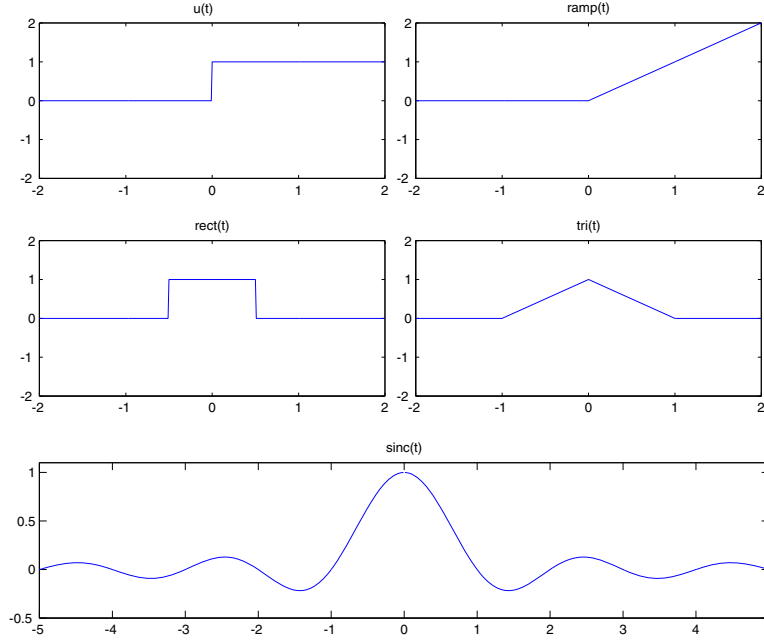


Figure 2.15: (a) Step function. (b) Ramp function. (c) Rectangle function. (d) Triangle function. (e) Sinc function.

Definition: $\delta(t)$ is the Dirac delta function if it satisfies

$$\int_{-\infty}^{\infty} x(t)\delta(t) dt = x(0) \quad (2.1)$$

for every function $x(t)$ that is continuous at $t = 0$.

From this definition we can infer the following two properties of the delta function. First, by considering the function $x(t) = 1$ in Equation (2.1), we get

$$\int_{-\infty}^{\infty} \delta(t) dt = 1 \quad (2.2)$$

This result implies that the area under the delta function is equal to 1.

The second property gives the value of $\delta(t)$ for $t \neq 0$. Suppose $\delta(t)$ took on positive values in even a very small interval away from $t = 0$. Then we could choose a function $x(t)$ that also took positive values inside a portion of this same interval, but with $x(t) = 0$ elsewhere (including $t = 0$) and with $x(t)$ continuous at $t = 0$. However, in this case the left hand side of Equation (2.1) must be positive, but the right hand side is 0. Therefore, $\delta(t)$ cannot take on positive values in any interval. A similar argument leads us to the conclusion that $\delta(t)$ cannot take on negative values in any interval. Thus, $\delta(t) = 0$ for $t \neq 0$.

These two results (namely, that the area under $\delta(t)$ is 1 and that $\delta(t) = 0$ for all $t \neq 0$) are inconsistent with our usual notions of functions and integration. If $\delta(0)$ was any finite value, then the area under $\delta(t)$ would be zero. Strictly speaking, $\delta(0)$ is undefined although it is convenient to think of $\delta(0) = \infty$. Thus, although we call $\delta(t)$ the delta “function,” it is technically not a function in the usual sense. It is what is known as a distribution. However, it turns out that for many manipulations we can treat $\delta(t)$ like a function.

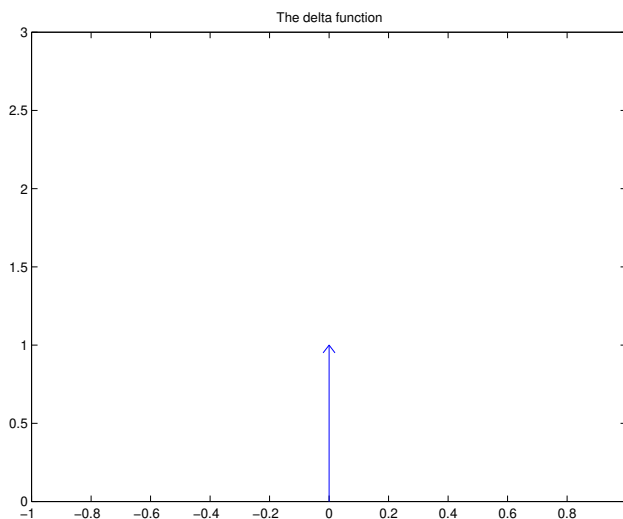


Figure 2.16: Representation of a delta function.

It is also convenient to have a graphical representation as shown in Figure 2.16. The arrow indicates that the value at $t = 0$ is infinite (or undefined), with the height of the arrow indicating the area under $\delta(t)$. To depict $A\delta(t)$ where A is some constant, we would draw the height of the arrow to be A .

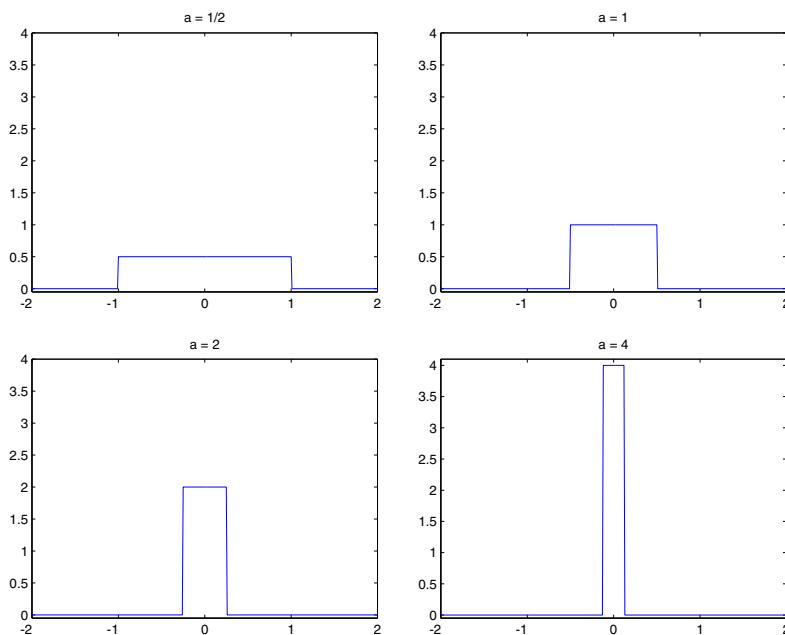


Figure 2.17: The delta function is a limit of rectangle functions with area 1.

It is sometimes also helpful to think of $\delta(t)$ as a limit of a sequence of approximating functions. Consider the function $a \operatorname{rect}(at)$. This has area 1, but if $a > 1$ it is more concentrated around $t = 0$. As we let $a \rightarrow \infty$ we get a sequence of approximations as shown in Figure 2.17, which intuitively

get closer and closer to $\delta(t)$. In fact, it is not hard to verify that for $x(t)$ continuous at $t = 0$ we have

$$\int_{-\infty}^{\infty} x(t) a \operatorname{rect}(at) dt \rightarrow x(0) \quad \text{as } a \rightarrow \infty$$

so that in the limit $a \rightarrow \infty$ the defining property of $\delta(t)$ is indeed satisfied. It turns out that many other choices for the approximating functions will also work if the area is 1 and scaling is done to get concentration at $t = 0$.

The primary motivation for introducing the delta function is that it is a useful tool in analyzing systems and signals as we'll see in Chapter 2. There is also a physical motivation involving modeling of physical phenomena that is actually closely related to the analytical motivation. In some situations we would like to model physical phenomena that occur in a time interval short compared to the resolution of a measuring device. For example, we may be interested in the energy entering a camera due to a flash of light as the camera shutter opens and closes very quickly. For a fixed shutter speed, many models for the light source will be good enough to represent a “flash” or “impulse” of light. However, if we fix a particular function to model the flash of light, it may not represent a true “flash” for faster shutter speeds. Modeling the flash of light as a delta function is an idealization that works for *any* (non-infinite) shutter speed.

A number of properties of $\delta(t)$ can be obtained directly from the definition by utilizing the usual rules of integration. Intuition can also sometimes be gained by considering approximations of $\delta(t)$, although a formal justification generally requires verifying the defining property.

For example, since $\delta(t) = 0$ for all $t \neq 0$, it seems obvious that $\delta(t)$ is even. However, since $\delta(t)$ is technically not a function, we should really verify directly that $\delta(-t)$ behaves just as $\delta(t)$ in the defining property of Equation (2.1). This can be done by a simple change of variable as follows. For any function $x(t)$ continuous at $t = 0$, we have

$$\int_{-\infty}^{\infty} x(t) \delta(-t) dt = \int_{-\infty}^{\infty} x(-\tau) \delta(\tau) d\tau = x(-\tau) \big|_{\tau=0} = x(0)$$

where the first equality is obtained by the change of variable $\tau = -t$, and the second equality follows from the definition of $\delta(\tau)$. The conclusion is that $\delta(-t)$ satisfies the required property of $\delta(t)$, and so $\delta(-t) = \delta(t)$.

By the change of variable $\tau = at$ and considering the cases $a > 0$ and $a < 0$ separately, it is easy to show that

$$\delta(at) = \frac{1}{|a|} \delta(t).$$

By the change of variable $\tau = t - t_0$, it follows that

$$\int_{-\infty}^{\infty} x(t) \delta(t - t_0) dt = x(t_0).$$

Therefore, the time-shifted delta function $\delta(t - t_0)$ behaves like we would expect. This property is sometimes called the sifting property of the delta function. The natural graphical depiction of $\delta(t - t_0)$ is shown in Figure 2.18.

We now turn to the discrete-time delta function, also called the Kronecker delta function. The Kronecker delta function is denoted by $\delta[n]$ and defined as

$$\delta[n] = \begin{cases} 1 & \text{if } n = 0 \\ 0 & \text{otherwise} \end{cases}$$

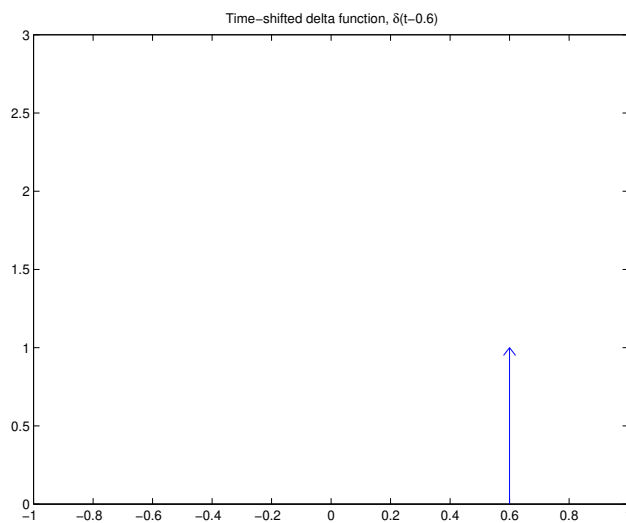


Figure 2.18: Time-shifted delta function.

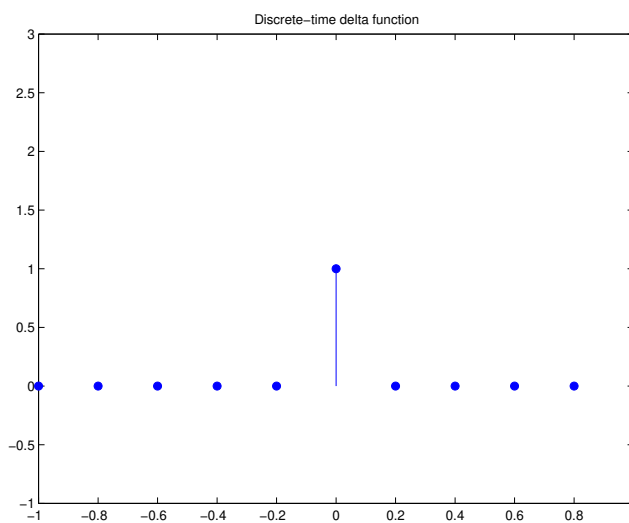


Figure 2.19: Discrete-time delta function.

Figure 2.19 shows the graph of $\delta[n]$. Hence, in discrete-time, the delta function is in fact a function in the proper sense. There are none of the mathematical subtleties/difficulties associated with the continuous-time delta function. In fact, $\delta[n]$ is rather simple to work with.

Many properties of $\delta(t)$ have analogous counterparts in discrete-time, and the discrete-time properties are generally easier to verify. For example, the result

$$\sum_{n=-\infty}^{\infty} x[n]\delta[n] = x[0]$$

follows trivially from the definition. Recall that in continuous time, the analogous property was actually the definition. Also trivial is the fact that $\delta[n]$ is an even function of $n = \dots, -1, 0, 1, \dots$. It is easy to see that the time-shifted delta function $\delta[n - n_0]$ satisfies the discrete-time sifting property

$$\sum_{n=-\infty}^{\infty} x[n]\delta[n - n_0] = x[n_0].$$

It turns out that for some properties the discrete-time counterpart is *not* analogous. For example, in discrete-time if a is an integer and $a \neq 0$, then we have $\delta[an] = \delta[n]$.

2.7 Separable Signals in 2-D

One useful notion that arises in two (and higher) dimensions is separability. A function $x(u, v)$ is called *separable* if it can be written as $x(u, v) = x_1(u)x_2(v)$. Many of the commonly encountered 2-D functions are simply separable extensions of the corresponding 1-D functions. For example, the 2-D version of the complex exponential is

$$e^{j(\omega_1 u + \omega_2 v)} = e^{j\omega_1 u} e^{j\omega_2 v}$$

where ω_1 and ω_2 are the radian frequencies in the u and v directions, respectively. That is, the 2-D complex exponential is simply the product of a 1-D complex exponential in each direction.

Likewise, the 2-D Dirac delta function $\delta(u, v)$ is given by

$$\delta(u, v) = \delta(u)\delta(v)$$

Formally, $\delta(u, v)$ would actually be defined by the property

$$\int_{-\infty}^{\infty} \int_{-\infty}^{\infty} x(u, v)\delta(u, v) du dv = x(0, 0)$$

for any function $x(u, v)$ continuous at $(0, 0)$, but showing equivalence with the separable expression is straightforward.

Similarly, in the discrete case, the Kronecker delta $\delta[m, n]$ is defined by

$$\delta[m, n] = \begin{cases} 1 & \text{if } m = n = 0 \\ 0 & \text{otherwise} \end{cases}$$

Clearly the Kronecker delta is also separable

$$\delta[m, n] = \delta[m]\delta[n].$$

Many other 2-D functions like $\text{rect}(u, v)$ and $\text{sinc}(u, v)$ have natural separable extensions from the 1-D versions as well.

Separability can offer some nice analytical and computational advantages. For example, terms involving the two independent variables can sometimes be separated, reducing a two-dimensional analysis to two separate one-dimensional analyses. This can result in computational savings by allowing processing to be done along the two dimensions separately.

Chapter 3

Basics of Systems

3.1 What are Systems?

As previously mentioned, a signal represents some underlying physical variable of interest. As an abstraction, we consider a signal as simply a real-valued (or sometimes complex-valued) function defined on some domain. For example, we might represent a signal as just a real-valued function of time $x(\cdot)$ where $x(t)$ represents the value of the signal at time t . Similarly, we will consider an abstraction of the general system of Figure 3.1 from Chapter 1.

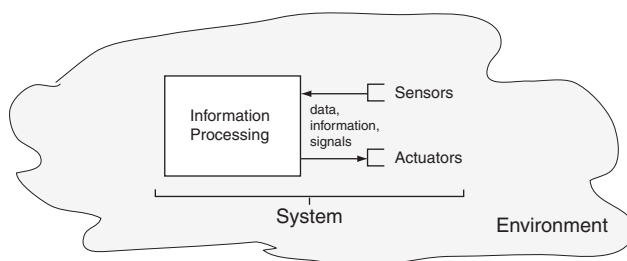


Figure 3.1: A general system immersed in its environment.

In this figure, we think of a system as some part of the physical world that interacts with its environment and is designed to carry out some task. The box labeled “information processing” receives input signals from various sensors and produces output signals for various actuators. Thus, we think of the system as transforming input signals into output signals. A number of subtasks within “information processing” themselves perform operations on a signal to produce another signal — i.e., they transform one function $x(t)$ to produce a new function $y(t)$. The term *system* is used in this abstract and technical sense to refer to such mappings that take a signal as input and produce another signal as output. As we’ll see, by making this abstraction and imposing additional assumptions, we’ll be able to study special types of systems in a precise way that leads to useful insights and results.

As with signals, we can have various types of systems such as continuous-time, discrete-time, analog, digital, etc. We will let $x(t)$ or $x[n]$ denote the input signal for continuous-time and discrete-time systems, respectively, and let $y(t)$ and $y[n]$ denote the corresponding the output

*©2023 by Sanjeev R. Kulkarni. All rights reserved. Please do not distribute without permission of the author.

[†]Lecture Notes for ECE201 Information Signals.

[‡]Thanks to Rich Radke and Sean McCormick for producing the figures.

signals. $\mathcal{H} = \mathcal{H}[\cdot]$ will denote the system, so that

$$y(t) = \mathcal{H}[x(t)]$$

and similarly in the discrete-time case with “ (t) ” replaced by “ $[n]$ ”. It is standard to represent this relationship for the system $\mathcal{H}$ with a schematic diagram as shown in Figure 3.2.

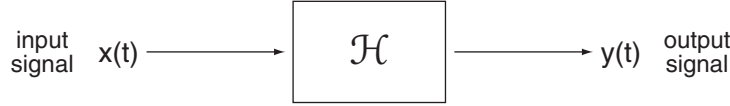


Figure 3.2: Abstraction of a general system.

The range of possible systems is so wide that it is difficult to say much that applies to completely general systems. However, by restricting attention to special classes of systems a set of tools can be developed that helps in the modeling, analysis, and design of systems. Two particularly useful properties are linearity and time-invariance, which we describe next. A rich theory has been developed for this class of systems. Although real physical systems are rarely exactly linear or time-invariant, results for the idealized assumptions still provide useful models for understanding more complex systems.

3.2 Linearity

One extremely useful class of systems is the class of linear systems. Roughly, a system is linear if adding and/or scaling inputs to the system simply results in adding or scaling the corresponding outputs. More precisely, a system $\mathcal{H}$ is *linear* if the following condition is satisfied for all constants a_1, a_2 and all input signals $x_1(t), x_2(t)$:

$$\mathcal{H}[a_1x_1(t) + a_2x_2(t)] = a_1\mathcal{H}[x_1(t)] + a_2\mathcal{H}[x_2(t)] \quad (3.1)$$

This property is often referred to as *superposition*. The definition of linearity can also be written for discrete-time systems in the obvious way.

We can split the superposition condition into the following two separate conditions:

$$\mathcal{H}[ax(t)] = a\mathcal{H}[x(t)] \quad (3.2)$$

for all constants a and input signals $x(t)$; and

$$\mathcal{H}[x_1(t) + x_2(t)] = \mathcal{H}[x_1(t)] + \mathcal{H}[x_2(t)] \quad (3.3)$$

for all input signals $x_1(t), x_2(t)$.

The first condition implies, for example, that once we know the output of the system due to $x(t)$, then the output due to $2x(t)$ will be twice the original output. The second condition implies that to find the output due to the sum of two signals $x_1(t) + x_2(t)$, we can first find the outputs due to each signal separately and then just sum these outputs. The two conditions (3.2) and (3.3) are equivalent to the single superposition condition (3.1). It might seem that the scaling condition (3.2) is implied by additive condition (3.3). If the scaling constants are restricted to be real numbers, then (3.3) implies (3.2) for rational values of a , and if we assume regularity then this implies (3.2) for all a . However, if we allow complex a , then (3.3) does not imply (3.2). And even for real-valued a , there are systems that satisfy (3.2) but not (3.3).

The beauty of linear systems arises from the superposition property, which lets us analyze outputs due to very complicated inputs by first just considering some standard set of simple inputs, and then considering linear combinations of these.

3.3 Time-Invariance

Time-invariance is another useful concept to consider in studying systems. Time-invariance means that whatever behavior a system has, if the inputs were delayed by some amount of time, then the behavior of the system would be unchanged other than simply delaying the output by that same amount of time. More precisely, suppose $y(t)$ denotes the output of the system due to $x(t)$, so that as before $y(t) = \mathcal{H}[x(t)]$. Then $\mathcal{H}$ is *time-invariant* if

$$\mathcal{H}[x(t - t_0)] = y(t - t_0) \quad (3.4)$$

for all t_0 and input signals $x(t)$. If a system is not time-invariant then it is said to be *time-varying*.

In this condition, t_0 denotes the delay, so that $x(t - t_0)$ is the signal $x(t)$ delayed by time t_0 . Note that t_0 is allowed to be negative, so that the condition also applies to starting the input early as well as delaying it.

The left hand side of (3.4) denotes the output of the system if we apply the delayed input $x(t - t_0)$. The right hand side is the output $y(t)$ delayed by time t_0 . That is, the left hand side first delays the input to get $x(t - t_0)$ and then applies this to the system, while the right hand side first puts $x(t)$ into the system and then delays the resulting output. Time-invariance requires these to give the same final result for all $x(\cdot)$ and all t_0 .

The notion of time-invariance can be defined in a similar way for discrete-time systems. Of course, in this case, since the time variable n only takes on integer values, we only need to consider integer shifts in the equation for time-invariance.

Like linearity, the power of time-invariance lies in the nice behavioral properties this condition imposes on the system. It allows us to use knowledge of the system behavior on certain inputs to determine what the system would do on other (namely shifted) inputs. The two conditions together (linearity and time-invariance) are particularly powerful, as we'll see in the next few sections. In fact, with these two conditions the behavior of a system to *any* input can be determined once we know the behavior due to just one special input – namely the delta function.

3.4 Impulse Response and Output of LTI Systems

We now focus on the class of linear time-invariant systems (also known as LTI systems). With these two properties (i.e., linearity and time-invariance) a rich theory can be developed. Here we will touch on just some of the basic tools for describing and analyzing such systems, but these tools form the basis for much of the field of signals and systems.

We will first consider the case of discrete-time systems. Suppose we apply the discrete-time delta function as the input to the system, namely we let $x[n] = \delta[n]$. The output will be some signal which we will denote as $h[n]$, so that

$$h[n] = \mathcal{H}[\delta[n]]$$

The function $h[n]$ is called the *impulse response* of the system. It is the output (response) of the system when the input is a delta function (impulse).

For the moment, let's not be concerned with what particular values we observe for the impulse response, but only that we observe some output that we will record and manipulate in various

ways. Our focus is on seeing whether we can use our knowledge that the output is $h[n]$ when the input is $\delta[n]$ to predict what output we will get for some other inputs.

Without some restrictions or assumptions on the system, we have no hope of predicting the output on some new inputs. However, recall that we have assumed that the system is LTI, which is a powerful condition indeed, as we shall now see.

First, since the system is time-invariant, we can easily determine the input to the shifted delta function $\delta[n - n_0]$. This input is just a delayed version of the delta function $\delta[n]$ that takes the value 1 at time $n = n_0$ and is 0 elsewhere. Time-invariance tells us that the output due to this delayed delta function is just $h[n - n_0]$, namely the output due to $\delta[n]$ but just shifted by the same amount. In equation form,

$$\mathcal{H}[\delta[n - n_0]] = h[n - n_0]$$

Now, we can use linearity to determine the output due to many other inputs. For example, what is the output due to applying $2\delta[n]$? Well, we already know that the output $h[n]$ results from the input $\delta[n]$. So by the scaling property of linearity, we must get the output $2h[n]$ when we apply $2\delta[n]$.

What about the output due to $\delta[n - 1]$? Time-invariance tells us that the the output must be the same as the output due to $\delta[n]$ but just shifted by one unit. I.e., the output is $h[n - 1]$. Likewise, if we apply $\delta[n] + \delta[n - 1]$, the additive property of linearity together with time-invariance tells us that the output must be $h[n] + h[n - 1]$.

Now, we can use both the scaling and additive properties of linearity together with time-invariance, to determine that if we apply the input $2\delta[n] + 3\delta[n - 1]$, the output will be $2h[n] + 3h[n - 1]$. Thus, by just knowing the output due to $\delta[n]$ and using linearity and time-invariance, we can determine the output due to some pretty complicated inputs by just “building up” these inputs in terms of shifted and scaled delta functions, and then combining the correspondingly shifted and scaled versions of the impulse response.

In fact, by carrying this idea further, we see that by appropriate combinations of scaled and shifted delta functions, we can in fact synthesize *any* desired input function. Suppose we want to create an arbitrary input $x[n]$. We can start by making sure we create the proper value at time 0, namely $x[0]$. To do this we include $\delta[n]$ scaled by $x[0]$, i.e., $x[0]\delta[n]$. Next, to get the desired value at time 1, $x[1]$, we simply add $x[1]\delta[n - 1]$, which is $\delta[n - 1]$ scaled by $x[1]$. Then to get the desired value at time -1 , we add $x[-1]\delta[n + 1]$. Notice that each shifted delta function is non-zero only at one time, so that adding new terms won’t ruin the values we’ve already created. Thus, continuing in this manner, we can write the input signal $x[n]$ as

$$\begin{aligned} x[n] = & \cdots + x[-2]\delta[n + 2] + x[-1]\delta[n + 1] + x[0]\delta[n] \\ & + x[1]\delta[n - 1] + x[2]\delta[n - 2] + \cdots \end{aligned}$$

or using summation notation

$$x[n] = \sum_{i=-\infty}^{\infty} x[i]\delta[n - i] \quad (3.5)$$

This is actually just a version of the sifting property of the delta function that we encountered in Section 2.6.

Although expressing $x[n]$ in this way might seem degenerate or not particularly useful, we have actually achieved a great deal. Initially, we had no idea what output we would get if the input $x[n]$ was applied to the system. But now, we can use linearity and time-invariance as we did before to see that the output will just be a combination of a bunch (albeit, potentially an infinite number) of shifted and scaled versions of the impulse response $h[n]$. The fact that the scaling terms are the

values of the input signal itself shouldn't bother or surprise us. On the contrary, that is exactly what we did above in finding the output due to $2\delta[n] + 3\delta[n-1]$, the only difference being that we used specific numbers (2 and 3) for values of the input signal. In fact, the output of the system generally *should* depend on the input signal, otherwise it would be a dull system.

Formally carrying out the manipulations, we get that the output $y[n]$ due to the input $x[n]$ is given by

$$\begin{aligned} y[n] &= \mathcal{H}[x[n]] \\ &= \mathcal{H}\left[\sum_{i=-\infty}^{\infty} x[i]\delta[n-i]\right] \\ &= \sum_{i=-\infty}^{\infty} x[i]\mathcal{H}[\delta[n-i]] \\ &= \sum_{i=-\infty}^{\infty} x[i]h[n-i] \end{aligned}$$

The first equality is the definition of $y[n]$. The second uses the expression (3.5) for $x[n]$ obtained above, which is really the sifting property of the delta function. The third equality follows from linearity, where we have used both the additive property to bring $\mathcal{H}$ inside the sum, and the scaling property to bring $x[i]$ out of $\mathcal{H}[\cdot]$. The fourth equality follows from time-invariance and the definition of the impulse response $h[n]$.

This final result is important enough to repeat as a separate equation. Namely, if $h[n]$ is the impulse response of a system, and we apply an input $x[n]$ then the output $y[n]$ is given by

$$y[n] = \sum_{i=-\infty}^{\infty} x[i]h[n-i] \quad (3.6)$$

A few comments on this important result are in order. The somewhat complicated/convoluted operation on the right hand side of Equation (3.6) combining $x[n]$ and $h[n]$ is appropriately called the *convolution* of $x[n]$ and $h[n]$. It arises so often that it is given the special notation $(x * h)[n]$. This operation will be discussed in more detail in the next section.

Although the expression for the convolution looks complicated, the remarkable thing is that this result implies that once we know the impulse response of a system (i.e., the output due to the single input $\delta[n]$), then we can find the output due to *any* input! In fact, the output is obtained by just convolving the input with the impulse response. We see that linearity and time-invariance impose so much structure on the system that knowing how the system behaves for just the one special input $\delta[n]$ determines the behavior of the system for all inputs. Thus, as far as just the input/output behavior of a system is concerned, the system is completely described by its impulse response. In principle, if we have a black box and know nothing about its operation except that it happens to be linear and time-invariant, we can find out everything about its behavior by doing one simple experiment. We apply the delta function $\delta[n]$ to the input and measure the output signal that results. We then know exactly how the black box will work under other inputs. Of course, in reality things aren't quite so simple for a variety of reasons, including the fact that most real systems are neither exactly linear nor time-invariant. Nevertheless, the result is extremely useful as we'll see on many occasions.

Finally, we should mention that a result analogous to the discrete-time case can also be obtained for continuous-time systems, as long as the continuous-time system satisfies certain mild continuity

conditions that in practical cases are always satisfied. In particular, we let $h(t)$ denote the impulse response of the system. That is, $h(t)$ is the output of the system due to the input $\delta(t)$, so that

$$h(t) = \mathcal{H}[\delta(t)]$$

Then (under suitable continuity conditions) the output $y(t)$ due to *any* input $x(t)$ is given by

$$y(t) = \int_{-\infty}^{\infty} x(\tau) h(t - \tau) d\tau \quad (3.7)$$

A rough derivation of this result follows very similar lines as in the discrete-time case. First, by the sifting property of $\delta(t)$, we can write

$$x(t) = \int_{-\infty}^{\infty} x(\tau) \delta(t - \tau) d\tau$$

Then, using linearity and time-invariance we get

$$\begin{aligned} y(t) &= \mathcal{H}[x(t)] \\ &= \mathcal{H}\left[\int_{-\infty}^{\infty} x(\tau) \delta(t - \tau) d\tau\right] \\ &= \int_{-\infty}^{\infty} x(\tau) \mathcal{H}[\delta(t - \tau)] d\tau \\ &= \int_{-\infty}^{\infty} x(\tau) h(t - \tau) d\tau \end{aligned}$$

The third equality uses linearity, and this is where the continuity conditions are needed in a rigorous derivation. The last equality uses time-invariance.

As in the discrete-time case, the right-hand-side of Equation (3.7) is called the (continuous-time) *convolution* of $x(t)$ and $h(t)$, and denoted $(x * h)(t)$. The result again implies that once we know the impulse response, we can determine the output of the system due to any input via a convolution. As a practical matter, finding the impulse response of a continuous-time black-box is more challenging since we are generally unable to produce a continuous-time delta function. However, approximations to the delta function can be produced and are useful, and in any case the result is extremely useful as a modeling and analysis tool.

3.5 Convolution

In the previous section, we introduced the notion of the convolution of two signals. We repeat the definition here for convenience. In discrete-time, the convolution of $x[n]$ and $h[n]$ is defined by

$$(x * h)[n] = \sum_{i=-\infty}^{\infty} x[i] h[n - i].$$

In continuous-time, the convolution $x(t)$ and $h(t)$ is defined by

$$(x * h)(t) = \int_{-\infty}^{\infty} x(\tau) h(t - \tau) d\tau.$$

We have seen that the convolution of two signals arose naturally as a way to understand the behavior of LTI systems. Specifically, the output of an LTI system is simply the input convolved with the impulse response. However, in addition to this interpretation, the convolution operation can be motivated by simply thinking of it as a rather general and useful way to transform one signal into another signal. If $y(t) = (x * h)(t)$, we think of $x(t)$ as the original signal, $y(t)$ as the new signal, and $h(t)$ as representing the particular type of transformation we wish to perform. With this perspective, it is immaterial that the convolution operation can be viewed as the output of an LTI system. In any case, regardless of the particular view or motivation of convolution, it is an operation that occurs so frequently in both the theory and practice of signals and systems that it is well worth spending time to understand the convolution operation.

Let's consider the discrete-time case first. Let

$$y[n] = (x * h)[n] = \sum_{i=-\infty}^{\infty} x[i]h[n-i].$$

First notice that the convolution of two signals results in another signal, not just a single number. Thus to understand what the convolution does, we need to understand what it does at each time. Notice that for any fixed n , the expression on the right hand side is the sum of a bunch of (infinitely many) terms. Each term is the value of $x[\cdot]$ at some time and the value of $h[\cdot]$ at some other time. Thus the value of $y[n]$ at some time n is really just a weighted sum of the signal $x[\cdot]$ where the weights depend on the signal $h[\cdot]$. At some different time, the convolution is still just a weighted sum where again the weights depend on $h[\cdot]$ but now different weights are assigned to values of $x[\cdot]$.

Notice that part of the expression for the convolution sum, namely the $x[i]$ term, does not depend on n . The summation index i is just a dummy variable to carry out the weighted sum. So we can think of the convolution as follows (and as illustrated in Figure 3.3):

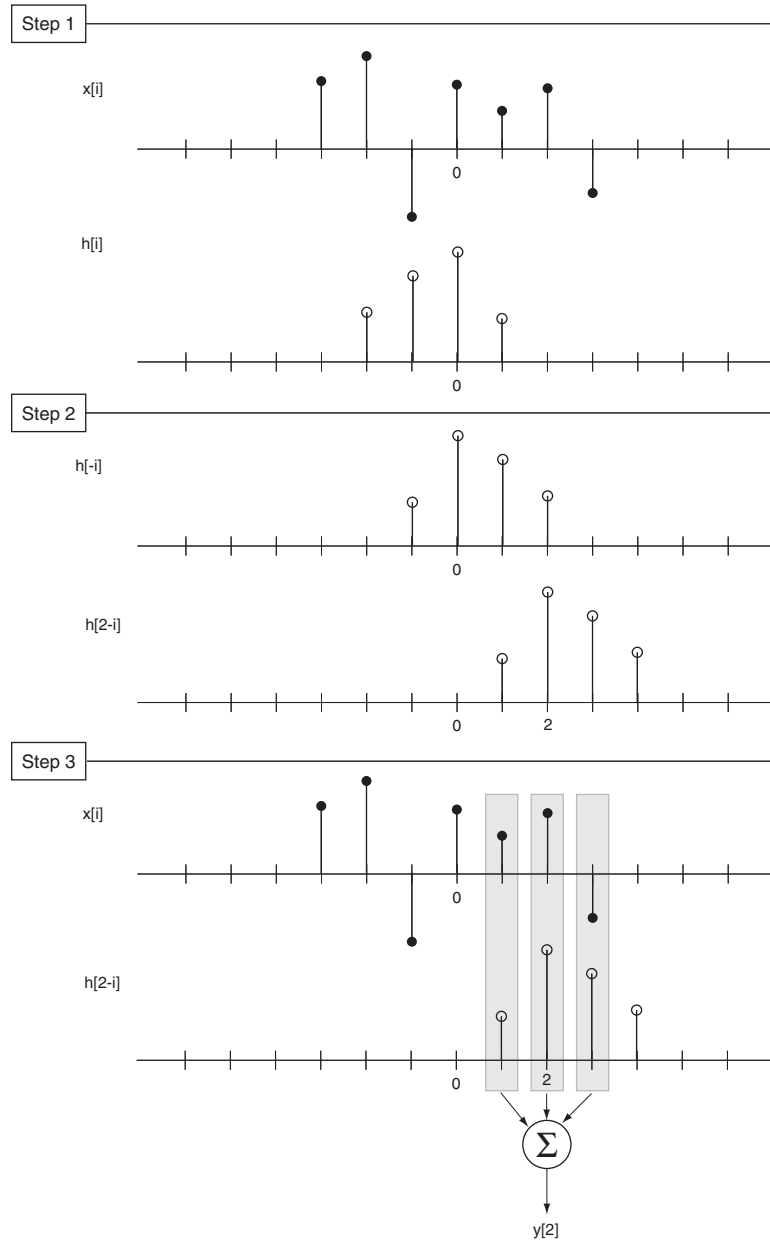
1. Draw the signal $x[\cdot]$ where we use the dummy variable i , since we will be taking a weighted sum.
2. Flip the signal $h[i]$ about the vertical (range) axis to get $h[-i]$ and then shift by n to get $h[n-i]$.
3. Multiply the corresponding values of $x[i]$ and $h[n-i]$ and add up to find $y[n]$.

In continuous-time, the convolution

$$y(t) = (x * h)(t) = \int_{-\infty}^{\infty} x(\tau)h(t-\tau) d\tau.$$

has a similar interpretation, but with the sum replaced by an integral. Namely, the convolution at time t is obtained as follows:

1. Consider $x(\tau)$ where we now use τ as the dummy variable for the weighted integral.
2. Flip the signal $h(\tau)$ about the vertical (range) axis to get $h(-\tau)$ and then shift by t to get $h(t-\tau)$.
3. Multiply the corresponding values of $x(\tau)$ and $h(t-\tau)$ and integrate to find $y(t)$.

Figure 3.3: Schematic convolution of $x[n]$ with $h[n]$

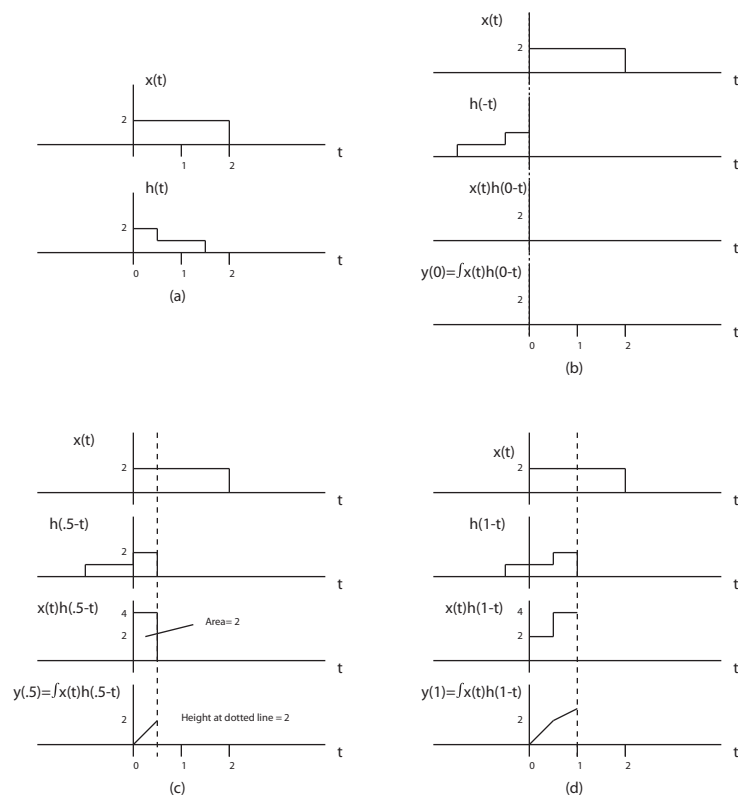


Figure 3.4: (a.) The signal, $x(t)$ top and system response $h(t)$ bottom. (b.) $x(t)$ top. $h(t)$ flipped about vertical axis middle top. Product of $x(\tau)$ and $h(t-\tau)$ middle bottom. Integral of product bottom. (c. - h.) Same as (b.) but further along in time.

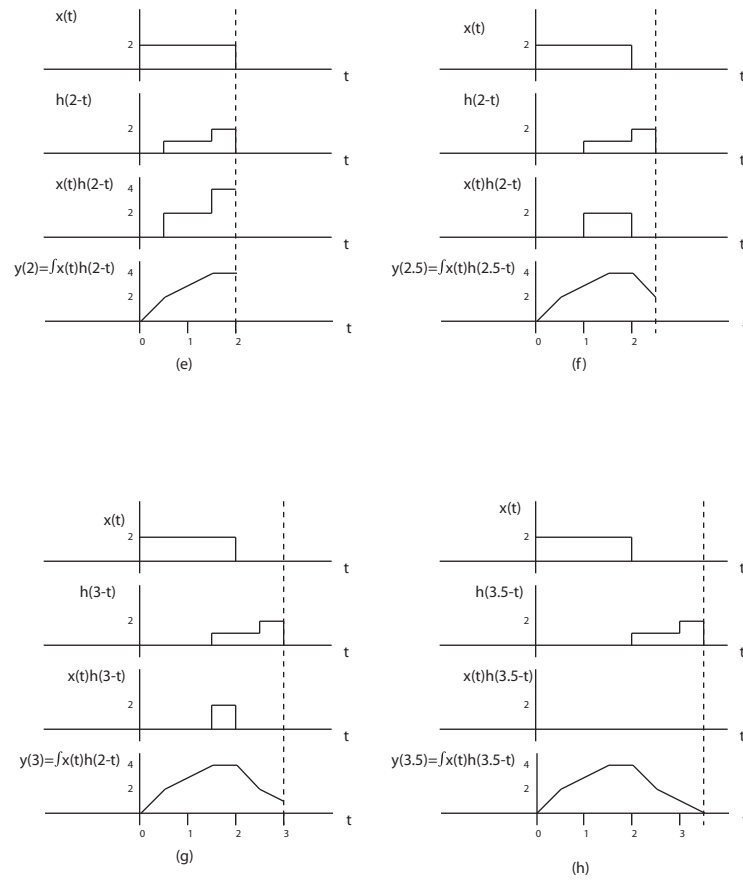


Figure 3.5:

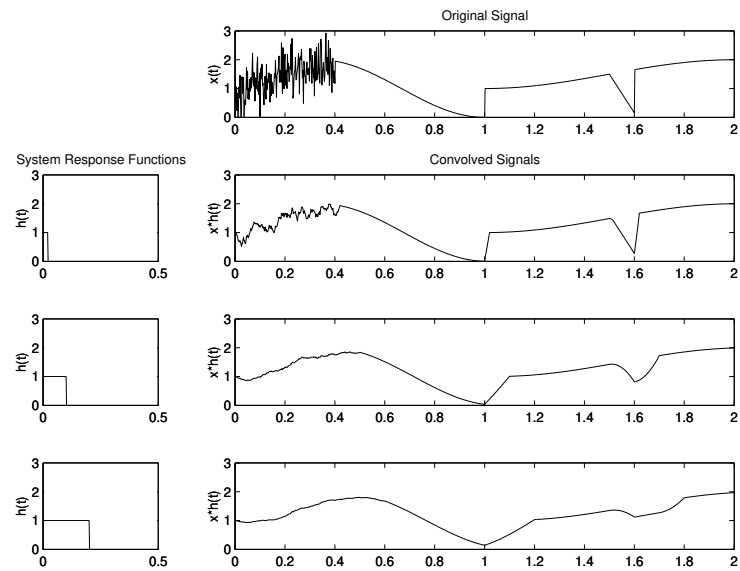


Figure 3.6: Convolving with rect functions of different lengths

3.6 Frequency Response of LTI Systems

In this section, we consider the response of an LTI system to a sinusoidal input. If the input to the system is a sinusoid at a particular frequency, we would like to know what can be said about the output of the system.

It turns out that the behavior of LTI systems to sinusoids is simple. All that an LTI system can do to a sinusoidal input is to change its amplitude and phase. That is, if the input to an LTI system is a sinusoid at some frequency ω then the output is also a sinusoid at frequency ω , but possibly with a different amplitude and phase.

The theory is cleaner if we allow complex sinusoids that were discussed in Chapter 2. Namely, we consider the input

$$x(t) = e^{j\omega t} = \cos \omega t + j \sin \omega t$$

Now, let's see what happens if we apply this input to an LTI system with impulse response $h(t)$.

Recall that the output $y(t)$ is just the convolution of $x(t)$ and $h(t)$, i.e.,

$$y(t) = (x * h)(t)$$

Since we have seen that convolution is commutative, we can write this as

$$y(t) = h(t) * x(t) = \int_{-\infty}^{\infty} h(\tau) x(t - \tau) d\tau$$

So far, this is completely general, i.e., we haven't yet used the fact that $x(t)$ is sinusoidal. We'll now use this fact by substituting $x(t) = e^{j\omega t}$ into the above expression to get

$$\begin{aligned} y(t) &= \int_{-\infty}^{\infty} h(\tau) e^{j\omega(t-\tau)} d\tau \\ &= \int_{-\infty}^{\infty} h(\tau) e^{j\omega t} e^{-j\omega\tau} d\tau \\ &= e^{j\omega t} \int_{-\infty}^{\infty} h(\tau) e^{-j\omega\tau} d\tau \end{aligned}$$

Notice that on the right hand side we have the original complex sinusoid $e^{j\omega t}$ times a term that does not depend on t . Thus, we have shown the desired result.

Of course, the term multiplying $e^{j\omega t}$ depends on ω . In fact, this term is called the *frequency response* of the system, since it tells us what the system does to sinusoidal inputs at each frequency. We will let $H(\omega)$ denote the frequency response, so that

$$H(\omega) = \int_{-\infty}^{\infty} h(\tau) e^{-j\omega\tau} d\tau \quad (3.8)$$

and then we can write

$$y(t) = H(\omega) e^{j\omega t}$$

Once we know the impulse response $h(t)$, then in principle we can compute the frequency response $H(\omega)$ by simply carrying out the integral in Equation (3.8).

The fact that the frequency response can be computed from the impulse response should no longer be surprising to us since we have previously seen that the impulse response actually determines the output for *any* input. So, in particular, it determines the output for a sinusoid at frequency ω . What may be surprising is that the behavior of LTI systems on sinusoids is so simple.

What may be more surprising is that not only does the impulse response determine the frequency response, but the reverse is also true. That is, if we know the frequency response for all ω then we can determine the impulse response, and hence we can determine the output due to any input. Thus, once we know what the system does to sinusoids at each frequency, we know how the system responds to all inputs. This result is closely connected to extremely useful concepts of frequency domain representations of signals and Fourier transforms, which is the topic we will consider in the next chapter. In fact, as we'll soon see, the frequency response $H(\omega)$ is simply the Fourier transform (i.e., frequency domain representation) of the impulse response.

We finish this section by showing that similar ideas hold in the case of discrete-time systems. For the discrete-time case, we consider the discrete-time complex sinusoid

$$x[n] = e^{j\omega n}$$

As in the continuous-time case, the output $y[n]$ of an LTI system with impulse response $h[n]$ is given by

$$\begin{aligned} y[n] &= (x * h)[n] \\ &= (h * x)[n] \\ &= \sum_{k=-\infty}^{\infty} h[k]x[n-k] \end{aligned}$$

Now, substituting the expression for $x[n]$ we get

$$\begin{aligned} y[n] &= \sum_{k=-\infty}^{\infty} h[k]e^{j\omega(n-k)} \\ &= \sum_{k=-\infty}^{\infty} h[k]e^{j\omega n}e^{-j\omega k} \\ &= e^{j\omega n} \sum_{k=-\infty}^{\infty} h[k]e^{-j\omega k} \end{aligned}$$

Thus, in the discrete-time case the frequency response $H(\omega)$ is given by

$$H(\omega) = \sum_{k=-\infty}^{\infty} h[k]e^{-j\omega k} \quad (3.9)$$

The output can then be written as

$$y[n] = H(\omega)e^{j\omega n}$$

so that in the case of a discrete-time LTI system we also get a sinusoidal output at the same frequency as the input.

Chapter 4

Frequency Domain and Fourier Transforms

Frequency domain analysis and Fourier transforms are a cornerstone of signal and system analysis. These ideas are also one of the conceptual pillars within electrical engineering. Among all of the mathematical tools utilized in electrical engineering, frequency domain analysis is arguably the most far-reaching. In fact, these ideas are so important that they are widely used in many fields – not just in electrical engineering, but in practically all branches of engineering and science, and several areas of mathematics.

4.1 Frequency Content: Combining Sinusoids

The most common and familiar example of frequency content in signals is probably audio signals, and music in particular. We are all familiar with “high” musical notes and “low” musical notes. The high notes do in fact have higher frequency content than the low notes, but what exactly does this mean?

The place to start to answer this question is to consider sinusoids. Recall that the general expression for a sinusoid at frequency ω (or frequency f in Hertz) is

$$x(t) = a \sin(\omega t + \phi) = a \sin(2\pi f t + \phi)$$

When considered as an audio signal, $x(t)$ indicates the changes in air pressure on our ears as a function of time. What is important here is the time variation of the air pressure from some ambient value rather than the ambient value of the pressure itself. A negative value refers to that amount below the baseline (ambient) pressure, while a positive amount refers to a pressure higher than the baseline.

So, $x(t)$ being a sinusoid means that the air pressure on our ears varies periodically about some ambient pressure in a manner indicated by the sinusoid. The sound we hear in this case is called a pure tone. Pure tones often sound artificial (or electronic) rather than musical. The frequency of the sinusoid determines the “pitch” of the tone, while the amplitude determines the “loudness”. It turns out that the phase of the sinusoid does not affect our perception of the tone, which may not be surprising for a pure tone, but is somewhat surprising when we start combining sinusoids.

*©2023 by Sanjeev R. Kulkarni. All rights reserved.

[†]Lecture Notes for ECE201 Information Signals.

[‡]Thanks to Sean McCormick and Richard Radke for producing the figures.

We can combine two sinusoids by adding the signals in the usual way. For example,

$$x(t) = \sin(2\pi t) + \sin(4\pi t)$$

is a combination of a sinusoid with frequency 1 Hz and a sinusoid with frequency 2 Hz. Here the amplitude of each sinusoid is 1 and the phase of each is 0. A plot of $x(t)$ is shown in Figure 4.1. The “sound” created by $x(t)$ is the combination of the two pure tones that make $x(t)$. Unfortunately, as we’ll discuss in more detail in Chapter 6, humans can’t hear the pure tones that comprise the signal $x(t)$ above since the frequencies are too low.

However, we can make a similar combination with signals at frequencies humans can hear. For example, consider the signal

$$d(t) = \sin(2\pi \times 350 \times t) + \sin(2\pi \times 440 \times t)$$

Each of the two sinusoids (at frequencies 350 Hz and 440 Hz) alone corresponds to a pure tone that can be heard by the normal human ear. Their combination, i.e., the signal $d(t)$, makes a very familiar sound, namely the dial tone on a standard U.S. telephone line. A plot of $d(t)$ is shown in Figure 4.2. Note that in this figure only 2 hundredths of a second are shown. Because the frequencies are high, if we showed even a whole second, the signal would oscillate so many times (350 and 440 for the constituent sinusoids) that not much useful detail would be seen.

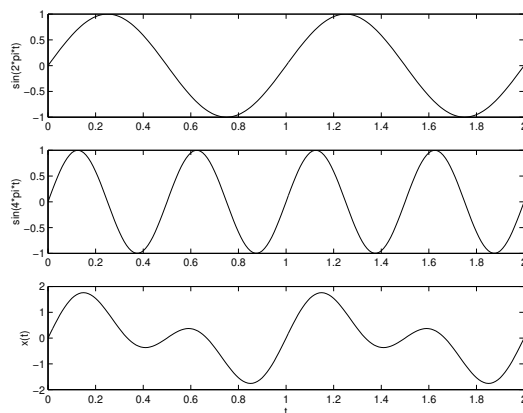


Figure 4.1: Combining sinusoids.

Although the dial tone is a simple example of a sound that still sounds artificial, by combining more sinusoids at different frequencies we can get many other sounds. Musical notes that we find pleasing largely consist of pure tones near the pitch of the musical note, but also contain other frequencies that give each instrument its particular qualities. Voice and other natural sounds are also comprised of a number of pure tones.

Amazingly, all sounds can be built up out of pure tones, and likewise all time signals can be constructed by combining sinusoids. Similarly, starting with a general time signal, one can break this signal down into its constituent sinusoids. How to do this and the consequences of such constructions/decompositions is the subject of frequency domain analysis and Fourier transforms. First, we briefly discuss two other different motivating examples.

4.2 Some Motivating Examples

Hierarchical Image Representation

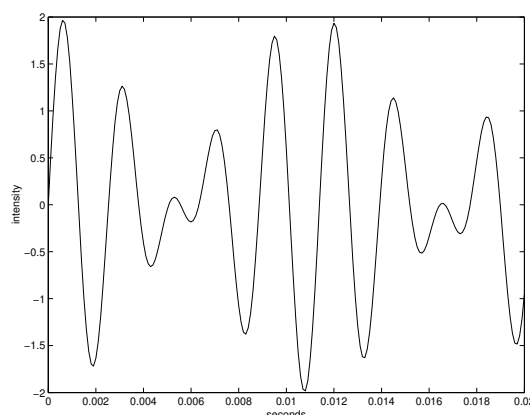


Figure 4.2: Graph of a dial tone.

At some point you have probably experienced delays in downloading files. This is due to various factors including traffic on the network and the size of the files requested. Images and especially video, as we will see in Chapter 9, require a substantial amount of data, and as a result, downloading these can take time.

Suppose we are browsing through a large database/archive of images and wish to find a collection of images of a particular type. Downloading each image completely and then deciding that the bulk of the images are not what we are after can be time-consuming. Of course, we may be able to make this determination (i.e., whether or not the image suits our purposes) with much less quality than the full image possesses. Certainly we may be able to reject most of the images with only a very rough idea of their content. Wouldn't it be nice to be able to make this decision with only 1/10 of the data? That way we could download and go through the database much faster to find what we're after.

The problem is that if we take the standard representation of images (8 bit gray level for each pixel) and send the first 1/10 of the pixels, we will simply get a portion (namely, 1/10) of the original image, albeit at high resolution. On the other hand if we had a way to represent the “coarse” (or low frequency) information separately from the “fine” (or high frequency) information, we could request the coarse information first and only request the additional detail if desired.

Figure 4.3 shows 4 versions of an image, the top two consisting of 25% of the pixels and 6.25% of the pixels in the usual format. The bottom row shows both 25% and 6.25% of the information at the lowest frequencies. For many purposes, the low frequency version may be adequate to make decisions, and it certainly seems more valuable than if we had only the first 25% of the pixels. This notion of a “hierarchical” representation can be formalized using ideas from frequency domain analysis.

It turns out that frequency domain ideas can help with this browsing problem in a different way as well. In addition to creating hierarchical representations that allow sending the most important information first, one might consider reducing the total amount of data in the first place. As we will see, one of the standard methods for image compression known as JPEG is based on frequency domain ideas. This is the subject of data compression, which will be discussed in Chapters 8 and 9.

Radio and TV Transmission

Radio, television, and some other forms of communication (e.g., cell phones) transmit infor-

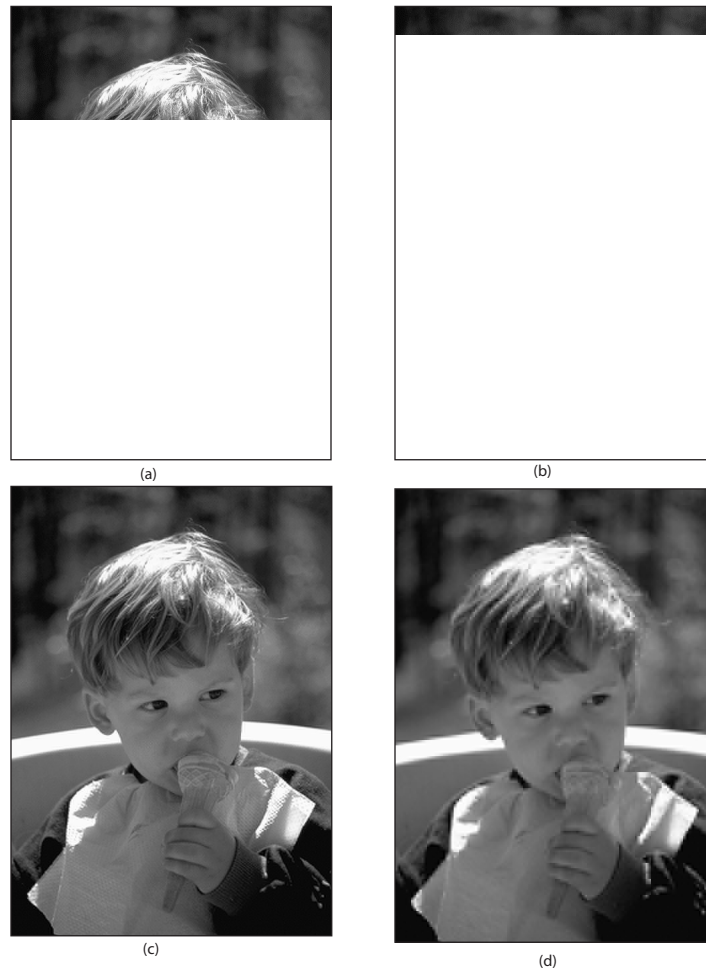


Figure 4.3: Sending image data in usual format vs. sending only low frequency data. a) 25% of data in usual format. b) 6.25% of data in usual format. c) 25% of lowest frequency data. d) 6.25% of lowest frequency data.

mation via electromagnetic waves. The various sources in these applications can be transmitting simultaneously and in the same geographic region. But how is it that we can “tune in” to a specific radio station, television program, or individual with whom we’re communicating, rather than hearing the jumble of all the various transmissions put together?

One approach to this is that different transmissions can agree to use different frequencies. Thus, even though all the signals *are* “jumbled” together in the time domain, they are distinct in the frequency domain. With some basic frequency domain processing, it is straightforward to separate the signals and “tune in” to the frequency we are interested in.

4.3 A Trivial Frequency Decomposition

Before discussing frequency representations for general signals, we consider an example that is trivial but is still somewhat illustrative. Consider a situation in which we are interested in the values of a signal $x[0]$ and $x[1]$ at only two times $n = 0$ and $n = 1$.

Suppose we have access to the two values of the signal, and wish to convey these values to a friend. We could simply convey the values $x[0]$ and $x[1]$ themselves and we’re done.

However, consider the the following alternative scheme. Define a new signal $X[0]$ and $X[1]$ by

$$\begin{aligned} X[0] &= \frac{1}{2}(x[0] + x[1]) \\ X[1] &= \frac{1}{2}(x[0] - x[1]) \end{aligned}$$

We can then convey the values $X[0]$ and $X[1]$ to our friend, and using these values the friend can recover the original signal $x[0]$ and $x[1]$. Namely, given $X[0]$ and $X[1]$, we get $x[0]$ and $x[1]$ by

$$\begin{aligned} x[0] &= X[0] + X[1] \\ x[1] &= X[0] - X[1] \end{aligned}$$

This example seems trivial, but even this simple case illustrates some interesting properties. For example, in the new scheme, information on *both* values of $x[0]$ and $x[1]$ are being sent simultaneously – partially in $X[0]$ and partially in $X[1]$. If we receive only $X[0]$ then we don’t know *either* of the original signal values, but we know something about both.

Roughly, $X[0]$ is the “low frequency” content of the signal $x[0], x[1]$. This is the part of the signal that doesn’t change, or that’s common to both $x[0]$ and $x[1]$. It is simply the average of $x[0]$ and $x[1]$. $X[1]$ is the “high frequency” content of the signal – the part that changes between $x[0]$ and $x[1]$. In fact, the equation shows that it is simply half of the difference between $x[0]$ and $x[1]$.

We can think of the signal $X[\cdot]$ as just a different representation for the signal $x[\cdot]$ since we can easily go back and forth between the two representations (using the equations above). $X[\cdot]$ will be called the frequency domain representation, while the original signal $x[\cdot]$ will be called the time domain representation. The term “time domain” refers to the fact that when describing the values of $x[\cdot]$ directly, we simply give the values of $x[n]$ where $n = 0, 1$ denotes time. On the other hand, the “frequency domain” description gives the values of $X[k]$ where k also happens to take on the values 0 and 1, but k denotes frequency. That is, $X[0]$ is *not* the value of the original signal at time 0, but is rather the frequency content of the original signal at the low frequency (frequency 0).

As we will see, this alternate representation can be immensely useful in analyzing signals and the effects of systems on signals. Something very complicated in time domain can turn out to be very simple in frequency domain. Obviously, to be useful we need to extend this notion of frequency representations to longer signals. Fourier transforms and frequency domain analysis do just that.

4.4 Summary of the Four Cases

The details of how to formulate and extend the ideas in the sections above depend on whether the underlying signal is continuous-time or discrete-time, and whether it is periodic/finite-duration or aperiodic/infinite-duration. This results in four distinct cases, although there are many similarities and connections between these four cases.

The names of the transform in the four cases are summarized in Figure 4.4. The nomenclature is unfortunately somewhat confusing, but is rather entrenched, and so we will use the standard terminology.

Time-domain Properties	Extent	
	Finite extent	Infinite extent
Continuous	Fourier Series (FS)	Fourier Transform (FT)
Discrete	Discrete Fourier Transform (DFT)	Discrete-Time Fourier Transform (DTFT)

Figure 4.4: The four Fourier transforms broken down by time domain properties.

Figure 4.5 gives the definitions of the transforms in each of the four cases. First, we'll make some general remarks about all the transforms, and then we'll discuss a little more of the details for the case of the Fourier transforms (continuous-time, infinite duration) and the DFT (discrete-time, finite duration). For now it's enough to get just a rough idea of the transforms.

Notation, Frequency Variable

In each case, the original (time-domain) signal is represented by lower case x – in particular, $x(t)$ in continuous-time and $x[n]$ in discrete-time. The appropriate Fourier transform in each case is represented by upper case X .

Just as different symbols are used to denote time in the continuous-time and discrete-time cases, different symbols are used for the frequency variable depending on whether there are a continuous range of frequencies or a discrete set of frequencies. For a continuous range of frequencies, the variable ω is used. As before, ω denotes frequency in radians/sec. For the DFT and Fourier series, only a discrete set of frequencies are used. The discrete frequencies are all a multiple of a specific frequency, denoted ω_0 , called the *fundamental frequency*. The variable k is used to specify the particular multiple of the fundamental frequency. Of course, the actual frequency of the complex exponential is $k\omega_0$, again in units of radians/sec.

Forward and Inverse Transform

Fourier Series (FS) For $x(t)$ of duration T, set $\omega_0 = \frac{2\pi}{T}$. $x(t) : 0 \leq t \leq T$ $X[k] : k = \dots, -2, -1, 0, 1, 2, \dots$ $X[k] = \frac{1}{T} \int_{t=0}^T x(t) e^{-jk\omega_0 t} dt$ $x(t) = \sum_{k=-\infty}^{\infty} X[k] e^{jk\omega_0 t}$	Fourier Transform (FT) $x(t) : -\infty < t < \infty$ $X(\omega) : -\infty < \omega < \infty$ $X(\omega) = \int_{t=-\infty}^{\infty} x(t) e^{-j\omega t} dt$ $x(t) = \frac{1}{2\pi} \int_{\omega=-\infty}^{\infty} X(\omega) e^{j\omega t} d\omega$
Discrete Fourier Transform (DFT) For $x[n]$ of length N, set $\omega_0 = \frac{2\pi}{N}$. $x[n] : n = 0, 1, \dots, N-1$ $X[k] : k = 0, 1, \dots, N-1$ $X[k] = \sum_{n=0}^{N-1} x[n] e^{-jk\omega_0 n}$ $x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] e^{jk\omega_0 n}$	Discrete-Time Fourier Transform (DTFT) $x[n] : n = \dots, -2, -1, 0, 1, 2, \dots$ $X(\omega) : -\pi \leq \omega \leq \pi$ $X(\omega) = \sum_{n=-\infty}^{\infty} x[n] e^{-j\omega n}$ $x[n] = \frac{1}{2\pi} \int_{\omega=-\pi}^{\pi} X(\omega) e^{j\omega n} d\omega$

Figure 4.5: Definitions of the forward and inverse Fourier transforms in each of the four cases.

Note that there are two equations in each case. The first equation has the transform X on the left-hand-side and either a sum or integral involving the original signal x on the right-hand-side. This equation gives the expression for the transform in terms of the original signal. For each frequency, the expression tells how much of this frequency is contained in the original signal x .

The second equation in each case has the original signal x on the left-hand-side, and either a sum or integral involving the transform X on the right-hand-side. This equation tells how to recover a signal if we know the frequency content of the signal. This equation is often referred to as the *inverse transform*, since it “un-does” the transform. Sometimes to emphasize the distinction with the inverse, the term *forward transform* is used to refer to the first equation where X is obtained from knowledge of the original signal x .

Complex Values

Because of the complex exponential in the equations for X , it is possible (in fact, it is typical), that the transform X will take on complex values. This is not a problem. With a little thought this should be expected.

Recall we can write a complex number in terms of its magnitude and phase (i.e., its polar representation). These quantities determine the magnitude and phase of the underlying complex exponential. $X(\omega)$ tells how much content the original signal has at frequency ω . More precisely, $X(\omega)$ tells both the magnitude and phase to use for the complex exponential $e^{j\omega t}$ at frequency ω . If $X(\omega) = Ae^{j\phi}$ in polar form, then $X(\omega)e^{j\omega t} = Ae^{j(\omega t + \phi)}$ so that A and ϕ are the magnitude and phase, respectively, of the complex sinusoid.

If X can be complex-valued, it is natural to wonder why x isn’t also complex-valued (since the forward and inverse transform equations look so similar). The simple answer is that we started with a real-valued signal x . Applying the forward transform then the inverse transform just gives us back the original signal. So, we have to end up with a real-valued signal, since that’s what we started with! Of course, it also means that the transform X probably has some special structure (reflecting the special structure that the original signal is real). This is in fact the case and is listed as one of the properties in Table 4.2 later in this chapter.

This discussion also suggests that we could allow the original signal to be complex-valued. This can be done and it is sometimes useful, although in most applications the original signal, representing a physical variable of interest, is real-valued.

Interpretation as Frequency Representation

The equation for the inverse transform is natural once we accept that $X[k]$ (or $X(\omega)$) indicates the “amount” of frequency $k\omega_0$ (or frequency ω , respectively) contained in the original signal. For example, consider the case of Fourier series. In this case, the expression for $x(t)$ is just a sum of sinusoids where the amplitude of the sinusoid of frequency $k\omega_0$ is $X[k]$. The case of the DFT is similar. In the cases where there is a continuous range of frequencies (Fourier transform and DTFT), rather than a sum of terms of different frequencies, we have an integral. All of these equations show that the transform X really does give a frequency representation of x by expressing the original signal in terms of constituent sinusoids, where the amplitude and phase of the sinusoid at frequency $k\omega_0$ (or, frequency ω) is given by $X[k]$ (or, $X(\omega)$).

The fact that we have a forward and inverse transform justifies the interpretation that the transform X is really just a different representation, the frequency domain representation, of the signal x . If we know the transform then we can recover the signal using the inverse transform. Likewise, if we know the signal then we can obtain the frequency domain representation by using the forward transform. Specifying the signal in either time domain or frequency domain completely determines the signal.

Duality

The similarity in the equations for the forward and inverse transforms suggest that there may be some duality. This is indeed the case. Here, we make note only of the two attributes: whether the time/frequency variable is continuous or discrete, and whether the signal in time/frequency domain is finite-extent or infinite extent.

It turns out that being finite extent in one domain means the variable in the other domain is discrete. So, a finite extent in time (the case of Fourier Series and DFT) means the frequency variable in these cases is discrete. Likewise, infinite extent in one domain means the variable in the other domain is continuous (the case of the DTFT and FT).

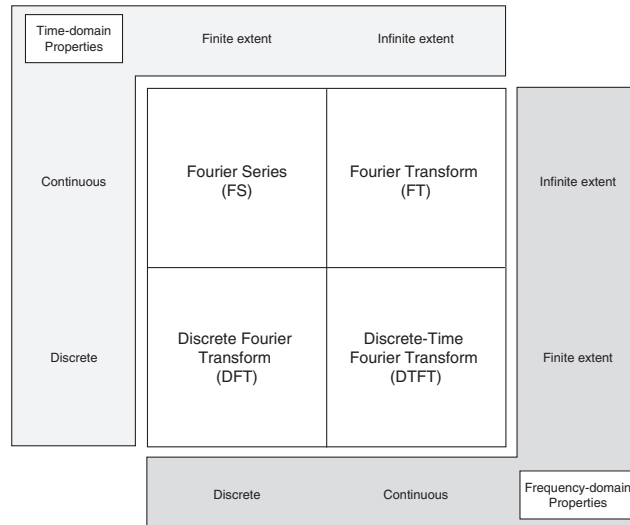


Figure 4.6: Time and frequency domain properties for the four cases.

This relationship is shown in Figure 4.6, which repeats much of Figure 4.4 but with additional labeling showing the frequency domain properties in each case. The two cases we will discuss in more detail are either continuous and infinite extent in both domains (FT), or discrete and finite extent in both domains (DFT). For the other two cases these properties toggle as we switch domains. That is, for Fourier series, the signal is continuous and finite duration in time, but in frequency it is discrete and infinite duration. For the DTFT, the signal is discrete and infinite extent in time, but continuous and finite extent in frequency.

There are further duality properties as well, which we will discuss in the context of the FT and DFT.

4.5 FT and DFT

In this and the next two sections, we consider in more detail the continuous-time Fourier transform, or simply the Fourier transform (FT), and the discrete Fourier transform (DFT). These are the transform for continuous-time, infinite-duration signals (FT), and for discrete-time, finite-duration signals (DFT). The FT is most amenable to analytical insights and manipulations, and is used in analyzing analog systems. The DFT the workhorse that is widely used in digital computations, because of a fast algorithm called the FFT (fast Fourier transform) for computing the DFT.

For convenience, we repeat the definitions of the forward and inverse transforms here. For the FT, both the time variable t and frequency variable ω are continuous-valued and vary from $-\infty$ to

∞ . The transform equations are

$$X(\omega) = \int_{-\infty}^{\infty} x(t)e^{-j\omega t} dt \quad \text{forward transform} \quad (4.1)$$

$$x(t) = \frac{1}{2\pi} \int_{-\infty}^{\infty} X(\omega)e^{j\omega t} d\omega \quad \text{inverse transform} \quad (4.2)$$

The DFT is used in the case of a finite-duration, discrete-time signal, $x[0], x[1], \dots, x[N-1]$. The DFT $X[0], X[1], \dots, X[N-1]$ is also finite-duration and discrete-frequency. The definitions of the forward and inverse DFT are:

$$X[k] = \sum_{n=0}^{N-1} x[n]e^{-jk\omega_0 n} \quad k = 0, 1, \dots, N-1 \quad (4.3)$$

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k]e^{jk\omega_0 n} \quad n = 0, 1, \dots, N-1 \quad (4.4)$$

where

$$\omega_0 = \frac{2\pi}{N} \quad (4.5)$$

Before considering some examples and properties of Fourier transforms, we introduce some notation and discuss duality a bit further. If $X(\omega)$ is the Fourier transform of a signal $x(t)$ (and so $x(t)$ is the inverse transform of $X(\omega)$), then we say that $X(\omega)$ and $x(t)$ are a Fourier transform pair. If $X(\omega)$ and $x(t)$ are a Fourier transform pair, we may write

$$x(t) \longleftrightarrow X(\omega)$$

Or, to emphasize that one is dealing with Fourier transforms (as opposed to some other transforms, of which there are many), this is sometimes written

$$x(t) \xrightarrow{\text{FT}} X(\omega)$$

We also may use the notation $\mathcal{F}\{\cdot\}$ to denote the Fourier transform operator, and $\mathcal{F}^{-1}\{\cdot\}$ to denote the inverse transform operator. That is,

$$X(\omega) = \mathcal{F}\{x(t)\}$$

and

$$x(t) = \mathcal{F}^{-1}\{X(\omega)\}$$

Similar notation applies in the discrete case for DFT's, with the obvious changes of $x[n]$ and $X[k]$ instead of $x(t)$ and $X(\omega)$, respectively.

Regarding duality, notice again the similarity between the forward and inverse transform equations. Aside from the constants and time and frequency being interchanged, the only difference is the sign in the exponent of the complex exponential. This suggests that whenever we have a transform pair, there is a dual pair with the time and frequency variables interchanged. For example, we will see in Section 4.6 below that the FT of a rect function in time is a sinc function in frequency. From duality, this means that the FT of a sinc function in time will be a rect function in frequency. In general, one should be a little careful making sure all the constants are worked out properly.

Duality also suggests that whenever we have a property of the FT, then there will be a dual property with the roles of time and frequency interchanged. For example, we will see in Section 4.6 below that multiplying a signal in the time domain by a complex exponential corresponds to a shift in the frequency domain. Duality then suggests that a shift in the time domain corresponds to multiplication by a complex exponential in the frequency domain.

4.6 Some Examples of Transform Pairs

Some Fourier transform pairs can be computed quite easily directly from the definition. Given $x(t)$, we can substitute directly into the integral defining $X(\omega)$ and carry out the integration.

Example 4.1 (Fourier transform of the $\text{rect}(\cdot)$ function)

Consider $x(t) = \text{rect}(t)$ shown in Figure 4.7. From the definition of $X(\omega)$ we have

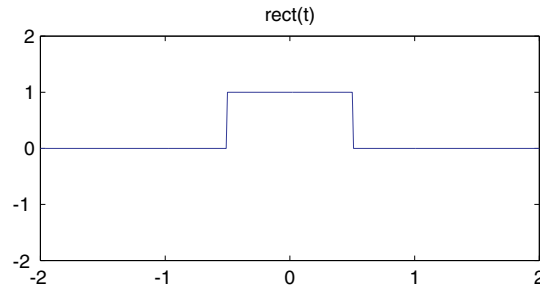


Figure 4.7: Rect function.

$$\begin{aligned}
 X(\omega) &= \mathcal{F}\{x(t)\} = \int_{-\infty}^{\infty} x(t)e^{-j\omega t} dt \\
 &= \int_{-\frac{1}{2}}^{\frac{1}{2}} e^{-j\omega t} dt \\
 &= \left. \frac{e^{-j\omega t}}{-j\omega} \right|_{t=-\frac{1}{2}}^{\frac{1}{2}} \\
 &= \frac{e^{j\frac{\omega}{2}} - e^{-j\frac{\omega}{2}}}{j\omega} \\
 &= \frac{2\sin(\frac{\omega}{2})}{\omega} \\
 &= \text{sinc}\left(\frac{\omega}{2\pi}\right)
 \end{aligned}$$

Therefore,

$$\mathcal{F}\{\text{rect}(t)\} = \frac{2\sin(\frac{\omega}{2})}{\omega} = \text{sinc}\left(\frac{\omega}{2\pi}\right)$$

A plot of $X(\omega)$ is shown in Figure 4.8. ■

Example 4.2 (FT of the delta function)

One Fourier transform that is particularly easy to compute directly from the definition is that of the Dirac delta function introduced in Chapter 2. Using the defining property of $\delta(t)$, we get

$$\mathcal{F}\{\delta(t)\} = \int_{-\infty}^{\infty} \delta(t)e^{-j\omega t} dt = 1$$

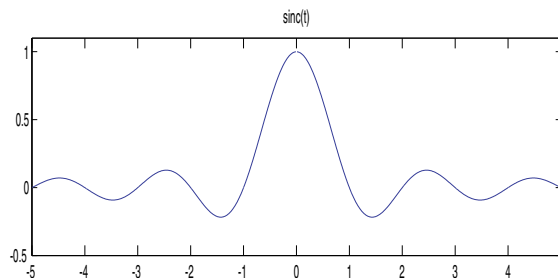


Figure 4.8:

Sometimes a Fourier transform pair can more easily be established by considering the inverse transform. Remember that since a function is uniquely determined in either time or frequency domain, considering the inverse transform is a perfectly valid way to obtain transform pairs. Also, because of the similarity between the definitions of the forward and inverse transforms, there is often a duality that can be exploited to obtain transform pairs.

For example, by considering the delta function $\delta(\omega)$ in frequency domain and using the inverse transform definition, we can see (similar to Example 4.1) that

$$\frac{1}{2\pi} \longleftrightarrow \delta(\omega)$$

or, equivalently

$$1 \longleftrightarrow 2\pi\delta(\omega)$$

This could also be guessed (except perhaps for the exact constants) by the duality between forward and inverse transforms.

For another example of duality, consider the $\text{rect}(\cdot)$ and $\text{sinc}(\cdot)$ functions. We saw in Example 4.1 that the Fourier transform of the rect function in time-domain is a sinc function in frequency domain. One might guess that the Fourier transform of a sinc function in the time domain is a rect function in frequency domain. This turns out to be correct, as could be easily established by considering a rect in frequency domain and working through a calculation as in Example 4.1. Of course, one needs to be careful of the exact constants, but the underlying integrals are almost identical.

Table 4.1 shows several other Fourier transform pairs. Extensive tables of Fourier transforms have been developed and the pairs shown in Table 4.1 are a very small subset of known transform pairs.

Direct computation through the definition is not the only way to derive transform pairs. Once we have some transform pairs, many others can be obtained using general properties of the Fourier transform discussed in the next section.

4.7 Some Transform Properties

Fourier transforms satisfy a number of general and quite useful properties. These properties give some nice insights on the behavior of Fourier transforms. They are often handy when doing analysis using Fourier transforms, and can also be helpful in using known transform pairs to compute new

Some Fourier Transform Pairs	
Signal	Fourier Transform
$\delta(t)$	1
$e^{j\omega_0 t}$	$2\pi\delta(\omega - \omega_0)$
$\sin(\omega_0 t)$	$\frac{\pi}{j}[\delta(\omega - \omega_0) - \delta(\omega + \omega_0)]$
$\cos(\omega_0 t)$	$\pi[\delta(\omega - \omega_0) + \delta(\omega + \omega_0)]$
$x(t) = 1$	$2\pi\delta(\omega)$
$\text{rect}(t)$	$\text{sinc}(\frac{\omega}{2\pi})$
$\frac{\sin(Wt)}{\pi t}$	$X(\omega) = \begin{cases} 1, & \omega < W \\ 0, & \omega > W \end{cases}$
$u(t)$	$\frac{1}{j\omega} + \pi\delta(\omega)$
$\text{tri}(t)$	$\text{sinc}^2(\omega/(2\pi))$

Table 4.1: Some Fourier Transform Pairs

ones. The properties tell us how the Fourier transform changes when the original signal undergoes certain changes. They are generally derived by going back to the definitions and manipulating the equations appropriately.

Example 4.3 (Linearity)

One of the simplest but most useful property is that the Fourier transform is linear. That is, if $X(\omega)$ and $Y(\omega)$ are the Fourier transforms of two signals $x(t)$ and $y(t)$, respectively, then the Fourier transform of $ax(t) + by(t)$ is $aX(\omega) + bY(\omega)$. This is often written using the notation described earlier as

$$ax(t) + by(t) \longleftrightarrow aX(\omega) + bY(\omega)$$

where it is assumed as given that $x(t) \longleftrightarrow X(\omega)$ and $y(t) \longleftrightarrow Y(\omega)$.

To show linearity, we can substitute directly in the definition and use the fact that integration is linear. Namely,

$$\begin{aligned}
 \mathcal{F}\{ax(t) + by(t)\} &= \int_{-\infty}^{\infty} [ax(t) + by(t)]e^{-j\omega t} dt \\
 &= a \int_{-\infty}^{\infty} x(t)e^{-j\omega t} dt + b \int_{-\infty}^{\infty} y(t)e^{-j\omega t} dt \\
 &= a\mathcal{F}\{x(t)\} + b\mathcal{F}\{y(t)\}
 \end{aligned}$$

■

Example 4.4 (Frequency shift property)

The so-called frequency shift property states that

$$e^{j\omega_0 t}x(t) \longleftrightarrow X(\omega - \omega_0)$$

This states that if a signal is multiplied by a complex exponential at some frequency ω_0 , all that happens to the Fourier transform is that it gets shifted by the same frequency ω_0 .

With a little thought, this result should be expected. If the complex exponential $e^{j\omega t}$ is multiplied by $e^{j\omega_0 t}$, then we simply get $e^{j(\omega+\omega_0)t}$. That is, we still have a complex exponential, but at the new frequency $\omega + \omega_0$. So, to determine how much content the signal $e^{j\omega_0 t}x(t)$ has at a given frequency ω , we need only check what content $x(t)$ has at the frequency $\omega - \omega_0$. This result is at the heart of certain techniques in communication known as modulation, which will be described in Chapter 7.

The frequency shift property can be obtained by simply combining the complex exponentials in the integral and then noticing that the resulting expression is precisely the definition of $X(\cdot)$ evaluated at the frequency $\omega - \omega_0$. Namely,

$$\begin{aligned}\mathcal{F}\{e^{j\omega_0 t}x(t)\} &= \int_{-\infty}^{\infty} (e^{j\omega_0 t}x(t)) e^{-j\omega t} dt \\ &= \int_{-\infty}^{\infty} x(t) e^{-j(\omega-\omega_0)t} dt \\ &= X(\omega - \omega_0)\end{aligned}$$

■

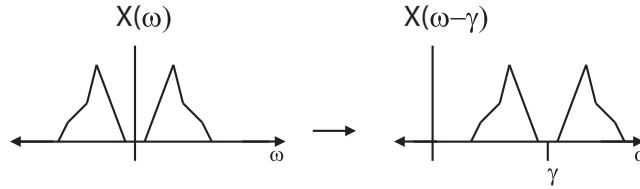


Figure 4.9: Frequency shift due to multiplication by complex exponential

Example 4.5 (Time shift property)

The time shift property states that

$$x(t - t_0) \longleftrightarrow e^{-j\omega t_0} X(\omega)$$

In words, shifting a signal in the time domain causes the Fourier transform to be multiplied by a complex exponential. Incidentally, this shows that if we view the Fourier transform as a mapping from input signals (the time-domain representation) to output signals (the frequency domain representation) then this mapping is *not* time-invariant, although as we saw above it *is* linear.

Because of duality, we might have guessed this result once we saw the frequency shift property above. One could prove the time shift property in a manner very similar to the way we proved the frequency shift property, but by manipulating the inverse transform equation instead of the forward transform equation.

However, a more direct method is to work with the forward transform and use a simple change

of variables in the defining equation. Namely,

$$\begin{aligned}
 \mathcal{F}\{x(t - t_0)\} &= \int_{-\infty}^{\infty} x(t - t_0) e^{-j\omega t} dt \\
 &= \int_{-\infty}^{\infty} x(u) e^{-j\omega(u+t_0)} du \quad \text{letting } u = t - t_0 \\
 &= e^{-j\omega t_0} \int_{-\infty}^{\infty} x(u) e^{-j\omega u} du \\
 &= e^{-j\omega t_0} X(\omega)
 \end{aligned}$$

■

The general method used above (of writing the definition and using a change of variable and rearranging expressions) can be a very useful method for proving other properties as well. It may be reminiscent of how we proved some properties of the delta function.

A number of other properties of the Fourier transform are known, some of which are summarized in Table 4.2. We will discuss one more extremely important property in the next section.

Fourier Transform Properties
Linearity $ax(t) + by(t) \longleftrightarrow aX(\omega) + bY(\omega)$
Frequency Shift $e^{j\omega_0 t} x(t) \longleftrightarrow X(\omega - \omega_0)$
Time Shift $x(t - t_0) \longleftrightarrow e^{-j\omega t_0} X(\omega)$
Time Reversal $x(-t) \longleftrightarrow X(-\omega)$
Time Scaling $x(at) \longleftrightarrow \frac{1}{ a } X\left(\frac{\omega}{a}\right)$
Differentiation in Time $\frac{d}{dt} x(t) \longleftrightarrow j\omega X(\omega)$
Convolution in Time $x(t) * y(t) \longleftrightarrow X(\omega) Y(\omega)$
Multiplication in Time $x(t)y(t) \longleftrightarrow \frac{1}{2\pi} \int_{\gamma=-\infty}^{\infty} X(\gamma) Y(\omega - \gamma) d\gamma$

Table 4.2: Some Fourier Transform properties

4.8 Convolution Property and Another Look at Impulse and Frequency Response

The convolution property is so important that it deserves a separate section. Although it is not difficult to derive, it is a little more involved than the properties from the previous section.

The convolution property states that

$$x(t) * y(t) \longleftrightarrow X(\omega)Y(\omega)$$

That is, if two signals are convolved in the time domain then the Fourier transform of the convolution is just the product of the two original Fourier transforms. In other words, convolving signals in time domain is equivalent to multiplying their Fourier transforms in frequency domain. Thus, what appears to be a complicated operation in time, is simple if viewed in terms of frequency.

As with some others, this property is derived by using the definition of the Fourier transform and manipulating the expression. But, now we will have two integrals – one for the transform and one for the convolution.

$$\begin{aligned} \mathcal{F}\{x(t) * h(t)\} &= \mathcal{F}\left\{\int_{-\infty}^{\infty} x(\tau)h(t-\tau) d\tau\right\} \\ &= \int_{-\infty}^{\infty} \left(\int_{-\infty}^{\infty} x(\tau)h(t-\tau) d\tau\right) e^{-j\omega t} dt \\ &= \int_{-\infty}^{\infty} x(\tau) \left(\int_{-\infty}^{\infty} h(t-\tau)e^{-j\omega t} dt\right) d\tau \\ &\quad \text{by changing the order of integration} \\ &= \int_{-\infty}^{\infty} x(\tau) \left(\int_{-\infty}^{\infty} h(u)e^{-j\omega(u+\tau)} du\right) d\tau \\ &\quad \text{by a change of variables to } u = t - \tau \\ &= \left(\int_{-\infty}^{\infty} x(\tau)e^{-j\omega\tau} d\tau\right) \left(\int_{-\infty}^{\infty} h(u)e^{-j\omega u} du\right) \\ &= X(\omega)H(\omega) \end{aligned}$$

Once we have the convolution property, duality suggests that multiplication in the time domain corresponds to convolution in the frequency domain. Specifically, this dual property is

$$x(t)y(t) \longleftrightarrow \frac{1}{2\pi}X(\omega) * Y(\omega) = \frac{1}{2\pi} \int_{-\infty}^{\infty} X(\eta)Y(\omega - \eta) d\eta$$

This property is called the multiplication property or the modulation property.

Recall that for an LTI system with impulse response $h(t)$, if we apply the input $x(t)$ then the output $y(t)$ is

$$y(t) = x(t) * h(t).$$

We can now ask what this output looks like in frequency domain. From the convolution property, we immediately see that

$$Y(\omega) = X(\omega)H(\omega).$$

Thus, the output of an LTI system can be viewed in either time domain or in frequency domain. In time domain, the output is the convolution of the input with the impulse response. In frequency

domain, the LTI system does something particularly simple. It simply multiplies the FT of the input with the function $H(\omega)$, which is the FT of the impulse response.

But this is precisely how we described the frequency response of an LTI system in Chapter 3. We now see that the frequency response of an LTI system is just the Fourier transform of its impulse response. Compare Equation (3.8) with the definition of the FT in Equation 4.1.

Although all of the properties in Table 4.2 are useful, the convolution result is *the* property to remember and is at the heart of much of signal processing and systems theory.

4.9 2-D Transforms*

So far, our discussion of Fourier transforms has focused on the case of 1-D signals – that is, signals as a function of time. Of course, nothing changes if the variable t were to represent a spatial variable instead of time. The crucial thing so far is that the signals were a function of just one independent variable.

To be able to apply the powerful tools of frequency domain analysis to images, mentioned in Section 4.2, we need to extend the Fourier transform to two dimensions.

In 2-D, the signal x is a function of two spatial arguments, which we will denote u and v . As discussed in Chapter 2, $x(u, v)$ denotes the value of the signal at the point (u, v) .

We already saw in Section 2.7 the notion of a sinusoid in two dimensions. For example,

$$x(u, v) = \sin(\omega_1 u) \sin(\omega_2 v)$$

is a 2-D sinusoid with frequency ω_1 in the u direction and frequency ω_2 in the v direction. This looks like an egg carton, or an array of hills and valleys. A surface plot and an image of this $x(u, v)$ are shown in Figure 4.10. In the image, the dark regions correspond to valleys while the bright regions correspond to hills.

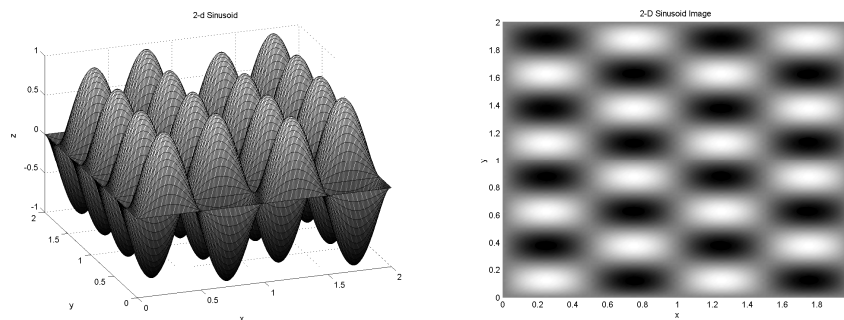


Figure 4.10: 2-d sinusoid surface plot (left) and image (right)

As in 1-D, it is more convenient to work with complex exponentials rather than the sinusoids above. Thus, the basic signals we will work with are of the form

$$e^{j\omega_1 u} e^{j\omega_2 v} = e^{j(\omega_1 u + \omega_2 v)}$$

This is a separable signal as defined in Chapter 2, in that it can be written as a product of a function of only u and a function of only v , with frequency ω_1 in the u direction and frequency ω_2 in the v direction.

One would expect that the frequency domain representation of a general signal $x(u, v)$ would be a function of two frequency variables, one for u and one for v . That is, the Fourier transform should be a function $X(\omega_1, \omega_2)$. The definitions of the forward and inverse transform in 2-D are

$$X(\omega_1, \omega_2) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} x(u, v) e^{-j(\omega_1 u + \omega_2 v)} du dv \quad \text{forward} \quad (4.6)$$

$$x(u, v) = \frac{1}{4\pi^2} \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} X(\omega_1, \omega_2) e^{j(\omega_1 u + \omega_2 v)} d\omega_1 d\omega_2 \quad \text{inverse} \quad (4.7)$$

The definition of the DFT can be extended to 2-D in the natural way as well (as can Fourier series and the discrete-time Fourier transform). As in 1-D, specific transform pairs can be computed and most of the properties in 1-D have natural extensions to 2-D. When the original signal $x(u, v)$ is separable (i.e., can be written as $x(u, v) = x_1(u)x_2(v)$) then the 2-D transform is particularly simple since it reduces to two separate 1-D transforms – one in the variable u , and one in the variable v .

An example of a natural image and its FT is shown in Figure 4.12. As usual, only the magnitude of the transform is shown, where lighter areas denote higher magnitude for the corresponding frequencies, and darker areas denote smaller magnitude. Actually, since the variation in magnitude is so large, what is shown for the Fourier transform is the log of the magnitude. This allows seeing more detail in the transform as opposed to one extremely bright spot in the center and almost black everywhere else. Displaying the log of the magnitude is typical.

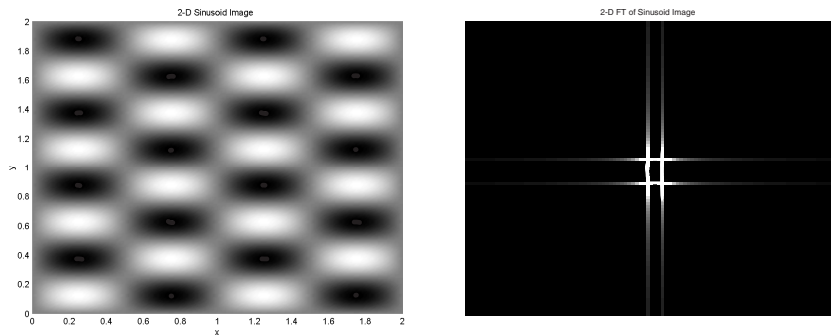


Figure 4.11: 2-D sine and its Fourier Transform



Figure 4.12: A natural image and its Fourier Transform

Chapter 5

Sampling and Quantization

Often the domain and the range of an original signal $x(t)$ are modeled as continuous. That is, the time (or spatial) coordinate t is allowed to take on arbitrary real values (perhaps over some interval) and the value $x(t)$ of the signal itself is allowed to take on arbitrary real values (again perhaps within some interval). As mentioned previously in Chapter 2, such signals are called *analog* signals. A continuous model is convenient for some situations, but in other situations it is more convenient to work with *digital* signals — i.e., signals which have a discrete (often finite) domain and range. The process of digitizing the domain is called *sampling* and the process of digitizing the range is called *quantization*.

Most devices we encounter deal with both analog and digital signals. Digital signals are particularly robust to noise, and extremely efficient and versatile means for processing digital signals have been developed. On the other hand, in certain situations analog signals are sometimes more appropriate or even necessary. For example, most underlying physical processes are analog (or at least most conveniently modeled as analog), including the human sensorimotor systems. Hence, analog signals are typically necessary to interface with sensors and actuators. Also, some types of data processing and transmission are most conveniently performed with analog signals. Thus, the conversion of analog signals to digital signals (and vice versa) is an important part of many information processing systems.

In this chapter, we consider some of the fundamental issues and techniques in converting between analog and digital signals. For sampling, three fundamental issues are (i) How are the discrete-time samples obtained from the continuous-time signal?; (ii) How can we reconstruct a continuous-time signal from a discrete set of samples?; and (iii) Under what conditions can we recover the continuous-time signal exactly? For quantization, the three main issues we consider are (i) How many quantization levels should we choose?; (ii) How should the value of the levels be chosen?; and (iii) How should we map the values of the original signal to one of the quantization levels?

5.1 Sampling a Signal

Figure 5.1 shows an analog signal together with some samples of the signal. The samples shown are equally spaced and simply pick off the value of the underlying analog signal at the appropriate times. If we let T denote the time interval between samples, then the times at which we obtain samples are given by nT where $n = \dots, -2, -1, 0, 1, 2, \dots$. Thus, the discrete-time (sampled) signal

*©2023 by Sanjeev R. Kulkarni. All rights reserved. Please do not distribute without permission of the author.

[†]Lecture Notes for ECE201 Information Signals.

[‡]Thanks to Richard Radke for producing the figures.

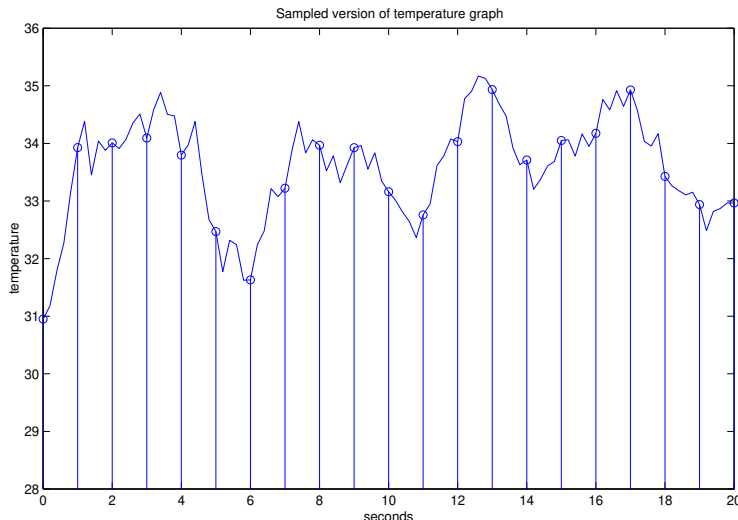


Figure 5.1: Sampling an analog signal.

$x[n]$ is related to the continuous-time signal by

$$x[n] = x(nT).$$

It is often convenient to talk about the sampling frequency f_s . If one sample is taken every T seconds, then the sampling frequency is $f_s = 1/T$ Hz. The sampling frequency could also be stated in terms of radians, denoted by ω_s . Clearly, $\omega_s = 2\pi f_s = 2\pi/T$.

The type of sampling mentioned above is sometimes referred to as “ideal” sampling. In practice, there are usually two non-ideal effects. One effect is that the sensor (or digitizer) obtaining the samples can’t pick off a value at a single time. Instead, some averaging or integration over a small interval occurs, so that the sample actually represents the average value of the analog signal in some interval. This is often modeled as a convolution – namely, we get samples of $y(t) = x(t) * h(t)$, so that the sampled signal is $y[n] = y(nT)$. In this case, $h(t)$ represents the impulse response of the sensor or digitizer. Actually, sometimes this averaging can be desirable. For example, if the original signal $x(t)$ is changing particularly rapidly compared to the sampling frequency or is particularly noisy, then obtaining samples of some averaged signal can actually provide a more useful signal with less variability. The second non-ideal effect is noise. Whether averaged or not, the actual sample value obtained will rarely be the exact value of the underlying analog signal at some time. Noise in the samples is often modeled as adding (usually small) random values to the samples.

Although in real applications there are usually non-ideal effects such as those mentioned above, it is important to consider what can be done in the ideal case for several reasons. The non-ideal effects are often sufficiently small that in many practical situations they can be ignored. Even if they cannot be ignored, the techniques for the ideal case provide insight into how one might deal with the non-ideal effects. For simplicity, we will usually assume that we get ideal, noise-free samples.

5.2 Reconstructing a Signal

In the previous section, we discussed the process of sampling, i.e., obtaining a discrete-time signal from a continuous-time signal. We now consider the reverse problem, namely how to construct a

continuous-time signal given discrete-time samples. Suppose we are given a set of samples $x[n]$ that we know came from some continuous-time signal $x(t)$. We also assume we know the sampling rate T , so that we know $x(nT) = x[n]$. How can we recover $x(t)$ for other values of t ?

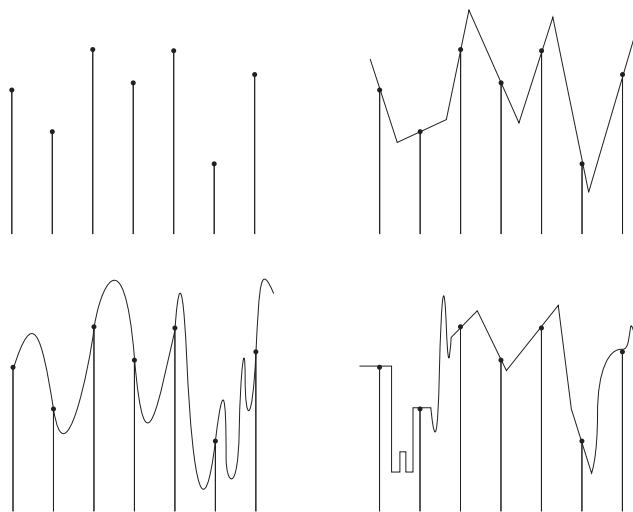


Figure 5.2: Possible continuous-time functions corresponding to samples.

Unless we have some additional knowledge or make some assumptions, this problem clearly has many solutions. Figure 5.2 shows some discrete-time samples and some possible continuous-time functions from which these samples could have been obtained. We have no way to know for sure what the values of the original signal are at times other than nT . However, there are some simple estimates we might make to approximately reconstruct $x(t)$.

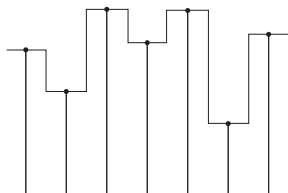


Figure 5.3: Nearest-neighbor reconstruction.

The first estimate one might think of is to just assume that the value at time t is the same as the value of the sample at some time nT that is closest to t . This *nearest-neighbor* interpolation results in a piecewise-constant (staircase-like) reconstruction as shown in Figure 5.3.

Actually, instead of nearest-neighbor interpolation, most devices implement a similar type of interpolation known as *zero-order-hold* interpolation shown in Figure 5.4. This is one of the most widely used methods and is easy to implement. As with nearest-neighbor interpolation, this results in a piecewise-constant reconstruction, but the discontinuities are at the sample points instead of between sample points.

Another conceptually simple method is *linear interpolation*, which is also called *first-order-hold* interpolation and is shown in Figure 5.5. With this method, the reconstruction is a continuous function that just connects the sample values with straight lines. Higher order interpolation schemes also exist that pass smoother functions through the samples. The higher the order, the more samples

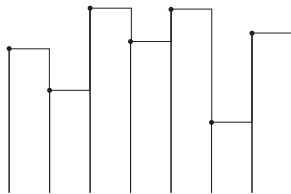


Figure 5.4: Zero-order hold reconstruction.

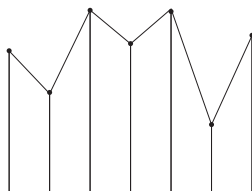


Figure 5.5: First-order hold reconstruction.

that need to be used to reconstruct a value at each time. One technique that actually uses *all* the samples will be mentioned in the next section.

These interpolation techniques can be viewed in terms of simple convolution operations. Suppose we replace the discrete-time signal with a continuous-time signal that has delta functions at the sample times nT . The height of (area under) each impulse is the value of the sample at that time. Then the reconstruction from zero-order-hold interpolation can be obtained by convolving this set of impulses with the function

$$h(t) = \begin{cases} 1 & \text{if } 0 \leq t < T \\ 0 & \text{otherwise} \end{cases}$$

In other words, the reconstruction is obtained by passing the discrete-time samples, as a set of impulses, into an appropriate system. In this case, the system has a very simple impulse response and can be easily implemented. Likewise, nearest neighbor and linear interpolation can be viewed as convolving the sample impulses by $\text{rect}(t/T)$ and $\text{tri}(t/T)$, respectively.

One useful property of zero-order-hold that is not satisfied by nearest-neighbor or first-order-hold interpolation is that it is *causal*. This means that the reconstruction at time t only depends on samples obtained *before* time t . Causality can be important in real-time systems when we must (or wish to) reconstruct the continuous-time signal without delays caused by waiting for future sample values. In many applications this is not an issue since either all the data is already stored and available or the delays induced are minimal.

5.3 The Sampling Theorem

In this section we discuss the celebrated Sampling Theorem, also called the Shannon Sampling Theorem or the Shannon-Whitaker-Kotelnikov Sampling Theorem, after the researchers who discovered the result. This result gives conditions under which a signal can be *exactly* reconstructed from its samples. The basic idea is that a signal that changes rapidly will need to be sampled much faster than a signal that changes slowly, but the sampling theorem formalizes this in a clean and elegant way. It is a beautiful example of the power of frequency domain ideas.

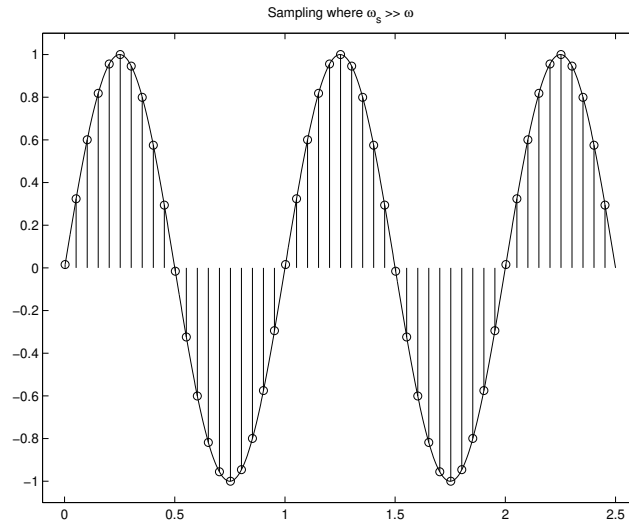


Figure 5.6: Sampling a sinusoid at a high rate.

To begin, consider a sinusoid $x(t) = \sin(\omega t)$ where the frequency ω is not too large. As before, we assume ideal sampling with one sample every T seconds. In this case, the sampling frequency in Hertz is given by $f_s = 1/T$ and in radians by $\omega_s = 2\pi/T$. If we sample at rate fast compared to ω , that is if $\omega_s \gg \omega$ then we are in a situation such as that depicted in Figure 5.6. In this case, knowing that we have a sinusoid with ω not too large, it seems clear that we can exactly reconstruct $x(t)$. Roughly, the only unknowns are the frequency, amplitude, and phase, and we have many samples in one period to find these parameters exactly.

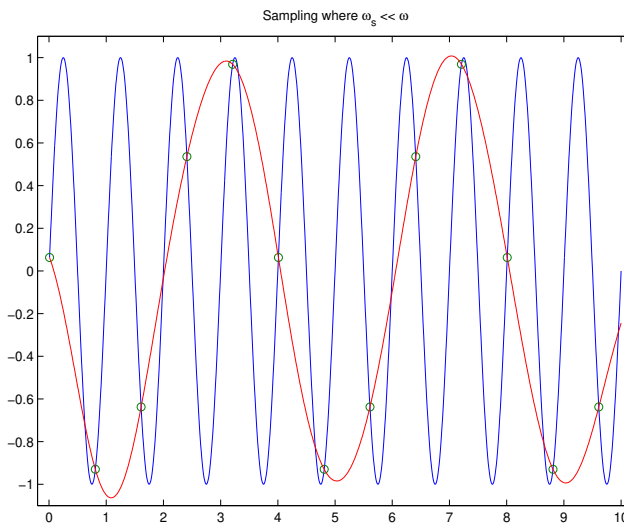


Figure 5.7: Sampling a sinusoid at too slow of a rate.

On the other hand, if the number of samples is small with respect to the frequency of the sinusoid then we are in a situation such as that depicted in Figure 5.7. In this case, not only does it seem difficult to reconstruct $x(t)$, but the samples actually appear to fall on a sinusoid with

a lower frequency than the original. Thus, from the samples we would probably reconstruct the lower frequency sinusoid. This phenomenon is called *aliasing*, since one frequency appears to be a different frequency if the sampling rate is too low. It turns out that if the original signal has frequency f , then we will be able to exactly reconstruct the sinusoid if the sampling frequency satisfies $f_S > 2f$, that is, if we sample at a rate faster than twice the frequency of the sinusoid. If we sample slower than this rate then we will get aliasing, where the alias frequency is given by

$$f_a = |f_s - f|.$$

Actually, we need to be a little bit careful in our thinking. No matter how fast we sample, there are always sinusoids of a sufficiently high frequency for which the sampling rate we happen to be using is too low. So, even in the case of Figure 5.6, where it seems we have no problem recovering the sinusoid, we can't be sure that the true sinusoid is not one of a much higher frequency and we are not sampling fast enough. The way around this problem is to assume from the outset that the sinusoids under consideration will have a frequency no more than some maximum frequency f_B . Then, as long as we sample faster than $2f_B$, we will be able to recover the original sinusoid exactly.

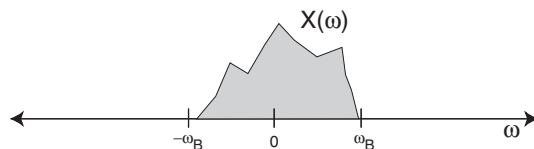


Figure 5.8: The Fourier Transform of a bandlimited signal.

The results for sinusoids would be of limited interest unless we had some way of extending the ideas to more general signals. The natural way to extend the ideas is to use frequency domain analysis and the Fourier transform. A signal $x(t)$ is said to be *bandlimited* if there is some frequency $\omega_B < \infty$ such that

$$X(\omega) = 0 \text{ for } |\omega| > \omega_B$$

That is, a signal is bandlimited if there is some maximum frequency beyond which its Fourier transform is zero. This is shown pictorially in Figure 5.8. This maximum frequency ω_B is the bandlimit of the signal, and can also be expressed in Hertz as

$$f_B = \frac{\omega_B}{2\pi}.$$

If we could recover each sinusoidal component of $x(t)$ then we would be done. To recover a sinusoid at frequency ω we need to sample at a rate at least 2ω . Thus, as long as the various sinusoids can be treated independently, we might guess that for a bandlimited signal we would be able to reconstruct $x(t)$ exactly if we sample at a rate above $2\omega_B$. This is in fact the case, which is a statement of the sampling theorem.

Sampling Theorem A bandlimited signal with maximum frequency ω_B can be perfectly reconstructed from samples if the sampling frequency satisfies $\omega_S > 2\omega_B$ (or equivalently, if $f_S > 2f_B$).

The frequency $2\omega_B$, or $2f_B$ in Hertz, is called the *Nyquist rate*. Thus, the sampling theorem can be rephrased as: a bandlimited signal can be perfectly reconstructed if sampled above its Nyquist rate.

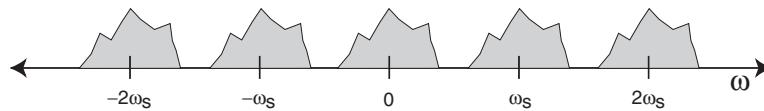


Figure 5.9: The Fourier transformation of a sampled signal consists of shifted replicas of $X(\omega)$.

In addition to just stating that perfect reconstruction can be achieved, formulas for *how* to reconstruct $x(t)$ can also be obtained. These fall out from the proof of the sampling theorem. Rather than give a precise proof and derivation, we describe the basic idea. When a signal is sampled, we can ask what happens in the frequency domain. It turns out that when a signal $x(t)$ is sampled, the Fourier transform of the sampled signal consists of shifted and scaled copies of $X(\omega)$, as shown in Figure 5.9. The spacing between the replicas of $X(\omega)$ depends on the sampling rate, so that the faster we sample, the further apart are the replicas of $X(\omega)$. Specifically, if we take one sample every T seconds then the spacing between the replicas in frequency domain is equal to the sampling frequency $\omega_s = 2\pi/T$. Alternatively, in Hertz, the replicas will be $f_s = 1/T$ apart. The scaling factor is $1/T$, so the replicas are actually $\frac{1}{T}X(\omega)$ rather than $X(\omega)$ itself, but this is not a problem since we can easily undo this scaling.

Thus, we can see that if we sample fast enough, the replicas will be non-overlapping. Then, in principle we could isolate the one un-shifted copy of $X(\omega)$ and take the inverse Fourier transform to recover $x(t)$. To get non-overlapping replicas, we see that for a bandlimited signal $x(t)$ with maximum frequency ω_B , we will need to sample faster than the Nyquist rate $2\omega_B$. In this case, we can get rid of the extra shifted copies of $X(\omega)$ by multiplying by $\text{rect}(\omega/(2\omega_B))$ in frequency domain. Actually, if the sampling frequency is larger than $2\omega_B$, a rect function with even a slightly larger width could also be used, as long as the width of the rect is not so large as to include any parts of other replicas. We will also multiply by T to get $X(\omega)$ instead of $\frac{1}{T}X(\omega)$.

Now, what we have done is to multiply the spectrum of the sampled signal by $\frac{1}{T}\text{rect}(\omega/(2\omega_B))$ in frequency domain to get $X(\omega)$. Then $x(t)$ can be recovered by an inverse Fourier transform. It's also useful to consider what operations are done in the time domain in the process of recovering $x(t)$. Recall that multiplication of two signals in the frequency domain corresponds to a convolution in the time domain – specifically, in this case a convolution of the sampled signal with the sinc function (since the Fourier transform of a sinc function is the rect function).

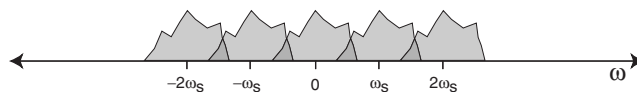


Figure 5.10: Sampling too slowly causes aliasing.

As with pure sinusoids, if a signal is sampled too slowly, then aliasing will occur. High frequency components of the signal will appear as components at some lower frequency. This can also be illustrated nicely in frequency domain. If sampled too slowly, the replicas of $X(\omega)$ overlap and the Fourier transform of the sampled signal is the sum of these overlapping copies of $X(\omega)$. As shown in Figure 5.10, high frequency components alias as lower frequencies and corrupt the unshifted copy of $X(\omega)$. In this figure, the copies are shown individually, with the overlap region simply shaded more darkly. What actually happens is that these copies add together so that we are unable to know what each individual copy looks like. Thus we are unable to recover the original $X(\omega)$, and hence cannot reconstruct the original $x(t)$.

5.4 Quantization

Quantization makes the range of a signal discrete, so that the quantized signal takes on only a discrete, usually finite, set of values. Unlike sampling (where we saw that under suitable conditions exact reconstruction is possible), quantization is generally irreversible and results in loss of information. It therefore introduces distortion into the quantized signal that cannot be eliminated.

One of the basic choices in quantization is the number of discrete quantization levels to use. The fundamental tradeoff in this choice is the resulting signal quality versus the amount of data needed to represent each sample. Figure 5.11 from Chapter 2, repeated here, shows an analog signal and quantized versions for several different numbers of quantization levels. With L levels, we need $N = \log_2 L$ bits to represent the different levels, or conversely, with N bits we can represent $L = 2^N$ levels.

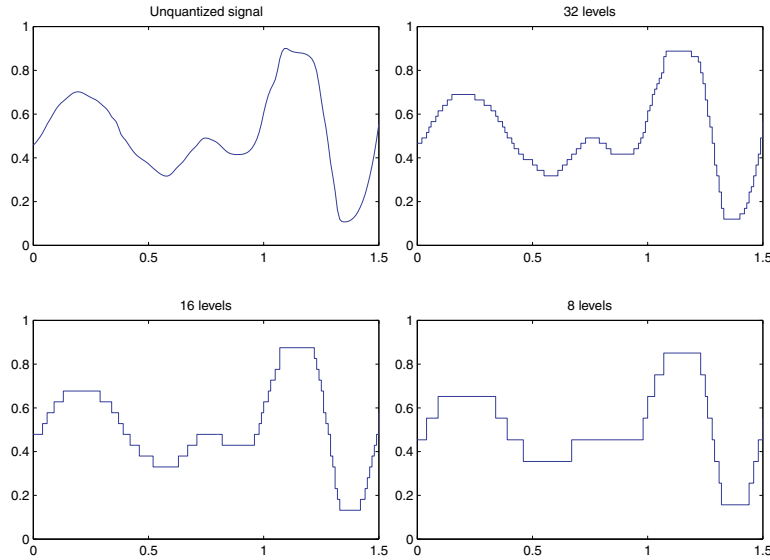


Figure 5.11: Quantized versions of an analog signal.

The simplest type of quantizers are called *zero memory* quantizers in which quantizing a sample is independent of other samples. The signal amplitude is simply represented using some finite number of bits independent of the sample time (or location for images) and independent of the values of neighboring samples. Zero memory quantizers can be represented by a mapping from the amplitude variable x to a discrete set of quantization levels $\{r_1, r_2, \dots, r_L\}$.

The mapping is usually based on simple thresholding or comparison with certain values, t_k . The t_k are called the transition levels (or decision levels), and the r_k are called the reconstruction levels. If the signal amplitude x is between t_k and t_{k+1} then x gets mapped to r_k . This is shown in Figure 5.12.

The simplest zero memory quantizer is the uniform quantizer in which the transition and reconstruction levels are all equally spaced. For example, if the output of an image sensor takes values between 0 and M , and one wants L quantization levels, the uniform quantizer would take

$$r_k = \frac{kM}{L} - \frac{M}{2L}, \quad k = 1, \dots, L$$

$$t_k = \frac{M(k-1)}{L}, \quad k = 1, \dots, L+1$$

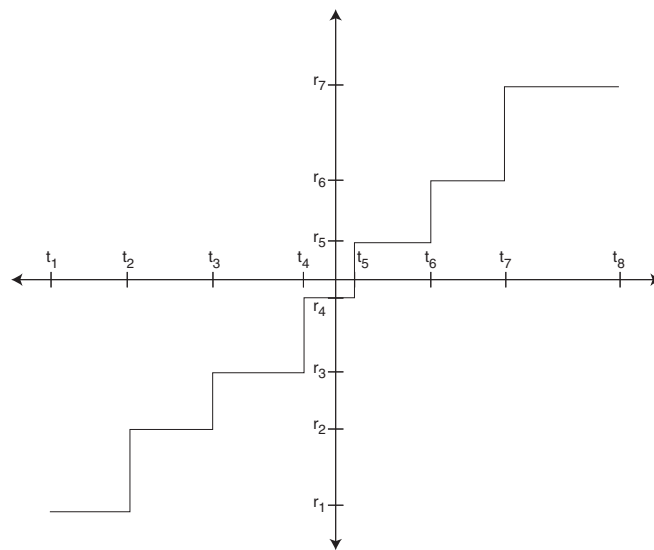


Figure 5.12: A zero-memory quantizer.

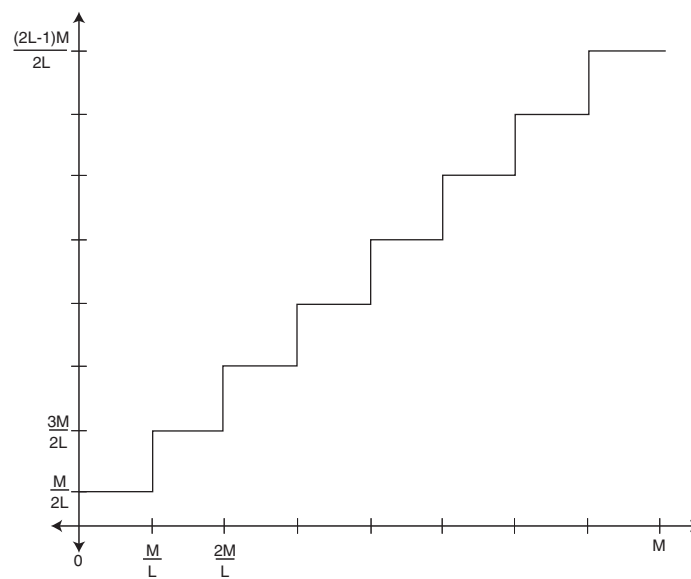


Figure 5.13: A uniform quantizer.

Figure 5.13 shows the reconstruction and transition levels for a uniform quantizer.

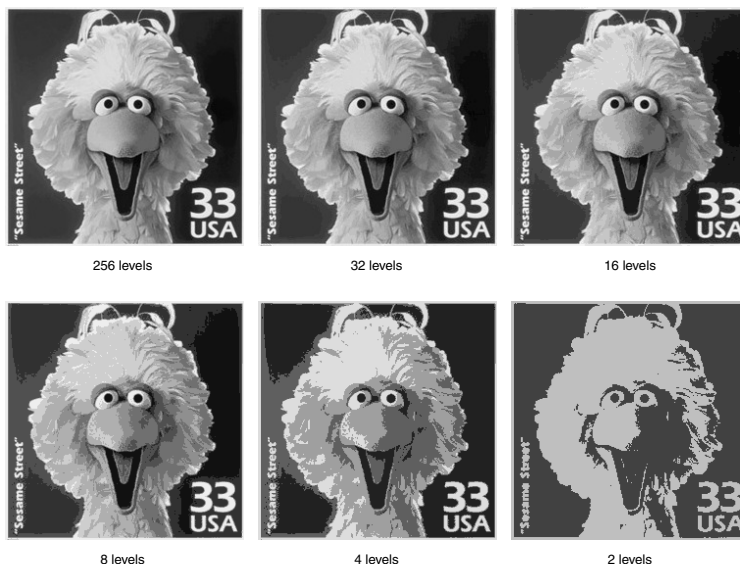


Figure 5.14: Quantizing a gray-level image.

Figure 5.14 show an example of the effects of reducing the number of bits to represent the gray levels in an image using uniform quantization. As fewer quantization levels are used, there is a progressive loss of spatial detail. Furthermore, certain artifacts such as *contouring* (or *false contouring*) begin to appear. These refer to artificial boundaries which become visible due to the large and abrupt intensity changes between consecutive gray levels. Using uniform quantization, images we usually start seeing false contours with 6 or fewer bits/pixel, i.e., about 64 or fewer gray levels. In audio signals, we can often hear distortion/noise in the signal due to quantization with about 8 bits per sample, or 256 amplitude levels. Of course, these figures depend on the particular image or signal. It turns out that with other quantization methods, as discussed in the following sections, objectionable artifacts can be reduced for a given number of levels.

5.5 Nonuniform Quantization

It is natural to expect that the amount of distortion introduced depends on the quantizer, and so one can try to minimize the distortion by choosing a “good” quantizer. For a zero memory quantizer, once we fix the number of levels (or bits per sample), the only choice available in designing a quantizer are the values for the transition and reconstruction levels. With a judicious choice of the quantization levels, however, we may be able to do better than blindly taking the levels uniformly spaced.

For example, if an image never takes on a certain range of gray levels then there is no reason to waste quantization levels in this range of gray levels. One could represent the image more accurately if these quantization levels were instead used to more finely represent gray levels that appeared often in the image.

One way to quantify this idea is to look at the histogram of intensity values of the original signal. This is a plot where the x -axis is the intensity level and the y -axis is the number of samples in the signal that take on that intensity level. The histogram just shows the distribution of intensity

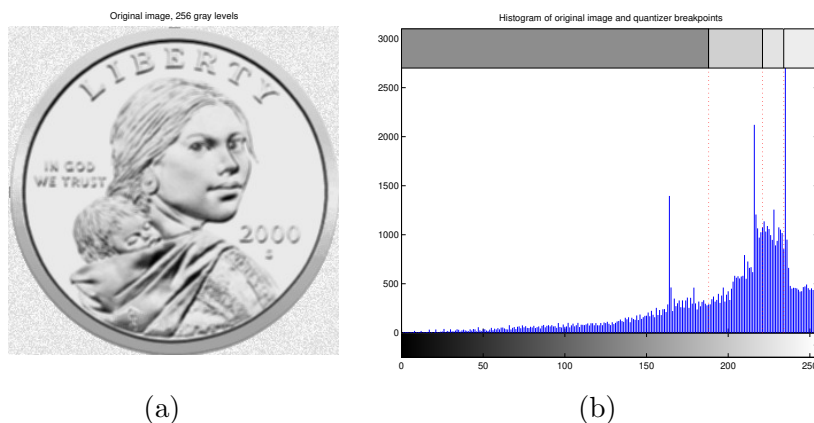


Figure 5.15: (a) Original image. (b) Image histogram.

levels. Levels that occur often result in large values of the histogram. From the histogram, we can often get a sense of where we should place more quantization levels (places where the histogram is large) and where we should place few levels (where the histogram is small). Figure 5.15b shows the histogram of gray levels for the image of Figure 5.15a. With just 4 levels, by looking at the histogram we might place them at 141, 208, 227, and 237.

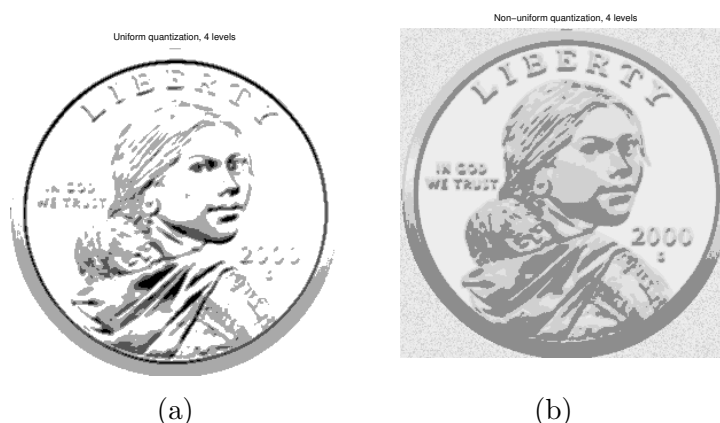


Figure 5.16: (a) Uniformly quantized image. (b) Non-uniformly quantized image

The resulting quantized image is shown in Figure 5.16b which compares very favorably with the image of Figure 5.16a where 4 uniform levels were used.

It turns out that if the distribution is known, then one can formulate the problem of selecting the “optimal” choice of levels (in terms of mean squared error). For the special case that the histogram is flat, for which all gray levels occur equally often, the optimum quantizer turns out to be the uniform quantizer, as one might expect. However, in general the equations for the optimum quantizer are too difficult to solve analytically. One alternative is to solve the equations numerically, and for some common densities, values for the optimal transition and reconstruction levels have been tabulated. Also, various approximations and suboptimal (but easy to implement) alternatives to the optimal quantizer have been developed. Although they are suboptimal, these can still give results better than simple uniform quantization.

If quantization levels are chosen to be tailored to a particular image, then when storing the image, one needs to make sure to also store the reconstruction levels so that the image can be displayed properly. This incurs a certain amount of overhead, so that usually one would not choose a different quantizer for each image. Instead, we would choose quantization levels to work for a class of images that we expect to encounter in a given application. If we know something about the images and lighting levels to be expected, then perhaps a non-uniform quantizer may be useful. If not, it may turn out that the most reasonable thing to do is just stick to the uniform quantizer. With increasing capacity and decreasing cost of memory and communication, such issues decrease in importance.

A different rationale for choosing non-uniform quantization levels involves properties of human perception. As we will discuss in Chapter 6, many human perceptual attributes satisfy Weber's law, which states that we are sensitive to *proportional* changes in the amplitude of a stimulus, rather than in absolute changes. For example, in vision, humans are sensitive to *contrast* not absolute luminance and our contrast sensitivity is proportional to the ambient luminance level. Thus for extremely bright luminance levels there is no reason to have gray levels spaced as close together as for lower luminance levels. Therefore, it makes sense to quantize uniformly with respect to contrast sensitivity, which is logarithmically related to luminance. This provides a rationale for selecting the logarithm of the quantization levels to be equally spaced, rather than the levels themselves being uniform. With this choice there are no claims for even approximating any "optimal" quantizer in general. The justification for ignoring the optimal quantizer is that what is *visually* pleasing does not always correspond to minimizing the mean square error. We will see this again in the next section when we discuss dithering and halftoning.

5.6 Dithering and Halftoning

Dithering As we have seen, coarse quantization often results in the appearance of abrupt discontinuities in the signal. In images, these appear as false contours. One technique used to try to alleviate this problem is called *dithering* or *pseudorandom noise quantization*. The idea is to add a small amount of random noise to the signal *before* quantizing. Sometimes same noise values are stored and then subtracted for display or other purposes, although this is not necessary.

The idea of adding noise intentionally might seem odd. But in the case of dithering, the noise actually serves a useful purpose. Adding noise before quantization has the effect of breaking up false contours by randomly assigning pixels to higher or lower quantization levels. This works because our eyes have limited spatial resolution. By having some pixels in a small neighborhood take on one quantized level and some other pixels take on a different quantized level, the transition between the two levels happens more gradually, as a result of averaging due to limited spatial resolution.

Of course, the noise should not be too large since other features of the signal can also get disrupted by the noise. Usually the noise is small enough to produce changes to only adjacent quantization levels. Even for small amounts of noise, however, there will be some loss in spatial resolution. Nevertheless, the resulting signal is often more desirable. Although dithering in images will introduce some dot-like artifacts, the image is often visually more pleasing since the false contours due to the abrupt transitions are less noticeable.

It is interesting to note that, according to certain quantitative measures, adding noise makes the resulting quantized signal further from the original than quantizing without the noise, as we might expect. For example, one standard measure for the similarity between two signals is the mean squared error. To compute this criterion, one computes the squared difference between corresponding values of two signals, and takes the average of this squared difference. It can be

shown that picking the quantization level that is closest to the signal value is the best one can do as far as mean squared error is concerned. Adding noise and then quantizing will only increase the mean squared error, yet we've seen that this can result in a perceptually more pleasing signal. This shows that standard quantitative criteria such as mean square error do not necessarily reflect subjective (perceived) quality.

Halftoning An interesting topic related to dithering is *halftoning*, which refers to techniques used to give a gray scale rendition in images using only black and white pixels. The basic idea is that for an oversampled image, a proper combination of black and white pixels in a neighborhood can give the perception of gray scales due to spatial averaging of the visual system. For example, consider a 4×4 block of pixels. Certain average gray levels for the entire 4×4 block can be achieved even if the 16 pixels comprising the block are allowed to be only black or white. For example, to get an intermediate gray level we need only make sure that 8 pixels are black and 8 are white. Of course, there are a number of specific patterns of 8 black and 8 white pixels. Usually, to allow a gray scale rendition to as fine a spatial resolution as possible, one would avoid using a pattern where all 8 white pixels are grouped together and all 8 black pixels are grouped together. Also, one usually avoids using some fixed pattern for a given gray scale throughout the image since this can often result in the appearance of undesirable but noticeable patterns. A common technique to avoid the appearance of patterns is to select the particular arrangement of black and white pixels for a given gray scale rendition pseudorandomly. Often one repeats a “noise matrix” (also called a halftone matrix or dither matrix). Almost all printed images in books, newspapers, and from laser printers are done using some form of halftoning.

Chapter 6

The Human Auditory and Visual Systems

In this chapter, some basic properties of the human auditory and visual systems are discussed. Understanding properties of human perception can help in designing algorithms and devices where either the end user is a human or even when there is a human somewhere in the loop. Many systems dealing with some aspect of multimedia fall into this category – for example television, computer monitors, cell phones, digital cameras, other imaging technologies (for medicine, surveillance, etc.), and printing technology. With a human as the ultimate user of the data processed by these systems, the technology can be tailored to suit the capabilities and limitations of humans.

There is a second way in which understanding properties of human, or more generally biological, systems can help in designing engineering systems. For systems whose goals are similar to tasks carried out by biological systems, it is natural to consider “biologically-inspired” engineering designs. This is particularly true in trying to design “intelligent” systems where the tasks are extremely difficult and relatively little is understood about how best to proceed. Rather than designing human interfaces, the goal of such systems is often to perform tasks with little or no human intervention. Understanding how biological systems function can be a valuable guide.

A great deal is known about the structure and function of the human auditory and visual systems. In the following sections, only a very brief account of some properties most relevant to us are discussed.

6.1 The Human Ear

The human ear consists of three main components called the external (or outer) ear, the middle ear, and the inner ear. The external ear consists of the pinna (the visible flesh we informally called the ear), the outer ear canal, and the eardrum (or tympanic membrane). The external ear gathers sound (variations in air pressure), carries them through the outer ear canal, and to the eardrum, which separates the outer ear from the middle ear.

The middle ear consists of three small bones called the malleus (hammer), incus (anvil), and stapes (stirrup). These convert minute vibrations of the eardrum into amplified movements of the stapes. This in turn causes motion of fluid in the inner ear.

*©2023 by Sanjeev R. Kulkarni. All rights reserved. Please do not distribute without permission of author.

[†]Lecture Notes for ECE201 Information Signals.

[‡]Thanks to Richard Radke for producing the figures.

The inner ear consists of a fluid-filled spiral structure with about $2\frac{1}{2}$ turns that is separated into three compartments called the scala tympani (tympenic canal), scala vestibuli (vestibular canal), and the scala media (cochlear duct). The floor of the cochlear duct is the basilar membrane that holds the organ of Corti. The organ of Corti consists of an intricate network of hair cells and supporting cells. The hair cells convert the mechanical "signal" of fluid motion into an electrical signal.

The cochlea seems to have frequency selectivity. That is, different parts of the cochlea respond to different frequencies in the original air pressure variations (which in turn correspond to different frequencies of the sound). If the cochlea was unwound, the portion connected to the middle ear is stiffer and responds to higher frequencies, while the other end is more flexible and responds to lower frequencies. Exactly how this is done is not well understood, but remarkably the human ear is effectively doing a kind of Fourier transform!

6.2 Properties of the Auditory System

Various properties of the auditory system are important in the design of technology dealing with sound.

Frequency Range Humans are sensitive to frequencies in range of about 15 Hz to 20,000 Hz. Actually, these are somewhat optimistic figures and many people (especially as they age) have a range smaller than this, perhaps more like 20 Hz to 16,000 Hz. This frequency range immediately impacts design choices such as the sampling rate in digital audio. For example, we know that to avoid aliasing, we should sample at a rate faster than twice the highest frequency present in the signal. Although, many sounds may have frequencies higher than 20,000 Hz, we can first remove these frequencies without altering the perceptual quality of the sound to humans (by filtering as discussed later in Chapter 7). Then, the highest frequency will be 20,000 Hz. In this case, sampling faster than 40,000 Hz will retain all the information necessary to fully reconstruct the filtered signal. The sampling frequency of digital audio is typically 44,100 Hz, which was chosen with human perceptual properties in mind. Oversampling (that is, choosing a frequency larger than the Nyquist rate) not only provides a buffer, but also simplifies processing in reconstructing an analog signal from samples.

Frequency Discrimination This refers to the ability to distinguish sounds at two different frequencies. The discrimination ability of humans is about 3 or 4 Hz for sounds with frequencies between 15 Hz and 2,000 Hz. Above 2,000 Hz, the change in frequency needed for discrimination is about 0.3 % of the frequency of the sound. So for a sounds near 10,000 Hz, the frequencies would have to differ by about $(0.003 \times 10,000) = 30$ Hz to be distinguishable. This gives about 600 distinguishable frequencies in the range of 15 Hz to 2,000 Hz, and about 720 distinguishable frequencies between 2,000 Hz and 16,000 Hz. Thus, humans can distinguish over 1,300 frequencies.

Sensitivity This refers to the weakest signal that is just audible at a given frequency. A standard way to measure sensitivity is with respect to a intensity of a reference signal. Humans are most sensitive to sounds in the frequency range between 1,000 Hz and 3,000 Hz. If we let P_r denote the intensity of the weakest signal in this range, then we can measure the sensitivity at other frequencies with respect to P_r . Because the dynamic range is so huge, it is standard to compare intensities to

P_r on a logarithmic scale. That is, we define the intensity of P_t in *decibels* (or dB) as

$$\text{intensity of } P_t \text{ in dB} = 20 \log_{10} \frac{P_t}{P_r}$$

By definition, P_r is at 0 dB, and physically this corresponds to about 0.2×10^{-9} of atmospheric pressure. (Atmospheric pressure is about 14.7 pounds per square inch.) Every 20 dB corresponds to a factor of 10 in signal amplitude. From the figure we can see that the sensitivity of human hearing varies by several orders of magnitude as a function of frequency.

Amplitude Discrimination This refers to the ability to distinguish changes in amplitude. Regardless of the level, humans can generally detect about a 3 dB change in intensity. However, since dB measures the log of the ratio of intensity with respect to the reference intensity, we need about the same *proportional* change in intensity to be able to just detect the change (rather than the same additive change in intensity). Specifically, for a given intensity, the just-noticeable new intensity satisfies

$$20 \log_{10} \frac{\text{just noticeable new intensity}}{\text{intensity}} = 3 \text{ dB}$$

Therefore,

$$\log_{10} \frac{\text{just noticeable new intensity}}{\text{intensity}} = 0.15$$

so that

$$\text{just - noticeable new intensity} \approx 1.4 \times \text{intensity}$$

Equivalently, we can write

$$\frac{\text{just noticeable change in intensity}}{\text{intensity}} = 0.4$$

This type of relationship (that the just-noticeable change divided by intensity being constant) is quite common in perceptual phenomena and is often referred to as Weber's Law or the Weber-Fechner Law.

Amplitude Range This refers to the loudest sound we can hear without pain compared with the weakest detectable sound (which has intensity P_r). The amplitude range is about 120 dB. This means that the loudest sound (without pain) is about 10^6 times the intensity of P_r , which is a huge dynamic range.

6.3 The Human Eye

Light enters through the lens and falls on the retina. Receptors in the retina are sensitive to light of wavelengths approximately 350-750 nm, and can operate over roughly 10 orders of magnitude in illumination, which is a remarkable dynamic range.

There are two types of photoreceptors in the human retina: rods and cones. The rods are long and thin and are primarily responsible for *scotopic vision*, which refers to visual capabilities in low illumination levels.

The cones are shorter and thicker than the rods and are responsible for *photopic vision*, vision in high illumination levels. The cones are also responsible for color vision, which is discussed further in Section 6.6.

The rods and cones are connected to various types of other cells which join to form the optic nerve that connects to higher brain centers.

6.4 Luminance Versus Brightness

Clearly the “brightness” perceived at a point is related to the amount of light falling on the receptors corresponding to that point. However, it turns out the *perceived* brightness at a point is also a function of the “brightness” of neighboring points. Thus a distinction needs to be made between our *perception* of brightness, which is a subjective quantity, and the actual energy incident on the receptors at a point and the resulting response of the receptors, which can be precisely defined.

Although the retina is spherical and there are not receptors at every point on the retina, for simplicity a point on the retina will be denoted by (x, y) and we will talk about the energy falling on a small retinal patch at (x, y) . The energy which falls on a retinal patch at a given point is normally light which has been reflected off of some object patch in the visual scene. This energy is a function of a number of quantities such as the amount of light falling on the object patch, the reflectivity and geometry of the object patch, the amount of light collected by the optical system (namely the eye), etc. The energy falling on a retinal patch at (x, y) is called the *irradiance* and will be denoted $E(x, y)$. The irradiance evokes a response in the receptors in the retinal patch, and the response is called the *luminance* at (x, y) , denoted $I(x, y)$. The receptors respond differently to light at different wavelengths. The response can be characterized by a function of wavelength $V(\lambda)$ which is called the *luminous efficiency function*. The luminance at (x, y) is given by

$$I(x, y) = \int_0^\infty E(x, y, \lambda) V(\lambda) d\lambda$$

where $E(x, y, \lambda)$ is now the irradiance at wavelength λ .

From the expression above, we see that the luminance at a point is independent of the luminance at surrounding points. On the other hand, there are two standard phenomena which show that the perceived brightness (or apparent brightness, or simply brightness) at a point depends not only on the luminance at that point, but also on the luminance at neighboring points. These phenomena are called Mach bands and simultaneous contrast, and they illustrate nicely the difference between luminance and brightness.

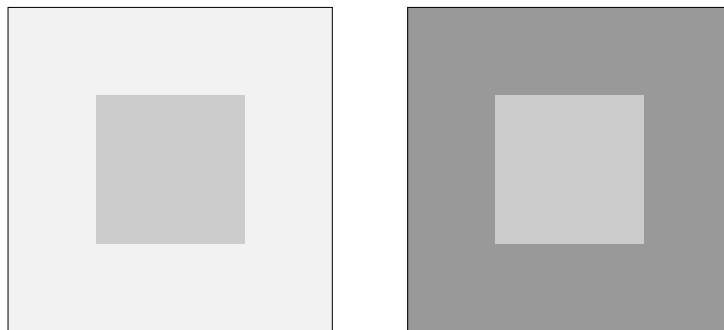


Figure 6.1: Simultaneous contrast.

Figure 6.1 illustrates the simultaneous contrast phenomenon. The two inner squares have the same luminance but the one on the right appears brighter. This illustrates the fact that our perception of brightness depends on luminance *contrasts* as well as absolute luminance values. Let I_0 be the luminance of object, and let $I_s = I_0 + \Delta I$ be the luminance of surround which results in

a just-noticeable difference in intensity. Weber’s Law states that

$$\frac{|\Delta I|}{I_0} = \text{constant}$$

For just noticeable differences

$$\frac{\Delta I}{I} \approx d(\log I) = \text{constant}$$

Thus, the change in intensity required for a just-noticeable change in contrast depends logarithmically on the ambient intensity level.

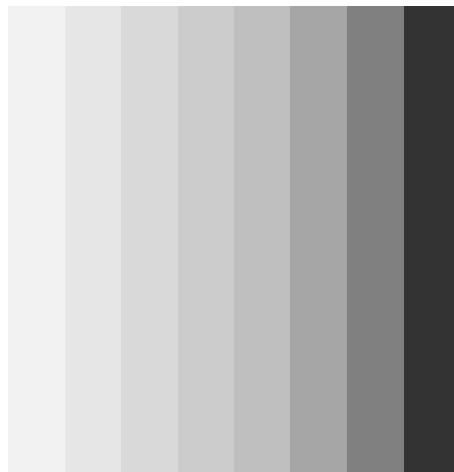


Figure 6.2: Mach bands.

Figure 6.2 illustrates the Mach bands phenomenon. Each band has constant luminance, but one perceives overshoots and undershoots in brightness in the neighborhood of the band boundaries. This phenomenon can be explained by the “on-center off-surround” or “lateral inhibition” of visual system.

6.5 Spatial and Temporal Properties

Acuity refers to the ability of the eye to detect fine spatial detail. The response of the visual system to sinusoids has been extensively studied. The ability to resolve spatial detail depends not only on the actual physical dimensions of the object being viewed, but also on the distance of the object to the observer. The important quantity is the angle over which the object extends as measured from the eye — i.e. the physical dimension of the detail per degree. That the response of the human visual system to sinusoidal patterns is greatest for approximately 3-10 cycles/degree.

Bloch’s Law Light flashes of different duration but same energy are indistinguishable for durations less than approximately 30 ms for moderate light levels. This duration is somewhat larger when eye is adapted to low light levels.

Critical Fusion Frequency A light flashing at a rate faster than a certain rate called the Critical Fusion Frequency (CCF) is indistinguishable from a steady light of same average intensity. The CCF is generally around 50-60 Hz. This has implications on refresh rates required in image display devices, such as televisions or computer monitors, to prevent flickering.

6.6 Color Perception and Representation

Most of the work done in image processing deals only with gray-level images. However, color is becoming increasingly important in image processing and it is worthwhile to know the basics of color for several reasons. The development of new technology in sensors, display, and printing is making the use of color more widespread. In some applications color can be a powerful cue for various image analysis tasks such as edge detection and image segmentation. Another important area is the use of *pseudo-color* for the enhancement of gray scale images. Gray levels can be mapped to different colors in order to bring out or emphasize particular features.

Monochromatic light refers to light of a single wavelength. *Chromatic* or *colored* light refers to light which has energy distributed at various wavelengths. To completely specify a chromatic light source, one needs to specify the energy distribution as a function of wavelength, which is referred to as the *spectral characteristics* or the *spectral distribution* of the light.

However, we know that we can get “all colors” by combining three primary colors — red (R), green (G), and blue (B). If the primary colors are modeled as monochromatic light sources at three different wavelengths (corresponding to R, G, B), then mixing colors would still only allow light with energy at these three wavelengths. How is it possible to generate all colors by combining different levels of R, G, B since we certainly can’t generate all spectral distributions this way? The reason is that there are only three different types of cones which are responsible for color perception.

The three cones have different absorption spectra $S_1(\lambda)$, $S_2(\lambda)$, $S_3(\lambda)$ which are shown in Figure 6.3. The absorption spectrum $S_3(\lambda)$ peaks in the blue region of the spectrum and so corresponds to B, $S_2(\lambda)$ corresponds to G, and $S_1(\lambda)$ corresponds to R (although its absorption spectrum actually peaks in the yellow-green region of the spectrum).

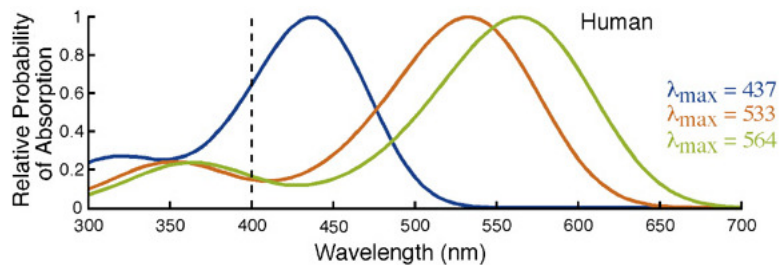


Figure 6.3: Absorption spectra of human cones.

Given any spectral distribution of light, $C(\lambda)$, the color sensation produced depends only on the spectral responses of each the three types of cones. The response of the i -th receptor ($i = 1, 2, 3$) can be written as $\alpha_i(C) = \int S_i(\lambda) C(\lambda) d\lambda$. Any light sources that give the same responses α_i for each of the three receptors will be perceived to be identical, even though the actual spectral distributions of the light sources might be different.

Color Matching and Reproduction Given three light sources, how can we match or reproduce arbitrary colors? Let $P_1(\lambda), P_2(\lambda), P_3(\lambda)$ be three sources called primary sources. We want to match a color $C(\lambda)$ using a combination of these three sources. That is we want to find constants β_k such that

$$\sum_{k=1}^3 \beta_k P_k(\lambda)$$

is perceived to be the same color as $C(\lambda)$. From our discussion above, we only require that the responses of each of the three receptors be the same. Therefore, we want

$$\begin{aligned} \alpha_i(C) &= \int \left[\sum_{k=1}^3 \beta_k P_k(\lambda) \right] S_i(\lambda) d\lambda \\ &= \sum_{k=1}^3 \beta_k \int S_i(\lambda) P_k(\lambda) d\lambda \\ &= \sum_{k=1}^3 \beta_k a_{i,k} \end{aligned}$$

where $a_{i,k}$ is the response of the i -th receptor type to the k -th primary light source. Recall that $\alpha_i(C) = \int S_i(\lambda) C(\lambda) d\lambda$ is the response of the i -th receptor to the source $C(\lambda)$.

In matrix form, we have

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \begin{bmatrix} \beta_1 \\ \beta_2 \\ \beta_3 \end{bmatrix} = \begin{bmatrix} \alpha_1(C) \\ \alpha_2(C) \\ \alpha_3(C) \end{bmatrix}$$

Thus, $\bar{\beta} = A^{-1} \bar{\alpha}(C)$ where A^{-1} is the inverse of the matrix $A = [a_{i,k}]$.

Formally this gives us the solution to our original question. To reproduce the color $C(\lambda)$ we use the k -th primary source with intensity β_k . However, note that for some colors $C(\lambda)$ the formal solution may result in negative β_k . In this case, it means that with the sources P_1, P_2, P_3 we cannot physically generate colors with negative β_k 's. It would be nice to find three primary sources for which *all* colors are producible — i.e., result in positive β_k . However, there is no known set of such primary sources. Two natural attempts at selecting primaries which will allow the reproduction of all colors are (1) let $P_i(\lambda) = S_i(\lambda)$ or (2) Define P'_i 's by requiring the β' 's to equal $\alpha_i(C)$. In the first case, it can be shown that for some colors at least one of the β_k will be negative. In the second case, it can be shown that the $P_i(\lambda)$ are not physically realizable since at least one of the $P_i(\lambda)$ will be negative for some λ . One can attempt to implicitly define a set of primaries such that the β_k are always positive, but again the resulting primary sources will not be physically realizable. Thus, one has to settle for a set primary sources which allow reproducing a sufficiently rich set of colors. The set of colors producible by a set of primaries is called the *gamut* of the primary sources.

The β_k can be thought of as a representation for a color $C(\lambda)$ in terms of primaries $P_1(\lambda), P_2(\lambda), P_3(\lambda)$. One common standard set of primary sources is the C.I.E. (International Committee on Color Standards) standard primary sources in which the $P_i(\lambda)$ consist of monochromatic sources corresponding to R, G, and B. Specifically, the sources are $P_i(\lambda) = \delta(\lambda - \lambda_i)$ where $\lambda_1=700$ nm (R), $\lambda_2=546.1$ nm (G), and $\lambda_3=435.8$ nm (B).

Often, rather than using the β_k directly to represent colors, the primary sources are calibrated against a reference white light source $W(\lambda)$. In this case, a color $C(\lambda)$ can be represented by the

normalized values

$$T_k(C) = \frac{\beta_k}{w_k}$$

where w_k is the amount of the k -th primary needed to match the reference white light source $W(\lambda)$. These normalized values are called the *tristimulus values* of the color $C(\lambda)$.

Chromaticity Diagram Another alternative representation that is commonly used are the *chromaticities* of a color which are defined by

$$t_k = \frac{T_k}{T_1 + T_2 + T_3}$$

However, note that $t_1 + t_2 + t_3 = 1$, so that only two of the chromaticities are independent. To get a complete representation of the color, two of the chromaticities need to be augmented with another parameter. This third parameter is the *luminance* of the color which is defined as

$$Y = \int \left(\sum_{k=1}^3 \beta_k P_k(\lambda) \right) V(\lambda) d\lambda = \int \left(\sum_{k=1}^3 w_k T_k P_k(\lambda) \right) V(\lambda)$$

Thus the color is represented by (t_1, t_2, Y) . The luminance Y is a measure of the total response of the visual system to the color or more informally the “brightness” of the color, while the first two components t_1, t_2 are the *chrominance components*, which specify the specific shade of the color.

Color Coordinate Systems The preceding discussion on color representations was based on a particular (fixed) set of primaries $P_1(\lambda), P_2(\lambda), P_3(\lambda)$. As mentioned, one common choice for the primaries is the C.I.E. R,G,B standard primary system. One could choose a different set of primaries and obtain a different color representation or *color coordinate system*. A number of different color coordinate systems have been used for various reasons. Some common ones are R_n, G_n, B_n N.T.S.C. (National Television Systems Committee) receiver primary system, Y, I, Q N.T.S.C. transmission system, and Y, U, V. All of these are simply linear transformations of the C.I.E. R,G,B system. Another common system is IHS (for Intensity, Hue, Saturation) which is *not* a linear transformation of R,G,B. Details on these and other color coordinate systems can be found in various books on image processing (e.g., by Jain, Pratt, etc.).

Chapter 7

Filtering and Enhancement

Signal filtering and enhancement include a broad range of operations that are performed on signals. It's a very important subject as some type of filtering is used in most systems that process information, and some type of enhancement is used in most applications, often as a final step to prepare an output for the end user.

Some goals of filtering and enhancement include detecting, extracting, or separating signals, reducing noise, or accentuating certain features of a signal. Filtering and enhancement techniques can be conveniently divided into the following groups

- Point/histogram operations
- Time/spatial domain operations
- Frequency domain operations
- Geometric operations

Before we proceed, we make some comments about terminology and our focus in this chapter. Often in signal processing, filtering refers to passing a signal through an LTI system. As we've seen, such operations can be viewed in the time domain as a convolution, or in frequency domain in terms of the frequency response of the system. These are the second and third groups of methods above, respectively. In fact, it is the frequency domain perspective that gives rise to the term “filtering” since this can be viewed as allowing certain frequencies of the original signal to pass through, while filtering out the unwanted frequencies. In filtering, there is often a well-defined objective, and part of this objective may be to carry out the desired operation on the signals while meeting some design constraints. The focus is often on designing special filters with desired characteristics that make them suitable for implementation.

In contrast, the goals of enhancement may be rather subjective, like trying to make an audio signal more pleasing to listen to, or an image more visually attractive. Despite the lack of a well-defined objective, enhancement is a very important subject since for many systems a human is the final “user” of the signals, and even simple enhancement techniques can greatly improve the experience of the user. For example, recall our discussion of halftoning from Chapter 5. In a broad sense, we can think of this as a type of enhancement to make a printed image more visually pleasing.

*©2023 by Sanjeev R. Kulkarni. All rights reserved.

[†]Lecture Notes for ECE201 Information Signals.

[‡]Thanks to Sean McCormick for producing the figures.

Thus, for our purposes, we take a rather broad view of filtering and enhancement rather than much more detailed discussions that would be appropriate for more advanced courses. Here, we focus on some general goals and techniques, rather than on details of filter design.

7.1 Point/Histogram Methods

In point/histogram enhancement techniques, the amplitude of the signal is modified in a way that does not depend on the location of the sample or the amplitudes of neighboring samples. For example, in an image, the gray levels would be modified pixel by pixel with the same transformation applied to each pixel separately.

7.1.1 Point Transformations

If x denotes the amplitude of a sample of the original signal, then the amplitude in the enhanced signal using a point transformation is given by

$$y = f(x)$$

where $f(\cdot)$ is the particular amplitude transformation. For convenience, we will assume that the amplitudes of both the input and output signals are confined to the range $[0, L]$. The key property that makes these point transformations, is that $f(\cdot)$ is independent of the location of the sample and of the amplitudes of neighboring samples.

A number of useful point operations arise from different choices for the transformation $f(\cdot)$. Some specific transformations are discussed below, where we focus on images since these allow us to illustrate a number examples in a visual way.

Contrast Stretching Suppose the original image doesn't occupy a full range of gray levels. Every gray level x of the original image lies in some range $[a, b]$ which is a subset of $[0, L]$.

In this case, a natural operation to perform is to "stretch" the the gray levels in the original image so as to take advantage of the full dynamic range available. This can be performed by the simple linear operation on the gray levels

$$f(x) = \frac{L}{b-a}(x-a) \quad (7.1)$$

Even if the gray levels in the original image occupy whole range $[0, L]$, a piecewise linear transformation of the type

$$f(x) = \begin{cases} \alpha x & 0 \leq x < a \\ \beta(x-a) + v_a & a \leq x < b \\ \gamma(x-b) + v_b & b \leq x \leq L \end{cases} \quad (7.2)$$

can often still be useful.

In each linear region, a slope greater than 1 results in stretching the domain, while a slope less than 1 results in compressing the domain. Usually the interval $[a, b]$ is chosen where most of the gray levels occur or where certain features of interest lie. The transformation is then chosen to stretch this region and compress the rest. Often, the parameters of the transformation are chosen such that $\gamma(L-b) + v_b = L$ so that the whole range for v is used. Note that in the special case that $\alpha = \gamma = 0$, we have *clipping*, which is the transformation in (7.1).

Range Compression Sometimes a nonlinear gray level transformation such as

$$y = c \log(1 + x) \quad (7.3)$$

can be useful in enhancing images which have a large dynamic range in the gray levels and have features of interest throughout the gray level range. Such a transformation, often referred to as *range compression* enhances small magnitude pixels compared with large magnitude.

Thresholding *Thresholding* refers to the case in which the output gray level is either 0 or L depending on whether the input gray level is respectively below or above some constant T . That is,

$$f(x) = \begin{cases} 0 & x \leq T \\ L & x > T \end{cases}$$

This is a special case of the general piecewise linear transformation (7.2) in which $\alpha = \gamma = 0$ and $a = b$.

Thresholding results in a binary image and can be useful in trying to distinguish the foreground in an image from the background. Thresholding can also be useful if the original image is some binary image corrupted with (non-binary) noise. The operation of thresholding is actually a very common operation in a number of other image processing algorithms as well. For example, thresholding is generally used at some stage whenever a binary decision is made based on non-binary data (as in deciding whether or not an edge or some other feature is present).

A slightly more general operation than thresholding is *intensity level slicing* in which

$$f(x) = \begin{cases} L & a \leq x \leq b \\ 0 & \text{otherwise} \end{cases} \quad (7.4)$$

or

$$f(x) = \begin{cases} L & a \leq x \leq b \\ x & \text{otherwise} \end{cases} \quad (7.5)$$

This allows the selection of features in a specific gray level interval of interest.



Figure 7.1: Digital Thresholding

Digital Negative Taking

$$f(x) = L - x$$

results in a *digital negative* of the original image which can be useful in bringing out certain features.



Figure 7.2: Digital Negative Example

Bit Plane Slicing *Bit plane slicing* refers to separating a gray scale image into multiple binary images of the same size as the original image. If the original image has gray levels between 0 and 255, then the gray level of each pixel is represented by 8-bits. The bit plane corresponding to the most significant bit is a binary image where each pixel is either 0 or 1 depending on the most significant bit of the 8-bit gray level. Similarly, a binary image can be defined based on each of the other bits representing the gray levels. Specifically, the k -th bit plane can be defined by mapping an input gray level x to an output gray level y as

$$y = \begin{cases} 1 & \text{if } k\text{-th bit of } x = 1 \\ 0 & \text{otherwise} \end{cases} \quad (7.6)$$

Bit plane slicing can be used in image analysis and compression. By looking at the various bit planes, one can get an idea how many bits are visually significant in the image, together with any spatial dependence on the number of visually significant bits. See Figure 7.3. Note that the binary bit plane images are displayed using 0 and 255 (as is typical in displaying binary images) even though the pixel values are 0 and 1.

7.1.2 Histogram Modification

The enhancement techniques discussed above are implemented by selecting a specific gray level transformation. Of course, such transformations have an effect on the histogram (frequency of occurrence of gray levels) of an image, but the effect on the histogram is not directly specified.

Another approach to gray level transformation techniques is to modify gray levels in order to shape the histogram of the image to certain specifications. The most widely used of such methods is *histogram equalization*, in which the goal is to produce an output image that has flat (uniform) histogram. The rationale is that if the histogram is very peaked in some areas and very low (or zero) in other areas, then some gray levels are over utilized and other gray levels are under utilized (or not used at all). We should be able to exhibit more features in the signals if we use all the gray



Figure 7.3: Bit Plane Slicing Example

levels roughly equally. This is similar to the rationale behind contrast stretching, but takes it even further. In many cases, histogram equalization does in fact enhance the image, bringing out detail that wasn't visible in the original. Figure 7.4 shows an example of this.

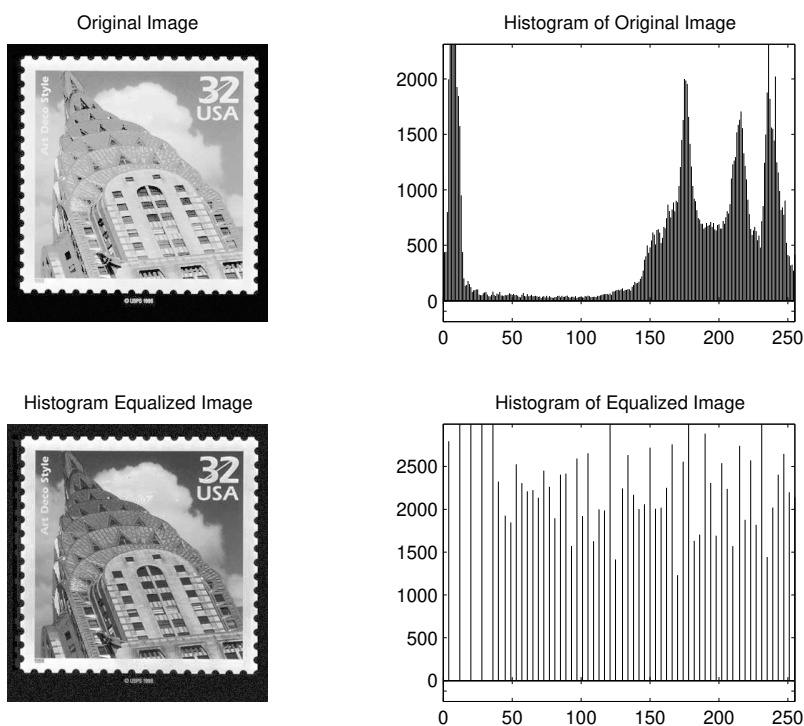


Figure 7.4: Example of Histogram Equalization

If the gray levels take on a continuous range of values, then with a suitable transformation, the histogram can be exactly equalized. However, in the usual case where the gray levels have been

quantized, we may not be able to exactly equalize the histogram because certain gray level may occur too often. No matter what value these gray levels are mapped to, the new value will also occur too often. However, we can try to flatten the histogram as much as possible.

If a strictly flat histogram is desired, the gray levels of the equalized image can be randomly redistributed to different quantization bands. However, this results in a loss of spatial detail, and the strictly flat histogram is seldom necessary.

Histogram equalization is a special case of *histogram specification* in which one performs a transformation to make the histogram of the output image satisfy particular specifications.

7.2 Time/Spatial Domain Methods

Two common goals in time/spatial and frequency domain filtering and enhancement are to reduce noise or enhance edges in the image. Edge enhancement (also called sharpening) is accomplished by emphasizing high frequencies (or de-emphasizing low frequencies). Noise reduction is accomplished by emphasizing low frequencies (or de-emphasizing high frequencies). Both of these operations can be performed in either the time/spatial domain or in frequency domain. Time/spatial domain operators are discussed in this section and frequency domain methods are discussed in the next section.

In the time/spatial domain, the operations are performed by a convolution. Usually a small mask is chosen, which corresponds to the impulse response of the filter. This original signal is convolved with this mask. The choice of the mask determines the type of operation done to the signal.

7.2.1 Noise Reduction

Noise reduction is accomplished by averaging, which corresponds to low pass (or low emphasis) filtering. This can be accomplished by convolving the image with a spatial mask that has all positive coefficients.

Some common common low pass filters are

$$\frac{1}{9} [111111111]$$

and

$$\frac{1}{5} [1 \ 111 \ 1]$$

Entries that are not shown are assumed to be zero. The gray level of a pixel in the filtered image is the weighted average of the gray levels in a neighborhood from the original image. The center pixel of the mask is assumed to be the center of the impulse response for the convolution.

Usually the filters are normalized so that the sum of the entries is one. This avoids amplitude bias so that average gray levels are kept the same. For example, in an area of the image that has constant gray levels, the filtered image will also have this same constant gray level. If the sum were not one, then the gray level in a constant region would get multiplied by the sum of the pixels in the mask.

Filters can also be used which give more weight to certain pixels. Usually the weights near the center of the mask are larger since this gives more weight to those pixels in the original image close to the corresponding location in the filtered image. Some examples are

$$\frac{1}{10} [111121111]$$

$$\frac{1}{6} \begin{bmatrix} 1 & 121 & 1 \end{bmatrix}$$

$$\frac{1}{16} [121242121]$$

Of course, we are not restricted to using only 3×3 masks. We could use smaller or larger masks, e.g., 2×2 , 5×5 , or 7×7 . However, there is a fundamental trade-off that drives the choice of the mask size.

These low pass filters do reduce noise, particularly i.i.d noise at each pixel, but they also blur edges. The larger the mask, the more averaging, and hence the more noise reduction we can obtain. On the other hand, larger masks increase the blurring of edges and result in a filtered image of lower resolution.

There are some techniques that try to better preserve the edges. One such technique is directional smoothing in which one can try to estimate an edge and then perform averaging along the direction of the edge, but not across it. These techniques can sometimes greatly reduce the blurring due to low emphasis filtering.

Another popular and often very effective technique is *median filtering*. In this method, the input gray level is replaced by the median of the gray levels of pixels in some neighborhood. The median of a set of numbers is middle value (or the average of the two middle values if the number of values is even).

Typically the neighborhood is taken to be a 3×3 , 5×5 , 7×7 window, or the pixel with its four nearest neighbors. The value of a pixel in the filtered image is given by the median of the values of the pixels in the corresponding neighborhood of the original image.

Median filtering is a nonlinear operation. It cannot be represented by a convolution. It works particularly well for a kind of noise that is often called salt/pepper noise (or impulsive noise, or binary noise). With this type of noise, most of the pixels are unchanged, but occasionally a pixel is very noisy – e.g., it might get replaced by a pure white or pure black pixel. With this type of noise, the corrupted image looks like the original but with a sprinkling of random white and black pixels (hence the term salt/pepper noise). When not too many pixels in the window are noisy, median filtering can have a dramatic effect. For example, if only one pixel is corrupted in a smooth area of the image, then the median value in a 3×3 neighborhood does not change much. On the other hand, the average of nine pixels in a 3×3 neighborhood can change by a large amount if just a single pixel is changed from its original gray level to 255. Median filtering also tends to preserve edges well.

However, if the noise is the usual type in which each pixel is corrupted by adding some random value, then median filtering won't work so well. In this case, averaging tends to work better. Also, median filtering can be time consuming if the window is large since a direct implementation requires a large number of comparisons of values. However, this can be somewhat reduced using more efficient implementations of median filtering.

7.2.2 Sharpening

The basic idea in image sharpening is to emphasize high frequencies (or relatively de-emphasize low frequencies). Often high emphasis filtering is performed by either subtracting a low pass filtered version of the original image or adding a high pass filtered version. Namely, the sharpened image y is given by

$$y(m, n) = x(m, n) - cx_{LP}(m, n) \quad (7.7)$$

or

$$y(m, n) = x(m, n) + cx_{HP}(m, n) \quad (7.8)$$

As discussed above, the low pass filtered image x_{LP} is some kind of averaging filter. A high pass filtered image x_{HP} is generally an approximation to some type of spatial derivative operator such as a Laplacian operator. A common approximation for the Laplacian is

$$\begin{bmatrix} -1 & -14 & -1 & -1 \end{bmatrix}$$

As with the averaging operators for noise reduction, the sum of the coefficients in a sharpening operator is also commonly taken to be one to preserve amplitudes in regions of constant gray level.

Some examples are

$$\begin{aligned} &\begin{bmatrix} -1 & -15 & -1 & -1 \end{bmatrix} \\ &[-1 - 1 - 1 - 19 - 1 - 1 - 1 - 1] \\ &[1 - 21 - 25 - 21 - 21] \end{aligned}$$

7.3 Frequency Domain Methods

As with the time/spatial domain operators, two common goals of frequency domain filtering and enhancement are noise reduction (low emphasis filtering) or sharpening (high emphasis filtering). The only difference is that the low or high emphasis filtering is done in the frequency domain.

The general method is to apply a transform such as the DFT to the original image. Then in frequency domain, the appropriate frequencies are either emphasized (increased in magnitude) or de-emphasized (decreased in magnitude, possible even set to 0).

For sharpening the high frequencies are emphasized and/or the low frequencies are de-emphasized, while for noise reduction/smoothing the opposite is done.

In general these enhancement operations are performed by multiplying the frequency coefficients by some weights representing the frequency response of the filter. The enhanced image is then obtained by performing the inverse transform.

As in time/spatial domain, certain nonlinear frequency domain enhancement techniques have also been found to give good results. In root filtering the transform coefficients $v(k, l)$ are expressed in polar form

$$v(k, l) = |v(k, l)|e^{j\theta(k, l)} \quad (7.9)$$

Then enhancement is performed by replacing these coefficients with

$$v^\bullet(k, l) = |v(k, l)|^\alpha e^{j\theta(k, l)}. \quad (7.10)$$

and performing the inverse transform. For $0 < \alpha < 1$, the high frequency coefficients are generally emphasized (since the high frequency coefficients generally have smaller magnitude than the low frequency coefficients). Hence, this corresponds to a sharpening operation. Taking $\alpha > 1$ corresponds to a low emphasis operation.

Another nonlinear technique that can sometimes give dramatic results is homomorphic filtering. Here the transform coefficients are replaced by

$$v^\bullet(k, l) = [\log |v(k, l)|]e^{j\theta(k, l)} \quad (7.11)$$

or

$$v^\bullet(k, l) = [\log(|v(k, l)| + c)]e^{j\theta(k, l)} \quad (7.12)$$

where c is a positive constant to prevent the argument of $\log(\cdot)$ from being zero.

7.4 Geometric Transformations

A very different class of enhancement techniques from those discussed so far are *geometric operators*. These operators modify the geometric *positions* of pixels. That is, these techniques operate on the *domain* of the image rather than directly operating in the range (gray levels) of the image. Geometric transformations are useful in image registration and matching, reducing certain geometric degradations such as warping, and in various graphics applications.

The general geometric transformation maps the input image pixel coordinates i, j to some new coordinates i_o, j_o , which are the output image pixel coordinates. Then the transformed image is given by

$$y(i_o, j_o) = x(i, j) \quad (7.13)$$

That is, the gray level of the transformed image at i_o, j_o is the given by gray level of the original image at i, j . Of course, for digitized images the coordinates from the transformation often will not correspond exactly to the center of a pixel in which case (7.13) needs to be replaced by interpolation over several input pixels.

Two common methods for subpixel interpolation are nearest neighbor interpolation and bilinear interpolation. As its name implies, in nearest neighbor interpolation, the gray level at a particular point is taken to be the gray level of the pixel closest to the point. In bilinear interpolation, linear interpolation is performed along each row, then the result is linearly interpolated along each column. The net result is nonlinear. Bilinear interpolation can be much more effective than nearest neighbor interpolation in reducing artifacts. There are many other interpolation methods as well.

Below we discuss some of the standard geometric transforms. These are quite simple, but often can be very useful.

Translation For translating (shifting) the original image, the transform between the input and output pixels is

$$i = i_0 - t_i \quad (7.14)$$

$$j = j_0 - t_j \quad (7.15)$$

or equivalently

$$i_0 = i + t_i \quad (7.16)$$

$$j_0 = j + t_j \quad (7.17)$$

The output image is given by

$$y(i_0, j_0) = x(i_0 - t_i, j_0 - t_j) \quad (7.18)$$

If t_i, t_j are multiples of pixels, then the transformation is completely trivial. Otherwise some interpolation method must be used.

Rotation For rotating the original image, the transform is given by

$$i_0 = i \cos \theta - j \sin \theta \quad (7.19)$$

$$j_0 = i \sin \theta + j \cos \theta \quad (7.20)$$

or equivalently

$$i = i_0 \cos \theta + j_0 \sin \theta \quad (7.21)$$

$$j = -i_0 \sin \theta + j_0 \cos \theta \quad (7.22)$$

The rotated image is therefore

$$y(i_0, j_0) = x(i_0 \cos \theta + j_0 \sin \theta, -i_0 \sin \theta + j_0 \cos \theta) \quad (7.23)$$

Interpolation is necessary unless θ is an integer multiple of 90° .

Scaling For scaling the original image (i.e., for magnification or minification), the transform is given by

$$i_0 = S_i i, \quad j_0 = S_j j \quad (7.24)$$

or equivalently

$$i = \frac{1}{S_i} i_0, \quad j = \frac{1}{S_j} j_0 \quad (7.25)$$

$S_i, S_j > 1$ corresponds to magnification while $S_i, S_j < 1$ corresponds to minification. For minification, several pixels of input replaced by one pixel in the output image. This can be performed by subsampling (with interpolation if necessary) or averaging. For magnification, interpolation is necessary. The simplest method is simply to replicate the input image gray levels (which is basically nearest neighbor interpolation), but this can give a very blocky appearance in the output image. Bilinear interpolation can improve the results significantly.

There are other geometric modifications such as warping or camera transformations that are also quite useful in certain applications.

7.5 Modulation and Demodulation

So far we have studied filtering largely in the context of *image* signal processing. An important application of filtering is with respect to *radio* signals; we discuss this application here.

Radio signals. Examples of radio (or wireless) signals include AM/FM radio, WiFi, Cellular and Radar. Each of these signals are *spectrally* restricted to be in specific parts of the frequency band. For instance, WiFi signals were originally restricted to be within 2.4 and 2.4835 GHz; with the huge popularity of WiFi, newer “frequency bands” have been allowed to accommodate larger traffic. 5G Cellular signals in the low frequency range are restricted to be within 600 and 900 MHz. See Figure 7.5 for an illustration of one of these signals: the signal is centered around a “carrier” or *center* frequency ω_c and has a “bandwidth” of W centered around this carrier. Such signals are called *passband*. If the carrier frequency were zero, then the signal would be called *baseband* or *low pass*.

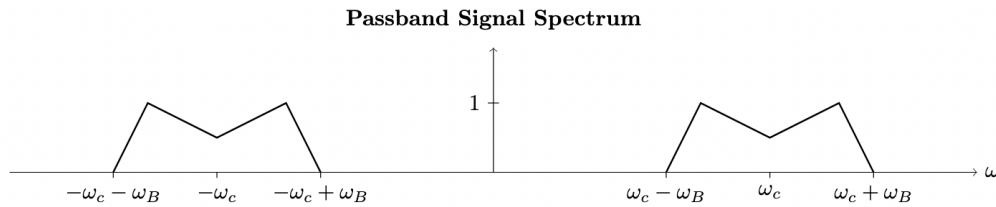


Figure 7.5: Example of Passband Signal

Note that the notion of bandwidth is subtly related to the definition of *bandlimit* that we introduced in Chapter 5: the bandlimit is largest frequency beyond which the Fourier transform of signal has zero amplitude. By this definition, the signal in Figure fig:passband-signal is bandlimited with maximum frequency $w_B = w_c + W$. In contrast, bandwidth is the range of frequencies where the amplitude of the Fourier transform of the signal is nonzero. Bandlimit and bandwidth are closely related due to a common way in which radio signals are constructed (during transmission) and deconstructed (during reception). These operations are called *modulation* and *demodulation*, respectively. Filtering and the Fourier transform play a very important role in both these two operations.

Modulation. The original information signal is typically low pass, see Figure 7.6.

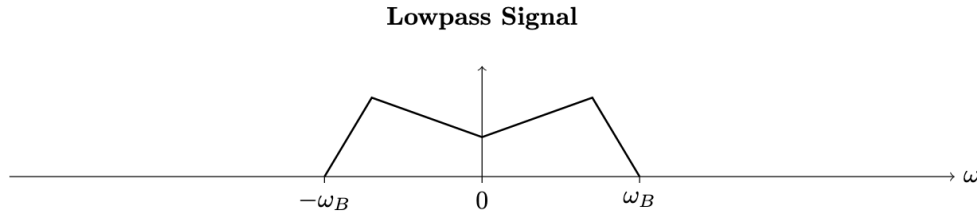


Figure 7.6: Example of Low Pass Signal

Spectral licensing and propagation requirements requires the information signal be transmitted as a *passband* signal. Modulation is the process of converting the low pass signal into a passband signal – essentially shift the signal to the higher frequency. A common modulation approach is the following:

$$x_p(t) = x(t) \cos(w_c t). \quad (7.26)$$

Using the fact that $\cos(\theta) = \frac{e^{j\theta} + e^{-j\theta}}{2}$ and the shift property of the Fourier transform, we see that

$$X_p(w) = \frac{X(w + w_c) + X(w - w_c)}{2}. \quad (7.27)$$

See Figure 7.7 depicts the modulated low pass signal as a passband signal.

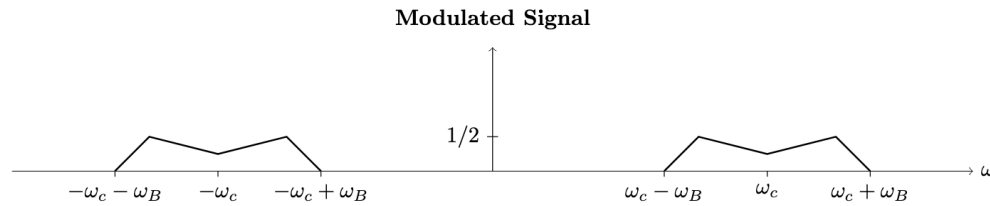


Figure 7.7: Example of Modulated Signal

Demodulation. During reception, one needs to first focus on the received passband signal of interest: this is done by passing the received signal (from an antenna) through a special type of

LTI system called a *passband filter*: this system allows input signal to go through unchanged when the frequencies are within a range (e.g., $[w_c - w_B, w_c + w_B]$), but otherwise zero output for all other frequencies. See Figure 7.10 for the Fourier transform of the impulse response of the passband-filter: $\text{rect}(\frac{w-w_c}{2w_B}) + \text{rect}(\frac{w+w_c}{2w_B})$.

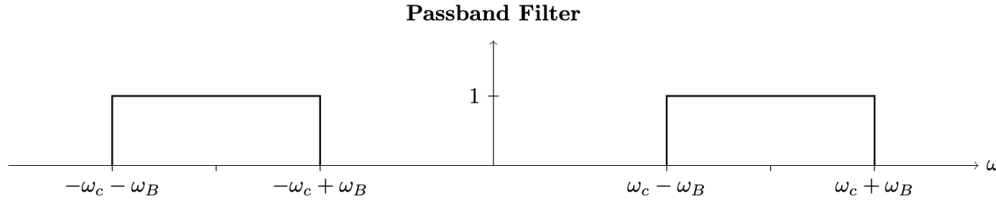


Figure 7.8: Example of Passband Filter

Once we have zoomed in on the information carrying passband signal (zeroing out signals in all other frequencies), the received signal is now essentially $x_p(t)$ (albeit diminished in magnitude due to the propagation and perhaps with noise added due to the environment, both of which effects we ignore here). Demodulation starts with simply repeating the same operation as modulation:

$$x_r(t) = x_p(t) \cos(w_c t). \quad (7.28)$$

See Figure 7.9 which depicts the modulated passband signal as a superposition of low pass signal and twice higher frequency passband signals:

$$X_r(w) = \frac{1}{2}X(w) + \frac{1}{4}(X(w - 2w_c) + X(w + 2w_c)). \quad (7.29)$$

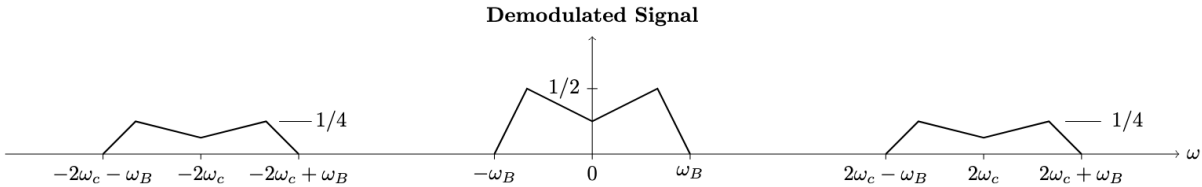


Figure 7.9: Example of Demodulated Signal.

To fully recover the information carrying low pass signal, we pass through this signal with *another* filter, this time a low pass one: see Figure 7.10 for the Fourier transform of the impulse response of the low pass filter: $\text{rect}(\frac{w}{2w_B})$.

Passing $x_r(t)$ through this low pass filter allows us to recover $x(t)$.

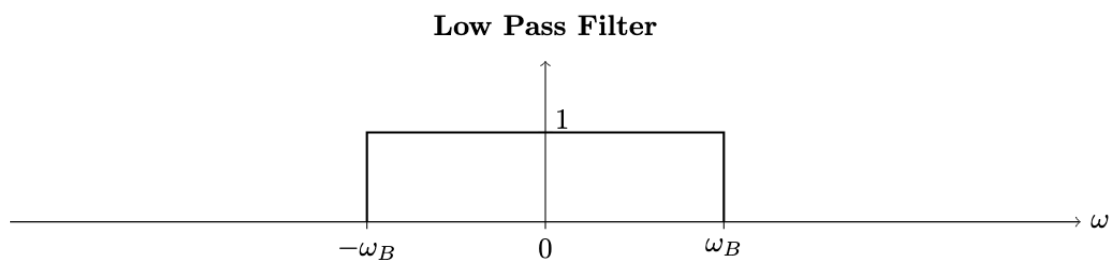


Figure 7.10: Example of Low Pass Filter.

Chapter 8

Information, Entropy, and Coding

8.1 The Need for Data Compression

To motivate the material in this chapter, we first consider various data sources and some estimates for the amount of data associated with each source.

- **Text** Using standard ASCII representation, each character (letter, space, punctuation mark, etc.) in a text document requires 8 bits or 1 byte. Suppose we have a fairly dense text with 50 lines of text per page and 100 characters per line. This would give 5 kB per page. If the text were a hefty 1000 pages, this would result in 5 MB. Alternatively, if we assume one word is on average 6 characters, then each word requires approximately 6 bytes. Therefore, a one million word document would require approximately 6 MB. Although these numbers may sound big, remember that this is for a very large text document. As we'll see other media result in significantly more data.
- **Audio** In digital audio, it's typical to use 16 bits per sample and 44,100 samples per second, resulting in 706 kb/sec or 88.2 kB/sec. With two channels for stereo we would get 176 kB/sec. Therefore, just 30 seconds of digital audio requires almost 5.3 MB, which is more than the 1000 page text document. An hour of two-channel audio results in about 635 MB.
- **Images** For a gray scale image, it's typical to use 8 bits/pixel. A good resolution image might be 512×512 pixels, which would result in 0.26 MB. For a color image, we need to multiply by 3 for the three color components, resulting in about 0.786 MB. Assuming our previous estimate, of about 6 characters per word, this means such an image (picture) is worth more 100,000 words, rather than 1,000 words! Only 7 such images would result in about 5.5 MB, more than the 1000 page text document.
- **Video** A standard frame rate for video is about 30 frames/sec. Using the estimates for images, we get about 7.86 MB/sec for black/white video and about 23.6 MB/sec for color video. This means that just one second of black/white video or 1/4 of a second of color video results in more data than the 1000 page text document. One minute of black/white video requires about 472 MB, while one minute of color video requires 1.416 GB. A two hour black/white video requires 56.6 GB, while a two hour color video requires almost 170 GB!

*©2024 by Sanjeev R. Kulkarni. All rights reserved.

[†]Lecture Notes for ECE201 Information.

[‡]Thanks to Keith Vallerio for producing the figures.

The estimates above assume the data is represented directly in raw format (e.g., one byte for each character, one byte per gray scale pixel, etc.). The goal of compression is to represent signals (or more general data) in a more efficient form. Compression is extremely useful for both storage and transmission.

Of course, the need for compression is dependent on storage and communication technology. The estimates above are best appreciated in the context of capacities of storage media and data rates for communication. Some historical digital storage media and their capacities include the floppy disk (about 1.4 MB), zip disk (100 MB or 250 MB), CD (about 700 MB), DVD (4.25 GB single-sided, 8.5 GB double-sided, 18 GB double-sided and dual density), and Blue-ray (25 GB single layer, 50 GB double layer). Some historical data rates for communication include early home modems from 300 to 1200 bps (bits per second), and faster modems from the mid 1980's through the 1990's with data rates from 2400 bps to 56 kbps. Both wired and wireless capacity for home/consumer use has steadily increased with both wired and wireless download speeds of hundreds of Mbps.

Although capacities of storage media and communication data rates available for most users have been increasing rapidly, the demand for data has kept pace. So far, it appears that “if the capacity is available the bits will come”. Moreover, the demand for throughput in wireless communication has also been rapidly increasing and outpacing the increases in efficiency in use of the limited spectrum. These forces are likely to ensure that compression remains an important topic for the foreseeable future.

8.2 Basic Idea of Compression

The basic idea of compression is to exploit (in fact, remove) redundancy in data. Compression can be broken down into two broad categories. In *lossless* compression, from the compressed data one is able to reproduce the original data exactly. In *lossy* compression, the original data cannot be reproduced exactly. Rather we allow some degradation in the reproduced data.

In a sense, sampling and quantization can be considered as forms of compression. Sampling is lossless if we sample above the Nyquist rate, and is lossy otherwise. Quantization is lossy compression. Clearly there is a tradeoff between the quality and amount of data. In sampling and quantization, the signal is converted from an analog to a digital signal. This “compression” has the dramatic effect of converting infinitely many samples each of which requires infinite precision to a finite number of samples that each have finite precision.

However, even after a signal is digitized, we can often compress the data still more. This is the usual goal of data compression. Thus, in this and the next chapter, we assume that we already have digital data, and we discuss theory and techniques for further compressing this digital data.

8.3 The Coding Problem

In this section, we formulate the general “coding problem”, so that the ideas we discuss will be applicable to general digital data rather than being developed for one specific application.

Suppose we are given M symbols denoted $s_1, s_2, \dots, s_M$. For example, for images and video we might have $M = 256$ with the symbols s_i denoting the 256 gray levels. Or in a certain application, we might have $M = 26$ with the s_i denoting the 26 letters of the alphabet, or $M = 10$ with the s_i denoting the 10 digits. For general text files, we might have $M = 128$ (as in ASCII) with the s_i denoting 128 characters including upper and lower case letters, digits, punctuation marks, and various special symbols. Regardless of the basic symbols being used for the application of interest

(letters, gray levels, etc.), we would like to represent these symbols using bits (i.e., just 0's and 1's). The bit strings used to represent the symbols are called the codewords for the symbols. The coding problem is to assign codewords for each of the symbols $s_1, \dots, s_M$ using as few bits per symbol as possible.

How many bits do we need per symbol? The obvious answer is that we need $\log_2 M$ bits. For example, if there are 8 symbols $s_1, \dots, s_8$, then we can use the codewords 000, 001, 010, 011, 100, 101, 110, 111. Likewise, for 256 gray levels we need $\log_2 256 = 8$ bits/symbol. For 26 letters, $\log_2 26 \approx 4.7$, which means we need 5 bits/symbol. With 4 bits we can only represent 16 unique symbols. Is $\log_2 M$ the best we can do?

First, note that implicit in our discussion so far is that each codeword must correspond to a unique symbol. Otherwise, the representation would not be very useful since we would not be able to recover the original symbols (and we're interested in lossless coding). We will say more about this and other conditions shortly.

Subject to the unique representation alluded to above, we cannot in general do any better than $\log_2 M$ without additional assumptions. However, if there's some structure in the data source or if we know something about the symbols we expect to encounter, then we *can* do better. The simplest example of such knowledge/structure is if some symbols are more likely than others, but are otherwise assumed to be random with each symbol being generated independently of other symbols. We will discuss this case in detail as it elucidates the main ideas. However, researchers in the field of information theory and coding have developed results in much more general settings.

8.4 Variable-Length Coding

Assuming that some of the symbols are more likely than others (and assuming we know the respective probabilities of occurrence), the key idea to obtaining a more efficient coding is to use *variable-length coding*. That is, we assign shorter codewords to the more frequently occurring symbols and longer codewords to the symbols that occur infrequently. The standard coding that uses the same number of bits for each codeword is called fixed-length coding.

Variable-length coding is a natural idea and has been used for some time. For example, Morse code exploits this idea in the relative frequency of occurrence of letters in the English language. As another simple example, consider the following:

Consider four symbols s_1, s_2, s_3, s_4 . With the standard (fixed-length) encoding we would need 2 bits/symbol – e.g., we might assign the codewords 00, 01, 10, 11, respectively.

Suppose we know that the probabilities for the symbols are $1/2$ for s_1 , $1/4$ for s_2 , and $1/8$ each for s_3 and s_4 . How might we exploit this knowledge?

Suppose we assign the codewords 0, 10, 110, and 111 for s_1, s_2, s_3 , and s_4 , respectively. In this case, occasionally we do better than the standard encoding (using only 1 bit for s_1 instead of 2 bits). However, sometimes we also do worse (using 3 bits for s_3 and s_4 instead of 2 bits). To precisely compare the new code with the standard encoding, we can compute the average number of bits/symbol of the codes. The standard coding *always* uses 2 bits, so obviously the average number of bits per symbol is also 2. For the new code, how should we compute the average? We should use the probabilities of the symbols, of course! We find that the average length (bits/symbol) is

$$(1)(1/2) + (2)(1/4) + (3)(1/8) + (3)(1/8) = 1.75.$$

Thus, on average we only need 1.75 bits/symbol instead of 2 bits/symbol. For a file with 1 million symbols, this would mean we only need 1.75 Mbits with the variable-length code instead of 2 Mbits with the standard (fixed-length) code.

8.5 Issues in Variable-Length Coding

With variable-length codes, the issue of codewords corresponding to “unique” symbols is a little more subtle than with fixed-length codes. Even if there is a unique correspondence, another subtlety can arise in decoding. We now discuss these issues.

Unique Decodability With variable length codes, in addition to not having two symbols with the same codeword, we also have to worry about some combination of symbols giving the same string of bits as some other combination of symbols. For example, several symbols with short codewords could combine to form a bit string corresponding to a longer codeword for some other symbol.

Suppose for four symbols s_1, s_2, s_3, s_4 we assign the codewords 0, 10, 01, 11, respectively. Then we can’t tell whether 0110 corresponds to s_3s_2 or $s_1s_4s_1$. This shows that this particular code is not uniquely decodable.

A code is said to be *uniquely decodable* if two distinct strings of symbols never give rise to the same encoded bit string. Equivalently, any bit string can arise from at most one string of symbols. Note that it is possible that *no* string of symbols could give rise to a given bit string. For example, if the codewords for s_1, s_2, s_3, s_4 are 000, 001, 010, 011, respectively, then the string 111 can never arise from any string of symbols.

Unique decodability is an absolute must if we wish to have useful (and lossless) compression. The following property, though not a “must”, is useful, and, as it turns out, can be achieved at no cost.

Instantaneous/Prefix-Free Codes A code is called *instantaneous* if each symbol can be decoded as soon as the corresponding codeword is completed. That is, it is not necessary to see bits of later symbols in order to decode the current symbol.

For example, the variable-length code of the first example from Section 8.4 is instantaneous. But the code 0, 01, 011, 111 for the symbols s_1, s_2, s_3, s_4 , respectively is *not* instantaneous (although it *is* uniquely decodable). To see this, consider the bit stream 011111...1. We can’t tell if the first symbol is s_1, s_2 , or s_3 , although it’s clear that after this first symbol we have a sequence of s_4 ’s. Once we see the last 1, we can then work backwards to eventually find out what was the first symbol.

It turns out that being an instantaneous code is equivalent to a property known as being prefix-free. A code is said to be *prefix-free* if no codeword is the prefix (the first part) of another codeword. It’s easy to see that the code in Section 8.4 is prefix-free, and so the code is also instantaneous. However, for the code above (0, 01, 011, 111), the codeword for s_1 is a prefix of the codewords for s_2 and s_3 . Likewise the codeword for s_2 is a prefix of the codeword for s_3 . Therefore, this code is *not* prefix-free, and as we saw above is also not instantaneous.

Requirements/Wish List In our consideration of coding, as we mentioned we need the code to be uniquely decodable, and we would like the code to be instantaneous.

There are two other things we would like. One is a systematic way to design good codes. Given any number of symbols and any probabilities for these symbols, it would be nice to have a method for assigning codewords with good performance (i.e., small average bits per symbol). The second is a benchmark. It would be useful to know what is the smallest number of bits/symbol that could possibly be achieved with a uniquely decodable (and hopefully instantaneous) code, and it would be nice to have if we had a method to achieve the best possible.

Thus, our requirements/wish list can be summarized as follows:

- Need a uniquely decodable code.
- Would like an instantaneous code.
- Would like a systematic design method.
- Would like a benchmark on the lowest possible bits/symbol that can be achieved.

Before addressing the first three items, we discuss the last.

8.6 Information and Entropy

Formalizing the notion of information and the limits to communication began in the late 1940's and continues to be an active area of research. This work forms another conceptual pillar of the information revolution and electrical engineering in general.

The formalization of “information” departs from the more usual notion relating to semantic meaning. Rather, the technical definition of the information provided by some event relates to the probability of occurrence of the event. Roughly speaking, the more surprise we experience upon observing a particular outcome, the more information provided by that outcome.

In particular, if the probability of an event is 1, then the event is certain. In this case, there is no surprise upon learning that the event occurred, and likewise we receive no information from knowledge of its occurrence (since we knew the event would occur). On the other hand, if the probability of an event is near 0, then the event is very unlikely and we would be very surprised to learn that the event occurred. In this case, we receive a great deal of information upon learning of its occurrence. In the intermediate case that the probability of occurrence is $1/2$, both the occurrence and non-occurrence are equally likely. In this case, we receive exactly 1 bit of information upon learning the event occurred.

The amount of information we receive upon learning an event occurred is inversely related to the probability p of the event. To get other natural properties of information, it can be shown that the appropriate definition is

$$\begin{array}{l} \text{information gained upon learning} \\ \text{event of probability } p \text{ occurred} \end{array} = \log_2 \frac{1}{p} \text{ bits.}$$

In many applications there are a number of possible mutually exclusive outcomes denoted by symbols $s_1, s_2, \dots, s_M$. For example, in an image processing application, we might have 256 symbols where $s_1, \dots, s_{256}$ denote the possible gray levels of a particular pixel. For audio signals, the s_i could denote the amplitude level of a sample of the signal. In text applications, the s_i might denote the various letters, digits, punctuation, and other special characters that are used. In most applications including those just mentioned, instead of having just a single symbol, we have a stream or “source” of many symbols. For example, in one 512×512 image we have over 260,000 symbols, in one second of digital audio we have 44,100 symbols, and in text document we will typically have thousands or millions of symbols. The motivation for introducing the s_i and talking about a source of symbols is to keep the development general enough to be applied to all these (and other) examples.

If someone is sending us a stream symbols from a source (such as those above), we generally have some uncertainty about the sequence of symbols we will be receiving. If not, there would no need for the transmitter to even bother sending us the data. A useful and common way to model this uncertainty is to assume that the data is coming randomly according to some probability distribution. The simplest model is to assume that each the symbols $s_1, \dots, s_M$ have associated

probabilities of occurrence $p_1, \dots, p_M$, and we see a string of symbols drawn independently according to these probabilities. Since the p_i are probabilities, we have $0 \leq p_i \leq 1$ for all i . Also, we assume the only possible symbols in use are the $s_1, \dots, s_M$, and so $p_1 + \dots + p_M = 1$.

How much information does a source provide? Consider just one symbol from the source. The probability that we observe the symbol s_i is p_i , so that if we do indeed observe s_i then we get $\log_2 \frac{1}{p_i}$ bits of information. Therefore, the average number of bits of information we get based on observing one symbol is

$$p_1 \log_2 \frac{1}{p_1} + \dots + p_M \log_2 \frac{1}{p_M} = \sum_{i=1}^M p_i \log_2 \frac{1}{p_i}$$

This is an important quantity called the entropy of the source and is denoted by H . Note that the entropy H should not be confused with the frequency response of a system $H(\omega)$ discussed in Chapter 3. These two areas developed somewhat independently and the use of $H(\cdot)$ for frequency response is very common and the use of H for entropy in information theory is universal.

Definition Given a source that outputs symbols $s_1, \dots, s_M$ with probabilities $p_1, \dots, p_M$, respectively, the *entropy* of the source, denoted H , is defined as

$$H = \sum_{i=1}^M p_i \log_2 \frac{1}{p_i}$$

The importance of the entropy of a source lies in its operational significance concerning coding the source. Since H represents the average number of bits of information per symbol from the source, we might expect that we need to use at least H bits per symbol to represent the source with a uniquely decodable code. This is in fact the case, and moreover, if we wish to code longer and longer strings of symbols, we can find codes whose performance (average number of bits per symbol) gets closer to H . This result is called the source coding theorem and was discovered by Shannon in 1948.

Source Coding Theorem

- (i) The average number of bits/symbol of any uniquely decodable source must be greater than or equal to the entropy H of the source.
- (ii) If the string of symbols is sufficiently large, there exists a uniquely decodable code for the source such that the average number of bits/symbol of the code is as close to H as desired.

This result addresses the fourth item on our wish list. It shows that the entropy of a source is the benchmark we are after. No code can do better than H bits/symbol on average, and if the string of symbols to be coded is long enough then there exist codes that can get us as close to optimal as we like. In the next section, we consider a particular type of code known as Huffman coding that gives us a systematic design method for finding good codes once we know the symbol probabilities. This addresses the first, second, and third items on our wish list.

8.7 Huffman Coding

Huffman coding is a simple and systematic way to design good variable-length codes given the probabilities of the symbols. The resulting code is both uniquely decodable and instantaneous (prefix-free). Huffman coding is used in many applications. For example, as we will see in Chapter

9, it is a component of standard image compression (JPEG), video compression (MPEG), and codes used in fax machines.

The Huffman coding algorithm can be summarized as follows:

1. Think of the p_i as the leaf nodes of a tree. In constructing a Huffman code by hand it's sometimes useful to sort the p_i in decreasing order.
2. Starting with the leaf nodes, construct a tree as follows. Repeatedly join two nodes with the smallest probabilities to form a new node with the sum of the probabilities just joined. Assign a 0 to one branch and a 1 to the other branch. In constructing Huffman codes by hand, it's often helpful to do this assignment in a systematic way, such as always assigning 0 to the branch on the same side.
3. The codeword for each symbol is given by the sequence of 0's and 1's starting *from* the root node and leading *to* the leaf node corresponding to the symbol.

The order of reading off the 0's and 1's in constructing the codewords is important. If the order is reversed, the resulting code will generally not be instantaneous.

Example 8.1 (A Simple Huffman Code)

Consider a source with symbols s_1, s_2, s_3, s_4 with probabilities $1/2, 1/4, 1/8, 1/8$, respectively. The Huffman code is constructed as shown in Figure 8.1.

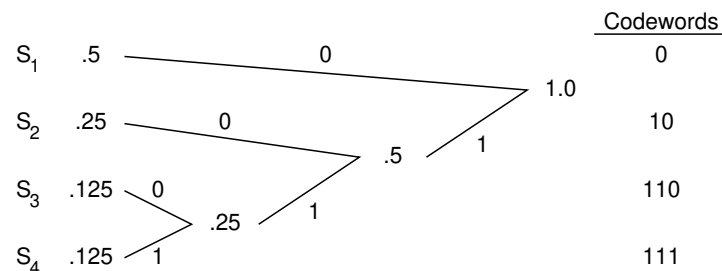


Figure 8.1: Construction of simple Huffman code with 4 symbols.

From the tree constructed, we can read off the codewords. For example, the codeword for s_2 is found by starting at the root node and following the branches labeled 1 then 0 to arrive at s_2 . Therefore, the codeword for s_2 is 10.

The average number of bits per symbol for this code is

$$\begin{aligned}
 \text{average length} &= (1) \left(\frac{1}{2} \right) + (2) \left(\frac{1}{4} \right) + (3) \left(\frac{1}{8} \right) + (3) \left(\frac{1}{8} \right) \\
 &= 1.75 \text{ bits/symbol}
 \end{aligned}$$

Note that a fixed length code would require 2 bits/symbol.

What is best we can do for the given source? The answer is provided by the entropy, which is given by

$$\begin{aligned}
 H &= \frac{1}{2} \log_2 2 + \frac{1}{4} \log_2 4 + \frac{1}{8} \log_2 8 + \frac{1}{8} \log_2 8 \\
 &= \frac{1}{2} + \frac{1}{2} + \frac{3}{8} + \frac{3}{8} \\
 &= 1.75
 \end{aligned}$$

■

In the example above, the rate of the Huffman code is exactly the entropy, so that the Huffman code is the best possible. This is not always the case, as can be seen in the example below. However, it can be shown that that

$$H \leq \text{average length of Huffman code} \leq H + 1$$

Example 8.2 (A Slightly More Involved Huffman Code)

Consider a source with eight symbols $s_1, \dots, s_8$ with probabilities 0.25, 0.21, 0.15, 0.14, 0.0625, 0.0625, 0.0625, 0.0625, respectively. The Huffman code is constructed as shown in Figure 8.2.

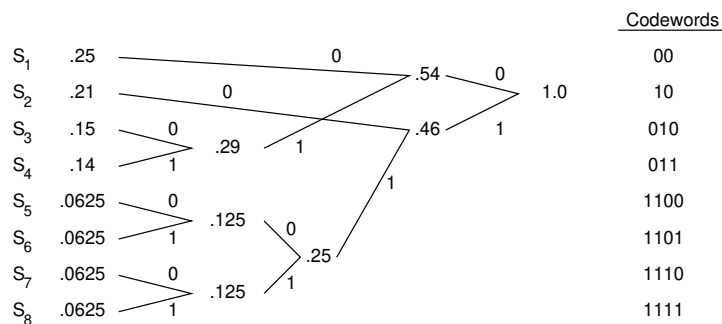


Figure 8.2: A slightly more involved Huffman code with 8 symbols.

Note that in this example, at the stage when we combined the node with probability 0.21 (corresponding to s_2) with the node with probability 0.25 (leading to s_5, s_6, s_7, s_8), we could have instead combined s_2 with s_1 since the node corresponding to s_1 also has probability 0.25. Whenever there is more than one choice, they are all acceptable and lead to valid Huffman codes. Of course, the specific codewords and even the lengths of the codewords may be different for the different choices. However, all will result in the same average codeword length. The average number of bits per symbol for this code is

$$\begin{aligned}
 \text{average length} &= (2)(0.25 + 0.21) + (3)(0.15 + 0.14) \\
 &\quad + (4)(0.0625 + 0.0625 + 0.0625 + 0.0625) \\
 &= 2.79 \text{ bits/symbol}
 \end{aligned}$$

Note that a fixed length code would require 3 bits/symbol.

For this source the entropy is given by

$$\begin{aligned} H &= 0.25 \log_2 \left(\frac{1}{0.25} \right) + 0.21 \log_2 \left(\frac{1}{0.21} \right) + 0.15 \log_2 \left(\frac{1}{0.15} \right) \\ &\quad + 0.14 \log_2 \left(\frac{1}{0.14} \right) + (4)(0.0625) \log_2 \left(\frac{1}{0.0625} \right) \\ &\approx 2.78 \end{aligned}$$

■

In this example, the average length of the Huffman code is close, but not equal, to the entropy of the source. In other cases, the Huffman code may not be quite as close, but as mentioned previously, the average length of a Huffman code will never differ from the entropy by more than one. However, the source coding theorem claims that we can find a code whose average length is arbitrarily close to the entropy. The next section shows how with a simple extension, we can use Huffman codes and to get an average length arbitrarily close to H .

8.8 Block Coding

Consider a very simple source with just two symbols s_1, s_2 with probabilities 0.9 and 0.1, respectively. The Huffman code for this source (shown in Figure 8.3) results in the codewords 0 and 1.

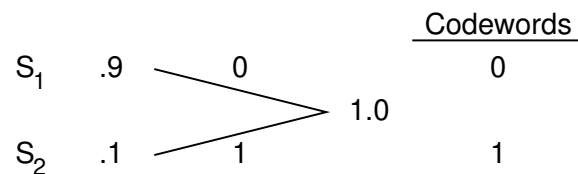


Figure 8.3: A trivial Huffman code with 2 symbols to motivate block coding.

Thus, the Huffman code is just a fixed length code and has average length of 1 bit/symbol. Of course, with just two symbols this is really the only reasonable choice for the codewords. On the other hand, the entropy of this source is

$$H = 0.9 \log_2 \left(\frac{1}{0.9} \right) + 0.1 \log_2 \left(\frac{1}{0.1} \right) \approx 0.47 \text{ bits/symbol}$$

In this case, the Huffman code does poorly compared with the entropy. This isn't surprising. Since there are only two symbols, we have no choice but to assign one bit to each symbol, even though the probabilities of the two symbols are quite different. We can reap more benefits from variable-length coding when there are many symbols to choose from in the design of codeword lengths.

A technique known as block coding is a way to better exploit the advantages of assigning different codeword lengths using variable length coding. The idea is to consider more than one symbol at a time and assign codewords to these "blocks" of symbols. The advantage comes not only from

more flexibility in assigning codeword lengths, but also from the fact that the different probabilities of groups of symbols get magnified when we consider groups of symbols. Blocks containing low probability symbols have extremely small probability compared with blocks containing more commonly occurring symbols.

For example, using blocks of length 2 for the source above with two symbols, we have four possibilities we need to encode: s_1s_1 , s_1s_2 , s_2s_1 , and s_2s_2 . These pairs of symbols have probabilities 0.81, 0.09, 0.09, and 0.01, respectively. The Huffman code for blocks of two symbols is designed as shown in Figure 8.4.

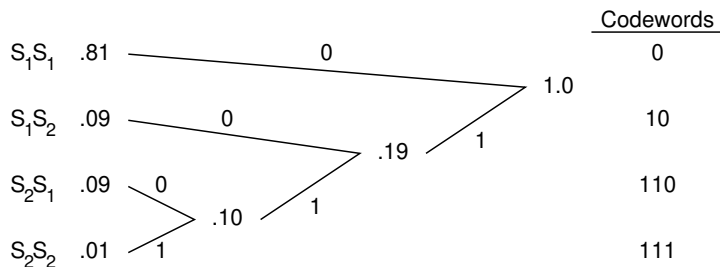


Figure 8.4: An example of block coding using blocks of length 2 for the source in Figure 8.3.

The average length for encoding *two* symbols is

$$1(0.81) + 2(0.09) + 3(0.09 + 0.01) = 1.29.$$

Therefore, the average number of bits per *original* symbol is $1.29/2 = 0.645$ bits/symbol.

Using blocks of length 2 helps significantly by reducing the average codeword length from 1 bit/symbol to 0.645 bits/symbol. We could get even smaller average codeword length with longer blocks. In fact, as the length of the block gets larger, the average codeword length gets closer to the entropy. We can get as close to the entropy as desired by taking sufficiently large blocks. However, there are practical considerations that prevent the use of very large blocks. These and some other issues are discussed in the next section.

8.9 Comments on Huffman Coding

Huffman codes are uniquely decodable and instantaneous codes. Recall that an instantaneous code is equivalent to the prefix-free property. Both unique-decodability and the prefix-free property can be seen from the fact that the codewords of a Huffman code are the leaves of a binary tree. If we see a string of bits and follow from the root node down the tree taking branches according to the bits seen. Then we will eventually reach a leaf node corresponding to the correct symbol. Once the bits for a symbol are completed then we can immediately decode (so the code is instantaneous), and we start again from the root node to continue the decoding. We can also see that there is only one way to decode a given string of symbols (so the code is uniquely decodable).

A Huffman tree is not wasteful in the sense that all the leaf nodes correspond to symbols. If there were empty leaf nodes, then we could remove these and shorten certain codewords and still have a tree code. In fact, it can be shown that for a given block size, Huffman codes are in fact optimal.

However, the average length of the Huffman code may not equal the entropy. But the Huffman code does satisfy

$$H \leq \text{average length of Huffman code} \leq H + 1$$

To approach the entropy, we can use block coding. As the block size gets larger, the average length of the corresponding Huffman code approaches the entropy. In fact using a block size k , it can be shown that

$$H \leq \text{average length per original symbol} \leq H + \frac{1}{k}$$

Unfortunately, for practical reasons the block size cannot be taken to be too large. In fact, if the source has M symbols, then with blocks of length k there are M^k strings of symbols to consider. This grows exponentially in the block size k , and even for modest M and k this can be impractically large. Note that the encoder and decoder both need the codewords. The encoder has to form the Huffman tree using the probabilities, and this either has to be sent as overhead to the decoder, or the decoder needs the same probabilities to construct the Huffman tree itself.

There are two other major disadvantages/limitations of Huffman codes. One involves the assumptions on the source. We assumed that the symbols are randomly drawn (independent and identically distributed) and the probabilities of the symbols are known. Both of these assumptions are unrealistic in many applications. For example, in image processing, suppose we try to apply Huffman coding directly where the symbols are taken to be the gray levels of the pixels. Do we know the probability that a given pixel will take on each gray level? Surely, this depends greatly on the class of images under consideration, and even for very restricted applications this knowledge is hard to come by. The assumption that the gray levels are independent and identically distributed is perhaps even worse. In most images encountered there is a good deal of correlation in the gray levels of nearby pixels. Images with independent and identically distributed pixels look like pure noise. Basic Huffman coding can't fully exploit this additional structure.

Thus, although Huffman coding has a number of nice properties and is used in a number of applications (some of which we will discuss in the next chapter), there are significant limitations as well. The impracticality of using large block sizes together with the assumption that symbols are independent and identically distributed with known probabilities, are major reasons why Huffman coding is far from a panacea for data compression. In fact, other methods for data compression have been widely studied, with a number of general techniques as well as algorithms tailored to specific types of data. The next chapter discusses some of these techniques.

The second main disadvantage with Huffman coding is that any errors in the coded sequence of bits will tend to propagate when decoding. This is an issue for most variable length codes since, unlike fixed length codes, the boundaries between codewords is not known in advance, but rather is "discovered" during the decoding process. If there is an error, then unless the incorrectly decoded sequence of bits happens to end on the correct codeword boundary, the error will propagate into the next codeword.

There are several mechanisms that can be used to prevent error propagation. Some of these techniques have limited use beyond preventing error propagation and we will not discuss these. But one approach that can be used has much the much broader purpose of protecting the data itself against errors (whether or not errors propagate during decoding). These techniques are designed to detect and/or correct errors, and form one topic in the very important area of information protection. This is the subject of Chapter 10.

Chapter 9

Some Data Compression Algorithms

9.1 Compression for Specific Data Types

In the previous chapter, we described the coding problem and discussed one particular method for creating a code that satisfied some desirable properties. Specifically, given a set of symbols $s_1, \dots, s_M$ and corresponding probabilities $p_1, \dots, p_M$, Huffman coding provides a systematic method for creating a uniquely decodable and instantaneous code that works well for the given set of symbol probabilities. Moreover, using block coding, Huffman coding can give an average length as close as desired to the entropy of the source (which is the best possible that can be achieved), assuming the assumptions the source symbols are independent and identically distributed according to the specified probabilities.

However, Huffman coding has serious limitations in many applications. First, the using block coding is impractical when the block size is large. The number of different blocks grows exponentially as a function of the block length, and this can be prohibitive for even modest block sizes. Second, and more fundamentally, the assumption that the symbols are independent and identically distributed according to known probabilities is often far from reality in many applications.

To address these and other issues, a broad range of data compression techniques have been developed for many applications and under a range of different assumptions about the underlying data. By appropriately exploiting structure in the data, significant gains in compression performance can be achieved. Moreover, in some applications lossy compression can enable enormous gains in the amount of compression attainable. This is contrast with lossless compression, which is what we have discussed up to this point in the context of the coding problem.

As the terms suggest, in lossless compression, the original data can be exactly recovered from the coded sequence, whereas in lossy compression we cannot exactly recover the original data. In some applications (for example, image and video compression), a small amount of loss may be tolerable and/or perhaps not even noticed. By allowing this loss, the data can be compressed significantly more than if we insisted on lossless compression.

The coding problem as formulated in Chapter 8 assumes lossless compression, and Huffman coding is one example of a lossless compression algorithm. In this chapter, we discuss another lossless coding method (run-length coding) followed by a discussion of two lossy compression methods – transform coding (JPEG) for images and MPEG for video.

*©2023 by Sanjeev R. Kulkarni. All rights reserved.

†Lecture Notes for ECE201 Information Signals.

9.2 Run-Length Coding

Run-length coding (or run-length encoding) is a lossless data compression technique that can be applied to data where the same symbol appears consecutively a number of times – that is, when there are long runs of a given symbol. For example, suppose we have binary data that has long stretches of 0's followed by long stretches of 1's. A string of data might of this type might be something like the following:

0000000001111111100000000111111111111111

This could be part of a black and white image, or it might be the bit plane of a gray scale image corresponding the most significant bit.

Note that the bits in such a data sequence are most definitely not independent. Given that the most recent symbol is a 0, the probability that the next symbol is a 0 is much larger than the probability that the next symbol is a 1. Similarly, if the most recent symbol is a 1, then the probability that the next symbol is a 1 is much larger than the probability that the next symbol is a 0.

Run-length coding tries to compress such a string by explicitly using the fact that there are long runs of the same symbol in the data. There are many variations of run-length coding, but the basic idea is as follows:

1. Assign codewords for the lengths of the runs of the same symbol.
2. Use blocks of m bits to encode the run lengths. Thus, each successive block of m bits alternates between codings runs of 0's versus 1's.
3. Using m bits to code the run-lengths allows us to code runs of length 0 up to a maximum length of $M = 2^m - 1$. In order to code runs of length greater than M , we let the codeword $11 \cdots 1$ denote a run of length $M = 2^m - 1$ **not** followed by a character change. In this case, the next m -bit codeword would continue the run of the previous character. In this case, to get a run of length exactly M , we need to follow the codeword $11 \cdots 1$ with the codeword $00 \cdots 0$ to indicate the run is over and the character should change.
4. We also need to specify whether to start with a run of 0's or a run of 1's. We let the very first bit of the encoded sequence indicate whether to start with a run of 0's or a run of 1's. Then each successive m -bit codeword alternates between runs of 0's or runs of 1's, except when we have the codeword is $11 \cdots 1$ as explained in the previous step.

Example 9.1 (A Simple Example of Run-Length Coding)

Consider the following string of symbols:

00000 0 11111 111 00000 0000 11111 00000 00 11111 11111 11

This has alternating runs of 0's and 1's of lengths 6, 8, 9, 5, 7, and 12, for total length of 47 bits. Note that the spaces are not part of the string of symbols but are shown only to more easily count the lengths of runs. That is, a space is shown when the symbol changes from a 0 to a 1 or vice versa, and after five of the same symbols.

Using the method described above, the run-length code would be:

0 110 111 001 111 010 101 111 000 111 101

The first 0 indicates that the sequence begins with a run of 0's. The first 3-bit codeword 110, indicates that this first run of 0's has length 6. The next codeword 111 indicates a run of 1's of length 7 but we do **not** change symbols. Hence the next codeword 001 indicates another 1 for a run of 1's of total length 8. Then the next two codewords (111 and 010) indicate a run of 0's of length $7 + 2 = 9$, since the 111 codeword again indicates no symbol change.

■

Some remarks on run-length coding:

- As mentioned above, there are many variations that could be used. For example, the instead of having the codeword $11 \cdots 1$ indicate no symbol change, we could always change symbols, but then use the all zero codeword if we need to continue with a run longer than $M = 2^m - 1$. Likewise, instead of using the very first bit to indicate which symbol to start with, we could assume we always start with a run of 0's. Then if a given sequence actually starts with a run of 1's then the encode would begin with the all zero codeword $00 \cdots 0$. The important point is that many different choices can be made as long as the encoder and decoder agree on the algorithm implemented.
- The main parameter in run-length coding is the choice of the block length of m used to encode the run lengths. This choice should be made based on the distribution of run lengths we expect to encounter in the data. If m is too large, we will be using m bits in each block even though we might have done better using a smaller block size. If m is too small, then we won't be as efficient in coding long runs.
- Run-length coding as described above is a variable-to-fixed-length code. That is a variable number of original symbols get represented by a fixed number of bits. This contrasts with Huffman coding, which is a fixed-to-variable-length code (or more commonly simply called a variable-length code).
- Note that if we do not use a fixed block size to code the run-lengths then we would need a way to distinguish the codewords corresponding to separate runs. This could be done with a general uniquely decodable code, for example by combining run-length coding with Huffman coding. That is, instead of using codewords of some fixed length m to denote the run-lengths, we could use variable length codewords to encode the run-lengths. In this case, we would have a variable-to-variable-length code.
- For binary images, we can either code each row of the image separately, or we could order the pixels to get a single and longer bit stream.
- For gray scale images or other data sources with more than two symbols, instead of simply providing run-lengths, we could provide the character followed by the run-length. For example, the coding could be of the form

$$(g_1, \ell_1), (g_2, \ell_2), \dots$$

where g_1 is the first symbol (e.g., gray level) and ℓ_1 is the run-length of this symbol. Of course, this will work well only if there are large stretches of constant characters.

- In the case of gray scale images, another alternative is to separate the gray scale image in 8 binary images corresponding to the bit planes of the original image. Run-length coding often works well for the most significant bit planes, but generally will not work well for the least significant bit planes.

9.3 Lempel-Ziv Coding

In our discussion of Huffman coding, we noted that it requires prior knowledge of symbol probabilities $p_1, \dots, p_M$ to construct an optimal code. In many practical applications, these probabilities are either unknown or change dynamically as the data is processed. Furthermore, while block Huffman coding can approach the entropy of a source, the exponential growth of the codebook makes large block sizes impractical.

Lempel-Ziv (LZ) coding provides a powerful alternative because it does not require prior knowledge of the source statistics. Instead, it exploits redundancy by building a dictionary of sequences (or “phrases”) as it scans the data. When a sequence of symbols recurs, the encoder simply outputs a pointer (an index) to its previous occurrence in the dictionary.

There are two primary families of Lempel-Ziv algorithms:

- LZ77 (Sliding Window): This approach uses a look-back buffer to find the longest match for the current string of symbols within the recently processed data.
- LZ78 / LZW (Dictionary-based): This variation progressively builds an explicit dictionary table of strings. LZW (Lempel-Ziv-Welch) is a popular refinement used in the GIF image format and various file compression utilities.

The LZW algorithm starts with a dictionary initialized with all single-symbol expressions (e.g., 0 and 1 for binary data). As the encoder reads the input, it finds the longest string W already in the dictionary that matches the current input. It outputs the index of W and then adds W followed by the next symbol in the input to the dictionary as a new entry.

Example 9.2 (LZW Compression of a Binary String)

Consider the binary string: 01010101

Assume our initial dictionary is: Entry 0 = “0”, Entry 1 = “1”.

1. The first symbol is “0” (Index 0). The next is “1”. The string “01” is not in the dictionary. Output Index 0. Add “01” to the dictionary as Entry 2.
2. The current symbol is “1” (Index 1). The next is “0”. The string “10” is not in the dictionary. Output Index 1. Add “10” to the dictionary as Entry 3.
3. The current symbol is “0”. The next is “1”. The string “01” **is** in the dictionary (Entry 2). The next symbol is “0”. The string “010” is not in the dictionary. Output Index 2. Add “010” to the dictionary as Entry 4.
4. The current symbol is “0”. The string “01” **is** in the dictionary (Entry 2). The next string “010” **is** in the dictionary (Entry 4). The next string “0101” is not in the dictionary. Output Index 4. Add “0101” to the dictionary as Entry 5.
5. The current symbol is “1”. There are no more symbols to extend with. Output Index 1.

The resulting compressed output is the sequence of indices: 0, 1, 2, 4, 1. Note that as the dictionary grows, a single index can represent increasingly long strings of bits, leading to significant compression for highly repetitive data.

■

Some remarks on Lempel-Ziv coding:

- The algorithm is able to learn the structure of the data on the fly. For very long sequences from a stationary source, the average number of bits per symbol approaches the entropy of the source.
- A remarkable property of LZW is that the dictionary does not need to be transmitted to the decoder. The decoder can reconstruct the exact same dictionary bit-by-bit as it decodes the index stream.
- Since physical memory is finite, practical implementations must decide what to do when the dictionary is full (e.g., stop adding entries, or flush the dictionary and start over).

9.4 Transform Coding of Images (JPEG)

Transform coding is a lossy compression technique that works very well for images. Transform coding exploits correlations between neighboring pixel values in frequency domain (i.e., via a transform, though not necessarily a Fourier transform). The key idea is that in most images there is a large amount of correlation between neighboring pixels. Regions of an image where pixels are highly correlated correspond to smoother signals that have more energy at low frequencies. By adjusting the amount of quantization, we can compress the data but retain content in the image that is perceptually more important.

The main steps in transform coding are as follows:

1. Split the image into blocks. Usually the blocks are 8x8 or 16x16. This reduces computation and the small block size allows adjustment of parameters based on local image characteristics. For color images, we treat the three color components as three separate images.
2. Use a 2-D transform on each (8x8 or 16x16) block to get the same size blocks in transform domain. Often the transform used is the Discrete Cosine Transform (DCT), though other transforms can be used as well.
3. Quantize the transform coefficients.
4. Code the quantized coefficients with a zig-zag scan and a combination of Huffman and run-length coding.

The main steps in decoding are as follows:

1. Convert codewords back to transform coefficient values.
2. Perform the inverse of the 2-D transform used in coding to get back (an approximation of) the original image.

We now provide a little more details on each of the four steps in transform coding.

2-D Transform

For the 2-D transform, the DCT (Discrete Cosine Transform) is the most popular choice, although wavelet transforms have also received much attention. The DCT is very similar to the DFT,

but uses pure cosines instead of complex exponentials. In one dimension, the forward and inverse DCT are defined as follows:

$$X[k] = \sum_{n=0}^{N-1} x[n] c(k, n) \quad (9.1)$$

$$x[n] = \sum_{k=0}^{N-1} X[k] c(k, n) \quad (9.2)$$

where

$$c(k, n) = \begin{cases} \frac{1}{\sqrt{N}} & k = 0 \text{ and } 0 \leq n \leq N-1 \\ \sqrt{\frac{2}{N}} \cos \frac{\pi(2n+1)k}{2N} & 1 \leq k \leq N-1 \text{ and } 0 \leq n \leq N-1 \end{cases} \quad (9.3)$$

Note that the DCT is a real transform (as opposed to a complex transform). That is, the transform coefficients are all real values.

It turns out that for a wide range of natural images, the DCT compacts energy well into low frequency coefficients. Also, a version of the FFT can be used for a fast algorithm to compute the DCT.

Quantization/Bit Allocation

This is the main step that determines the amount of compression versus the quality of the decoded image. Usually only some fixed number of the transform coefficients are stored. The others are taken as 0.

Roughly speaking, the low frequency components are more important for the overall content – the appearance of larger objects and regions, colors, etc. The high frequency components represent the details and local variations in an image. What is “perceptually important” depends on the goals. To get a compressed image to look as close to the original image (say in terms of mean squared error) as possible, the low frequency components are more important. For the location and sharpness of edges or other fine detail, the high frequency components are more important. For example, if fewer bits were allocated to the low frequency components, colors might be off and the image might look blocky. If fewer bits are allocated to the high frequency components, edges and other fine details may be blurry.

In image compression, we generally want the compressed image to look as much like the original image in terms of overall content, and so more bits are generally allocated to the the lower frequency components and fewer bits are allocated to the high frequency components. A typical bit allocation table for an 8x8 block might look something like that shown in Figure 9.1. In the figure, the number in a given location indicates the number of bits used to quantize the corresponding coefficient. The lowest frequency coefficient (i.e., 0 frequency, or DC, in both directions) is in the upper left and is typically allocated the most bits (in this case, 8 bits). The missing entries (the last row, last column, and much of the lower right of the 8x8 block) indicate that no bits are allocated to those coefficients, and hence these are taken as 0.

Often the encoder and decoder agree on a fixed set of such tables, and the encoder send a “q-factor” (quantization factor) to let the decoder know which table was used. Usually, the same q-factor is used for every block in the image.

If the quantization tables are not agreed upon beforehand, then the decoder would have to send/store the quantization table as part of the coding. This could be done, for example, with threshold coding in which the largest coefficients are sent.

8	7	6	4	3	2	1
7	6	5	4	3	2	1
6	5	4	3	3	1	1
4	4	3	3	2	1	
3	3	3	2	1	1	
2	2	1	1	1		
1	1	1				

Figure 9.1: A possible bit allocation table for an 8x8 block. The missing entries indicate that the corresponding DCT coefficients are taken as 0.

Coding of Quantized Coefficients

For each block, the DC coefficients are usually coded separately from the other coefficients in the block. The DC coefficients from each of the blocks are usually coded together using some type of predictive coding.

The other coefficients (i.e., the AC coefficients) in each block are typically ordered using a zig-zag scan, starting with the low frequency coefficients and ending with the high frequency coefficients. See Figure 9.2. Many of the higher frequency coefficients tend to be zero, and the zig-zag scan groups these together so that there is often a long run of zeros at the end.

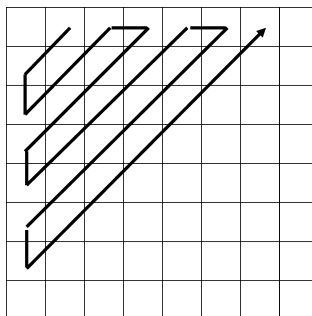


Figure 9.2: Zig-zag scan ordering of DCT coefficients. The DC coefficient (upper left in figure) is typically coded separately with DC coefficients from other blocks

These coefficients are then coded with a combination of Huffman and run-length coding. Standard tables for these codewords that tend to work well have been developed and these are part of the coding standard.

9.5 Video Coding (MPEG)

In this section, we discuss the basics of the MPEG standard for video coding. MPEG stands for Moving Pictures Expert Group and is a group that develops coding standards for a variety of media

including audio and video. We will focus on the the video coding aspect of the MPEG standards and will only cover some of the key ideas.

Recall the a video can be thought of as a sequence of still images or frames. In the case of transform coding of images, we exploited the spatial redundancy of the gray levels in an image. That is, nearby pixels tend to have similar gray levels, and by exploiting this correlation we can get significant gains in compression. For video coding, we exploit temporal redundancy as well as spatial redundancy. For many natural videos, much of the content of a given frame is very similar to the previous frame. We might be able to exploit this by considering just the change (or difference) between the current frame and the previous frame. This is a kind of predictive coding. Namely, if we have the decoder has the image/frame at time $t - 1$, then to code the new frame at time t , we don't need to start from scratch. Instead, we could code just the differences between the frame at time $t - 1$ and that at time t .

This simple idea results in a good amount of compression, but a natural question is whether we can do anything better. It turns out that much of the difference between two consecutive frames comes from objects moving between the two frames, with much of the background (non-moving parts of the frame) being the same in the two images. Hence, we might be able to further exploit the temporal redundancy if we are able to “compensate” for this motion.

One approach to describing the motion from one frame to the next is to say where each pixel move from the frame at time $t - 1$ to the frame at time t . However, this results in too much overhead to describe the motion of every pixel. A different approach is to segment the the frame into objects and give information about how each object moved from one frame to the next. The problem with this is that segmentation of an image into objects is a challenging problem for just one image, and much more so for video of about 30 frames per second. Also, describing the shape of the segmented objects would require overhead as well.

A compromise is to use block-based motion compensation. That is, we divide the image into blocks and describe how the entire block moved from one frame to the next. This gives efficiency by describing the motion of a block rather an each individual pixel. Actually, instead of describing where a block in the frame at time $t - 1$ moved to, we consider a block in the frame at time t and ask where it came from in the previous frame. We describe more detail below.

This idea of motion compensation is combined with two other ideas to form the basis of the MPEG standard of video coding. These other two ideas are transform coding (described in the previous section) and something called a group of pictures structure. Thus, the basic idea of the MPEG approach to video coding is:

- Block-based motion compensation
- Transform coding
- Group of pictures structure

We now give a bit more detail on these three steps.

Block-based Motion Compensation

Divide the current image that we are coding into blocks – 8x8 blocks is typical. We group four of the 8x8 blocks into a macroblock, so that a macroblock is 16x16. In the case of color video, two of the color components are usually subsampled by a factor of two. Hence, a macroblock has a 16x16 luminance blocks (i.e., four 8x8 luminance blocks) together with two subsampled 8x8 blocks for the color components.

For each macroblock, find where in the previous frame image the macroblock matches best – i.e., that minimizes some error criterion. A variety of criteria can be used to determine similarity, but two common criteria are the sum of the squared difference and the sum of the absolute difference between the current macroblock and the corresponding gray levels in the previous image at the location of the best match. Store the motion vector for this macroblock and compute the macroblock difference, or the displaced frame difference.

Transform Coding

For each macroblock, the displaced frame difference is coded using transform coding as described in the previous section. For example, it is typical to code the displaced frame difference using the DCT followed by quantizing and coding the transform coefficients.

Group of Pictures Structure

The frames coded using block-based motion compensation as described above are called P-frames, or predictively coded frames. Several problems arise with using motion compensation as described above on every frame, i.e., with all frames being P-frames. First, as with any predictive coding method, errors will propagate since new frames are coded in reference to previous frames. Another issue with predictive coding methods is that a decoder can't begin decoding at a random point in the transmission stream. Without the initial frame and all of the coded differences, a user that tries to start decoding midstream will simply get the residuals but won't be able to recover even an approximation to the actual current frame.

To address these issues, the encoder and decoder agree to periodically have some frames coded by themselves – i.e., *not* predictively. These frames are called *intracoded* frames, or I-frames. Errors might propagate, but only until the next I-frame. Once the next I-frame is encountered, the encoder will code it without reference to any previous frames. This also addresses the problem of a decoder that joins midstream. At first, a decoder that joins midstream will be getting residuals without the reference frame, but once the next I-frame is encountered the decoder will be able to decode as usual.

Another modification can allow for better compression. When an object moves, the background behind the object appears in the frame at time t but this background was not visible in the previous frame at time $t - 1$. Thus, this "uncovered" part of the image in the current frame has no matching portion in the previous frame. However, in the next frame at time $t + 1$, the uncovered area is likely present and still uncovered. If we could predictively code using a future frame, we might be able to improve the compression for that portion of the current frame. We can accomplish this by introducing some bidirectional frames, or B-frames, that can use a past frame or a future frame for the predictive coding using motion compensation.

Putting all this together leads to the creation of a structure along the lines of the following:

I B B P B B P B B P B B P B B I

This shows 16 frames in all including the two I-frames at the beginning and the end, or 15 frames counting the first I-frame but not counting the last I-frame. As described above,

- I-frames: are intracoded using transform coding without reference to any other frame.
- P-frames: are predictively coded using block-based motion compensation using the most recent I-frame or P-frame

- B-frames: are bidirectionally coded (or bipredicted) frames using the nearest I- or P-frames in the past and future. Therefore, we must wait to code a B-frame until after the next I- or P-frame.

The collection of frames from one I-frame to the next is called a group-of-pictures (or GOP) structure. I-frames and P-frames are called anchor frames. Two parameters that determine the GOP structure are the number of frames between anchor frames, and the total number of frames in the GOP, from one I-frame to the next I-frame.

To summarize, the main steps in MPEG coding are:

1. Create/choose a group-of-pictures structure.
2. Split the current frame into blocks and macroblocks.
3. For P- and B-frames find the best match for each macroblock in a search area from the previous anchor frame (for P-frames), and from the previous or next anchor frame (for B-frames). Note that to code a B-frame, we need to wait and first code the next anchor frame.
4. Store the motion vector for each macroblock and compute the macroblock difference.
5. Use transform coding (DCT, quantization, zig-zag scan, Huffman/run-length coding) to code the macroblock difference. Note that for I-frames, we don't use motion compensation or compute the macroblock difference. That is, for I-frames, use transform coding but no predictive coding.

Chapter 10

Information Protection

In many systems, there is often a desire and/or need to protect information for a variety of reasons and in a variety of ways. There are several areas of work dealing with information protection to address different needs including:

- **Error detection and correction codes** This area is also commonly called coding theory. The goal of these techniques is to add redundancy (extra bits) to protect against (usually) random errors. The data is expanded to try to get reliable communication through an unreliable (noisy) channel.
- **Cryptography** This term is often used interchangeably with the term cryptology, although sometimes these terms are used with slightly different meanings. The goal of these techniques is to encrypt data to make it unintelligible except for intended users. Using suitable keys, the intended users can decrypt the string of symbols to recover the original data. These techniques are typically used for security or authentication.
- **Spread spectrum communications** This area was originally developed for military applications to avoid jamming or interception of radio communications. If the adversary knows the frequency band which is being used for communication, then they could “tune in” and eavesdrop or transmit noise at the same frequency to jam/corrupt the transmission. Roughly, spread spectrum techniques use a broad range of frequencies in various ways to reduce eavesdropping and jamming. These techniques now play a major role in mobile/wireless communications.
- **Watermarking** This area has seen activity in recent years as various forms of media are widely available in digital form. Watermarking is closely related to topics known as steganography and data hiding. The goal is to keep data intelligible, but utilize redundancy in the data to embed additional information. For example, for an image, one might like to add either a visible or invisible watermark for ownership protection or other purposes.

In this chapter, we focus only on error detection and correction.

*©2026 by Sanjeev R. Kulkarni. All rights reserved.

[†]Lecture Notes for ECE201 Information Signals.

[‡]Thanks to Sean McCormick for producing the figures.

10.1 Expansion for Protection

As mentioned above, the goal of error detection and/or correction is to deal with unintended noise in data. The key idea is to add redundancy to the data so that there is some internal consistency or structure that can be used to combat the noise.

For example, a trivial idea to detect errors is to repeat the data twice. If we wish to send the message 00101, we will instead transmit 00101 00101. If the recipient gets something other than the same copy twice, they will know an error has occurred. However, the receiver will not be able to correct the error, since they will not know which copy contains the error. But, the receiver may then ask for yet another transmission!

To be able to correct a single error, we could repeat the message three times and send 00101 00101 00101. In this case, if the transmission incurs a single error anywhere in the message (e.g., becomes 00101 01101 00101), the receiver will still have two copies of the original (uncorrupted) message that match, and so will be able to correct the error. Unfortunately, these simple methods expand the data so much that they are not very practical. However, far better techniques exist to accomplish similar goals much more efficiently.

The main point is that we build resilience to noise by expanding the data. This contrasts with compression, where we were trying to eliminate redundancy to reduce the data as much as possible. Here we are adding back some redundancy. It turns out that doing both can be very productive. The reason is that redundancy in the original data is either not highly structured, or the structure is not known well enough to be fully exploited. Therefore, we can reap substantial gains by removing the original redundancy and then adding some redundancy back but in a careful and highly structured manner. In this way, we can reduce the overall data size *and* increase the resilience to noise.

The techniques are so useful that some kind of error detection or correction is routinely used in applications involving data transmission and/or storage.

10.2 Noise and Level of Protection

As one might expect, the suitability of a technique depends greatly on assumptions about the noise. Three common types of noise in binary data are substitution errors (in which a 0 turns into a 1 or vice versa), erasures (also called deletions, in which a bit is lost), and insertions (sometimes called additions, in which an extra bit is inserted). The most common models for noise are the erasures and substitution errors, which are the focus of this chapter.

In addition to the type of noise, the suitability of an error protection technique depends on both the tolerable error rate in the system as well as on the distribution of the noise. In some applications we can tolerate a large amount of error. For example, in sending a fax, a small amount of noise will not cause a problem and may not even be noticed. Even if one in every ten thousand pixels gets corrupted, the result will still be useful. On the other hand, if we are transmitting an executable program file of a couple megabytes, even one error will likely render the transmission useless. Financial data or other records may also require an extremely high level of reliability.

Typically one designs a method to protect up to some number of errors, for example, single error detection, triple error detection plus double error correction, etc. When we say a code provides a certain level of protection, we mean that it can detect and/or correct *any* error of that particular type. For example, a code that provides single error correction against substitution errors can correct any single substitution error, including errors in the check bits added to provide the protection. Some single error correcting codes may happen to be able to correct certain double

errors, but unless the code can correct all double errors it will not be considered a double error correcting code.

Here we will primarily focus on single error detection and single error correction, but we briefly discuss the conceptual approach for protecting against multiple errors. We will also focus almost exclusively on the case of binary data – i.e., just 0's and 1's (except for ISBN numbers that we will describe briefly). Many of the ideas in the binary case can be extended to non-binary data, and binary data is the most commonly encountered setting in applications.

Often, the protection method will be applied by dividing the data into units or blocks, e.g., 8 bits for each gray level or symbol, or 128 bits for a set of symbols or other packet of data. Redundancy is added to each data unit to provide the desired level of protection. The level of protection chosen may depend on the probability of error for each bit, the size of the data units, and the tolerable level of corrupted units.

For example, suppose we are working with 8 bit codewords and can tolerate an average of one erroneous codeword out of every million codewords. Let p be the probability that a given bit gets corrupted. If p is small enough (say less than 10^{-7} so that only one bit in every ten million gets corrupted), then the number of corrupted codewords may be within tolerable bounds. In this case, we may not need any error protection since the probability of error itself is sufficiently small. On the other hand, if a single error per codeword occurs more often than 1 in a million, but two or more errors are very unlikely, then single error correction may be enough. Even single error detection may be enough if re-transmission of corrupted data units is possible. The main point is that the level of protection we need to design for depends greatly on the probability of an error, size of data units, and tolerable losses in the data.

Actually, in some cases knowing just the probability of error is not enough. The distribution of noise may also be important. For example, if errors are expected to be “bursty,” then if an error occurs several consecutive bits may get corrupted. Refinements to the basic techniques may be necessary to deal with such cases, although we will not deal with such issues here.

10.3 Erasure Correction

An erasure occurs when the receiver does not decide that a transmitted bit is a 0 or a 1, but instead declares that the bit is unknown. In many applications this is not a fundamentally different type of noise, but rather an engineering choice made at the receiver. The first step of the receiver is *Demodulation*, mapping the analog received voltage to a finite number of discrete levels. For example, if the received analog signal falls clearly near one of the two expected levels, the receiver decodes it as a 0 or a 1, while if the signal falls in an ambiguous region near the decision boundary, the receiver may simply declare an erasure. Thus an erasure means that the receiver knows that something went wrong at a particular location. This is the main difference between erasures and substitution errors: in the erasure case the decoder knows where the uncertainty is, whereas in the substitution case it does not even know whether an error has occurred. For this reason, erasures are generally easier to detect and correct than substitution errors.

The simplest way to correct erasures is to use feedback. Since the receiver can tell whether a transmitted bit was received or erased, it can send a signal back to the transmitter indicating success or failure. If the bit is erased, then the transmitter simply sends it again, and this continues until the bit is received without erasure. If the probability of erasure is p , then the probability of successful reception on any given transmission is $1 - p$, and hence the expected number of transmissions required for one successful bit is $1/(1 - p)$. If we are using T time instants to communicate, this means that the number of coded bits is T , one per each time instant (or channel use). The coding

ratio R is the ratio of information bits to the total number of bits. With a coding rate of R , the number of information bits is $B = RT$. In this case is $R = 1 - p$. No scheme can do better, i.e., $R \leq 1 - p$, since on average only a fraction $1 - p$ of the transmitted bits are received at all.

If feedback is not available, the transmitter must add redundancy in advance so that erased bits can be recovered from the bits that are successfully received. This recovery is performed during the *Decoding* stage, in which the receiver reconstructs the original information bits from the received bits. A convenient way to describe this is with a linear code. Suppose the transmitter starts with RT information bits and maps them into T coded bits according to

$$\begin{bmatrix} \text{vector of} \\ \text{coded bits} \end{bmatrix} = C \begin{bmatrix} \text{vector of} \\ \text{information bits} \end{bmatrix}.$$

where C is a $T \times RT$ binary matrix and $R < 1$ is the coding rate. If the erasure probability is p , then the decoder receives roughly $(1 - p)T$ coded bits reliably, together with the knowledge of which positions were erased. The unerased outputs therefore give a linear system

$$\begin{bmatrix} \text{vector of} \\ \text{unerased bits} \end{bmatrix} = \tilde{C} \begin{bmatrix} \text{vector of} \\ \text{information bits} \end{bmatrix}.$$

where $\tilde{C}$ is the submatrix obtained from C by keeping only those rows corresponding to unerased positions. In order to recover original information bits uniquely, we first need at least as many equations as unknowns, i.e.,

$$(1 - p)T \geq RT,$$

or equivalently $R \leq 1 - p$, which is consistent with the statement in the previous paragraph. And $\tilde{C}$ also needs to have a full column rank RT , to ensure that the linear system has a unique solution. Thus, the coding problem becomes the problem of designing a coding matrix C so that the original information bits can still be recovered from a sufficiently large set of unerased bits.

One possible approach is to choose the coding matrix at random. If each entry of C is chosen independently to be 0 or 1 with equal probability, then with high probability the resulting code has the desired rank properties, and hence can recover from erasures whenever the rate satisfies $R \leq 1 - p$. This shows that good erasure codes exist. However, a completely random matrix is usually not practical, since both encoding and decoding may be computationally expensive for dense coding matrices.

In practice one wants codes that are explicit, sparse, and efficiently decodable. This leads to *Digital Fountain Codes* and related sparse linear codes. Here each transmitted symbol is a simple combination of only a few message symbols, so the matrix C has mostly zero entries. Decoding can then be carried out by a simple *message passing algorithm*. One first looks for any equation involving only a single unknown symbol and solves for that symbol. One then substitutes this value into all other equations, which may reduce some of them to new single-variable equations. The process is repeated until either all message symbols have been recovered or no single-variable equation remains, in which case decoding stalls. For example, suppose the message symbols are x_1, x_2, x_3, x_4 , and the receiver obtains the equations

$$y_1 = x_2 = 1, \quad y_2 = x_1 + x_2 = 0, \quad y_3 = x_2 + x_3 = 1, \quad y_4 = x_3 + x_4 = 0,$$

where addition is modulo 2, that is, an XOR operation ($\oplus$). From the first equation we obtain $x_2 = 1$. Then we apply an XOR operation on the known symbol x_2 and an equation containing x_2 , i.e., $x_2 \oplus y_2$, which eliminates x_2 from the equation and gives $x_1 = 1$. The third equation gives $x_3 = 0$, and finally the fourth gives $x_4 = 0$. In this way, sparse linear codes provide a practical method for correcting erasures without feedback.

10.4 Single Error Detection

As mentioned above, repeating the message (or each bit) twice provides a simple scheme for detecting single errors. The downside is that this doubles the amount of data. Some partial consolation for this excess data is that repetition is robust to essentially any type of single error (since the two copies won't match and hence error will be detected). It can even detect many types of double (or more) errors (except with presumably small probability when both copies happen to get corrupted in the same way). Nevertheless, the price of doubling the data is too great, and if one is concerned about double errors or other types of errors other techniques may be more suitable.

Parity Check Bits

A simple and elegant alternative to detect single substitution errors is to add an extra bit, so that the number of 1's in the data unit (including the check bit) is even. In this case, the resulting message including the check bit is said to have even parity. To determine whether or not there is an error in a received data unit, the receiver simply checks the parity of the received data. If it is odd, then clearly there was an error. If it is even, no error is assumed to have occurred. Of course, in this case, if two errors occur then these will not be detected. Instead, the receiver will assume the data received is error-free.

Of course, we could have chosen to use odd parity, instead of even parity. That is, we could have appended a check bit to make sure the number of 1's in the data unit is odd. This will work just as well. The main requirement is that the encoder and decoder must agree on whether to use even or odd parity.

A parity check bit provides single error detection with a single additional bit. If the probability of error for each bit is quite small, then the probability of a double error will be extremely small. In this case, a parity check bit can be quite adequate, especially in applications where the data can be re-transmitted or re-read. Thus, for single error detection parity check bits are the most widely used scheme.

ISBN numbers

The performance of a protection scheme depends on type of noise encountered, and therefore the scheme should be designed taking knowledge of noise properties into account. We end this section with an example designed for commonly occurring human errors.

In 1970, a 10-digit International Standard Book Number (or ISBN) format was introduced to give every book an identifying number. You can usually find the ISBN on the back of most books near the bottom. The ISBN gives a standardized way to identify a given book. E.g., the book *Information Theory: 50 Years of Discovery* published by the IEEE Press, has the ISBN 0-7803-5363-3. In 2007, a new 13-digit ISBN format was introduced.

Let's first consider the 10-digit ISBN and denote the 10 digits by $d_{10}d_9 \dots d_2d_1$. So, for the book mentioned above, $d_{10} = 0$, $d_9 = 7$, and so on.

The digit d_{10} indicates whether the book is published by a U.S. or international publisher. The next two digits d_9d_8 identify the publisher. The digits d_7 through d_2 are a book number assigned by the publisher.

The last digit, d_1 is a check digit. It is chosen so that the quantity

$$10d_{10} + 9d_9 + \dots + 2d_2 + d_1$$

is divisible by 11. For example, for the book mentioned above, we get

$$10 \times 0 + 9 \times 7 + \dots + 2 \times 3 + 3 =$$

A natural question is why the weighted sum was used (instead of a check using an unweighted sum)? Both a weighted sum and an unweighted scheme allow detecting any single substitution error. However, a common human error is to switch the order of adjacent digits. The weighted sum scheme can detect this (for any adjacent digits), while an unweighted sum clearly cannot (since switching digits would leave the sum unchanged).

What happens in a 10-digit ISBN if d_1 needs to be 10 in order to make the weighted sum divisible by 11 (for example, if $10d_{10} + \dots + 2d_2 = 210$)? The convention was to use the symbol “X” as the last “digit” in this case.

The 13-digit ISBN format introduced in 2007 uses a different method to determine the check digit. Let $d_{13}, d_{12}, \dots, d_1$ denote the 13 digits, where d_1 is the last digit. Then d_1 is chosen so that a weighted sum of all 13 digits is divisible by 10, where the weights alternate between 1 and 3. That is, d_1 is chosen so that

$$d_1 + 3d_2 + d_3 + 3d_4 + \dots + d_{11} + 3d_{12} + d_{13}$$

is divisible by 10. This scheme does not allow the detection of all transposition errors, but does not require the use of the special character “X” for d_1 as needed in some cases for the 10-digit ISBN.

10.5 Single Error Correction: Preliminaries

As with single error detection, a simple scheme for single error correction is repetition, namely just repeat the message 3 times. In this case, to correct an error the receiver can simply take a majority vote, bit by bit. Although this triplicate code is simple and quite robust to different types of errors, it’s not practical because it triples the amount of data. Much more efficient schemes exist that we will discuss in the next three sections, focusing only on substitution errors. However, first we discuss some preliminaries.

Some clarification is needed on our use of terms describing a certain level of protection. First, it’s clear that by repeating the message three times, certain (in fact many) double errors can be detected (as well as corrected). But, does the repetition code allow detecting *all* double errors?

In a sense one might say yes, since if there are any two errors then certainly not all three copies of the message will be the same. Therefore, the receiver will know that something is wrong. However, the problem with this is that if we insist on correcting single errors, then *certain* double errors (namely, an error in the same bit in two of the copies) will be incorrectly assumed to be *single* error, and therefore will be “fixed” incorrectly.

For example, suppose the original message is 00101 so that 00101 00101 00101 gets transmitted. If two errors occur and they happen to be the first bit in the first two copies, then the data received will be 10101 10101 00101. In this case, a receiver that is implementing single error correction will erroneously think that there is a single error in the first bit of the third copy, and will “correct” this error concluding that the original message was 10101.

Now, if we are not interested in correcting single errors, then any double errors could be detected properly. There might be cases in which we care more about detecting double errors than correcting a single error (for example, if we can ask for the message to be retransmitted). Of course, if a code can correct a certain number of errors, then it can surely detect that same (or fewer) number of errors. Thus, if we correct as many errors as a code is capable of correcting, then the hierarchy in terms of level of protection is (1) single error detection, (2) single error correction, (3) single error correction plus double error detection, (4) double error correction, (5) double error correction plus triple error detection, etc.

Now we introduce some notation and terminology that will let us better compare the performance of different codes. An (n, m) code will refer to a code with data units that have:

- n total bits
- m message/information bits
- $k = n - m$ check bits

The *redundancy* and *excess redundancy* of the code are defined to be

$$\text{redundancy} = \frac{n}{m} = 1 + \frac{k}{m} \quad (10.1)$$

$$\text{excess redundancy} = \frac{k}{m} \quad (10.2)$$

For a given level of protection, we would like the redundancy to be as small as possible.

As an example, consider the case of the triplicate code. With m message bits, we will have $n = 3m$ total bits and $k = 2m$ check bits. Therefore, the redundancy is 3 and the excess redundancy is 2. It so happens that for the triplicate code, the redundancy and excess redundancy do not depend on the size of the data unit (i.e., the number of message bits m or the number of total bits n). However, in general, for a given lass of codes the redundancy decreases as the size of the data unit increases.

For example, for single error detection with a parity check bit, the redundancy is $1 + 1/m$ and the excess redundancy is $1/m$. The larger the block to which we append the check bit, the smaller the redundancy. The tradeoff is that as the size of the block increases, the probability that one or more of the bits is corrupted increases. If it's too large, then an unacceptable number of blocks may have single or multiple (possibly undetected) errors.

10.6 Rectangular and Triangular Codes

Rectangular Codes

A clever yet simple method for single error correction is to arrange the block of message/information bits in a rectangular array, and then add a parity check bit to each row and each column. Figure 10.6 shows the arrangement of message and check bits with a 3x4 array of message bits.

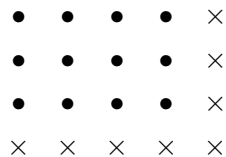


Figure 10.1:

The check bit at the end of a row is chosen to make all the one in that row (including the check bit) have the appropriate parity. Similarly for the columns. Any single error can be corrected by finding the row and column for which the parity check fails.

The check bit in the the corner is actually not necessary. If it's used, the check bit in the corner will check both the row and column of check bits. That is, the corner bit is chosen to make all the

check bits (including the corner) have the proper parity. In this case, if there's an error in a check bit, then both the corresponding row or column check will fail and the corner check will fail. If we don't use the corner bit, then everything will work as before if the error is in one of the message bits. If the error is in a check bit, then only the check for that row or column will fail. In this case, the arrangement of message bits and check bits is as shown in Figure 10.2.

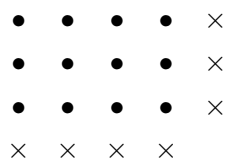


Figure 10.2:

Example 10.1 (Example of a 3x4 Rectangular Code)

Suppose the original message is 101101011000. Arranging this into a rectangular array we get

```

  1  0  1  1
  0  1  0  1
  1  0  0  0

```

Using even parity, the message with the check bits is

```

  1  0  1  1  1
  0  1  0  1  0
  1  0  0  0  1
  0  1  1  0

```

Now suppose the receiver observes a possible corrupted bit stream. The receiver would arrange the bits in the same way, and perform the parity checks on each row and column as shown below.

```

  1  0  1  1  1
  0  1  1  1  0  F
  1  0  0  0  1
  0  1  1  0
  F

```

Since the parity checks in row 2 and column 3 fail, this indicates that the position of the error is the bit in the 2nd row and 3rd column. The receiver can therefore correct the error and recover the original message. ■

There are several things to note about this error correction scheme:

- As before, it doesn't matter whether we use even parity or odd parity, as long as both the coder and decoder agree.

- The picture of a rectangular arrangement helps to easily explain the method, but of course we don't actually have to arrange the bits in a rectangle in implementing this. We just need to add the check bits that perform the necessary parity checks. The data are arranged in a stream, and the coder and decoder only need to know (agree) which subsets are supposed to satisfy the parity checks.

The redundancy of a general rectangular code with an array of a_1 by a_2 message bits can be computed, but for simplicity, we consider the special case of a square code. For $m = a^2$ message bits using a square code, we have $k = 2a$ and $n = a^2 + 2a$ (assuming we do not use the extra corner check bit). In this case, the redundancy is $1 + 2/a$ and the excess redundancy is $2/a$. Even for $a = 2$, this code is more efficient than the triplicate code, and the benefits clearly get much greater for larger a .

An alternate way to compare the codes is to fix the number of check bits k and see how many message bits can be accommodated. For the triplicate code, $m = k/2$, but for the square code, $m = (k/2)^2$.

Triangular Codes

A more efficient code can be obtained by arranging the message bits in triangular array, and adding another layer (diagonal) of check bits, as shown in Figure 10.3.

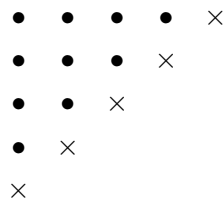


Figure 10.3:

Each check bit checks *both* the row and column to which it belongs. The check bit makes sure that the total number of ones in the row and column together, including the check bit itself, is of the right parity.

If there is a single error in a message bit, then two check bits (in both the corresponding row and column) will fail, and the receiver will know which message bit is in error. If a check bit is in error, then only that check bit will fail the parity check. In either case, a single error can be corrected. An example is shown in Figure 10.4.

As with the rectangular codes, although the picture helps conceptually, in an implementation one does not really need to arrange the bits in a triangular array. Instead, check bits are added that simply check the appropriate subsets of bits.

To get an idea of the efficiency of the triangular code, consider a fixed number of check bits k . The number of message bits that can be accommodated is

$$m = 1 + 2 + \cdots + (k - 1) = \frac{k(k - 1)}{2}$$

This is larger than the number of message bits for the square code for any $k \geq 2$, which is always the case for these codes.

0	0	x	0	x	
0	0	0	0	✓	(a.)
0	0	0	0	✓	
✓	✓	x	✓		

0	0	0	0	✓	
0	0	0	0	✓	(b.)
0	0	0	0	✓	
✓	✓	x	✓		

Figure 10.4: Locating a single error in a rectangular code: (a) shows an error in a message, and two check bits indicating an error; (b) shows only one check bit that indicates an error, implying the error is in the check bit.

10.7 Higher Dimensional Codes

An idea that leads to codes even more efficient than the triangular codes is to arrange the data in a 3-D cube. A check bit would be added for each plane. That is the check bits would be added along the three edges of the coordinate axes, as depicted in Figure 10.5. Each check bit would be chosen so that the number of 1's in all bits that lie in the corresponding plane have even parity. Note that, as before, we don't need to add a check bit in the corner.

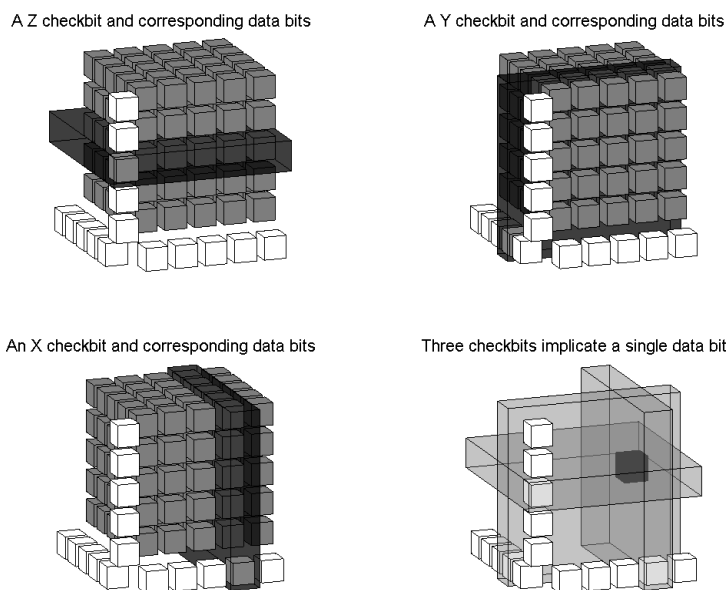


Figure 10.5: A rectangular code with parity checks in 3-D.

If the data (including the check bits) is arranged in a cube of size $s \times s \times s$, then there are $n = s^3 - 1$ total bits (the -1 is because of the missing corner). These include $k = 3(s - 1) = 3s - 3$ check bits

resulting in $m = s^3 - 3s + 2$ message bits. Thus, the excess redundancy is $(3s - 3)/(s^3 - 3s + 2)$. For large s , this is approximately $3/s^2$, which is better than the results in 2-D.

These results suggest that yet more efficiency can be obtained if we go to even higher dimensions. Of course, as before, in an implementation we won't actually arrange the data into a hypercube. This is even difficult to depict in a figure. Nevertheless, we can add check bits that perform the necessary checks. If this is done, then in 4-D we will get an excess redundancy of approximately $4/s^3$ for large s where the bits (message and check bits) are arranged in a hypercube with s bits along each dimension.

What's the highest dimension possible? Well, we need at least 2 positions for bits along each dimension (otherwise we haven't actually used the dimension). So, with 2 positions along each of the d dimensions, we will have $n = 2^d - 1$ total bits (again, the -1 is for the missing corner). We have only 1 check bit per dimension. This is because each edge has 2 positions, one of which is the corner. Hence, we will have $k = d$ check bits, resulting in $m = 2^d - d - 1$ message bits. Each check bit checks the hyperplane perpendicular the coordinate axis that contains the checkbit.

Can we do any better by using some other tricks? The answer is no, and we can actually see this with a relatively simple argument. Suppose we have k check bits and wish to correct single errors. When we receive a string, we will consider each check bit to see if the corresponding parity check passes. There are only two possibilities for each check bit — either the test passes or not. Thus, with k check bits, we can represent 2^k situations. On the other hand, the situations we must be able to represent include the case of no error, and n situations for the case that the error happens to be in each of the n bits. This gives $n + 1$ situations that we must be able to represent. Therefore, we must have $n + 1 \leq 2^k$ so $n \leq 2^k - 1$. The d -dimensional case described above achieves this constraint with equality.

The discussion above was rather abstract. Although it suggests what is the best we can do for single error correction, it would be nice to have a concrete algorithm that is easy to understand. This is the subject of the next section.

10.8 Hamming Codes

We'll describe the (7,4) Hamming code, which means there are 7 total bits, 4 message bits and 3 check bits. This satisfies the optimality condition $n = 2^k - 1$ as one can easily check.

Let the check bit positions be at powers of 2, so that out of the 7 bits, the 3 check bits will be at positions 1, 2, and 4. The message bits will be placed in the remaining 4 bits (positions 3,5,6,7), as shown:

1	2	3	4	5	6	7
↑	↑		↑			
c	c		c			

Every number between 1 and 7 can be expressed by adding some combination of the numbers 1, 2, and 4. This is just the binary expansion. Each check bit will check those positions whose binary representation uses the corresponding value. That is,

1	checks positions	1, 3, 5, 7
2	checks positions	2, 3, 6, 7
4	checks positions	4, 5, 6, 7

We will use even parity, so that, for example, check bit 1 is chosen to make the number of 1's in positions 1, 3, 5, 7 even.

Example 10.2 (Example of Coding)

Consider the message 1011. To find the codeword, we place the message in positions 3,5,6,7, as shown:

$$\begin{array}{ccccccc} _ & _ & 1 & _ & 0 & 1 & 1 \\ 1 & 2 & 3 & 4 & 5 & 6 & 7 \end{array}$$

Let b_i denote the value of bit i . The check bits are as follows:

check bit 1 : $b_1 = 0$ to make $b_1 + b_3 + b_5 + b_7$ even

check bit 2 : $b_2 = 1$ to make $b_2 + b_3 + b_6 + b_7$ even

check bit 4 : $b_4 = 0$ to make $b_4 + b_5 + b_6 + b_7$ even

Therefore, the codeword is 0110011. ■

Error correction is also quite simple. To correct a single error, we find which check bits fail the parity check (i.e., have odd parity), and then we add the corresponding values. For example, if check bits 1 and 4 fail, but check bit 2 passes, we add $1 + 4 = 5$. The result tells us which bit has the error. If all checks pass then we have a valid codeword (no errors), and the original message is in positions 3, 5, 6, 7 as they were received.

Example 10.3 (Example of Error Correction)

Suppose we receive 0110111. How can we tell which bit (if any) has an error? We consider each check bit:

check bit 1 : $b_1 + b_3 + b_5 + b_7$ is odd $\rightarrow$ fails

check bit 2 : $b_2 + b_3 + b_6 + b_7$ is even $\rightarrow$ passes

check bit 4 : $b_4 + b_5 + b_6 + b_7$ is odd $\rightarrow$ fails

Since check bits 1 and 4 failed, the error is in position $1 + 4 = 5$. Therefore, the corrected codeword is 0110011 and the original 4 bit message in positions 3, 5, 6, 7 is 1011. ■

Interestingly, the way the code is constructed, the position of the error can also be seen by considering the binary number formed by the check bit results. Specifically, c_1, c_2, c_4 represent the parity check results where 0 represents passing and 1 represents failure of a parity check. This is the same as adding the bits to be checked modulo 2. Then form the string $c_4c_2c_1$. The result gives the binary representation of the location of the error. For the codeword above, checks 1 and 4 failed while check 2 passed, giving the 101 which is the binary representation of 5. This means the error is in position 5 as we saw above.

Codewords for four symbols with a parity check			
Symbol	2 Bit Messages	Check bit	Valid Codewords
s1	0 0	0	0 0 0
s2	0 1	1	0 1 1
s3	1 0	1	1 0 1
s4	1 1	0	1 1 0
			Other (Invalid) Codewords
			0 0 1
			0 1 0
			1 0 0
			1 1 1

Figure 10.6: Valid and invalid 3-bit codewords with two message bits and one check bit.

10.9 Valid Codewords and Invalid Strings

We've seen that there are a variety of codes for single error detection and single error correction. Via a representation argument we were able to deduce the best we can do for single error correction using k check bits. The Hamming code achieves this optimality when the total number of bits in the data unit is of the form $n = 2^k - 1$. But how can we design good codes for other values of n ? And, what about other levels of protection?

In this section, we describe a unifying conceptual view of error detection and correction. It will give a useful way to think about the requirements for good codes of a given size and level of protection, although unfortunately it won't tell us exactly what the code should be. Finding good codes and studying their properties is still an active area.

Using codewords of a given length, a key property that allows any kind of error protection is that only *some* strings are “valid” codewords that correspond to messages. The others are “invalid” strings that arise due to errors.

We assume that the transmitter always sends us a valid codeword corresponding to the message they wish to send. If *every* codeword were “valid” then there's no hope for any kind of error detection or correction. In this case, no matter what bit string we receive, our best guess is that this is what the transmitter intended to send. On the other hand, consider the case where only some subset of strings are valid. We know that these are the only codewords the transmitter would send us. If we receive one of these valid codewords, then we assume there have been no errors, so that the bits received are exactly the ones that were sent. If we receive an invalid string, then we know that an error occurred.

For example, consider the case of two message bits and one even parity check bit, resulting in codewords with three bits. The codeword 101 is valid since it has even parity, while the codeword 001 is invalid since it has odd parity. Out of the 8 strings with 3 bits, only 4 of them are valid codewords. We should expect this since we started with only 2 message bits, which means there are only $2^2 = 4$ distinct messages that can be represented. Each valid codeword corresponds to one of these messages. This is depicted in Figure 10.6.

This view also frees us from thinking of the bits as either message bits or check bits. The point is there are 4 valid messages, which might correspond to 00, 01, 10, and 11, or more generally just four symbols s_1, s_2, s_3, s_4 . Either way, we transmit 2 bits. Separating the bits into message bits versus check bits isn't necessary. In fact, it's more general to not constrain our thinking in this way. A "code" is then just a subset of strings that we call valid codewords and that correspond to messages. The other strings are invalid. How we arrived at the valid codewords (whether by parity check bits or some other way) doesn't matter.

Notice that just picking any 4 valid codewords is not good enough to guarantee single error detection. We need to choose them cleverly. For example, suppose of the 8 strings, we chose 000, 001, 010 and 100 as the valid codewords and the others as invalid. Suppose 000 were transmitted and a single error occurred in the third bit so that we receive 001. Instead of detecting the error we would erroneously assume that 001 was the transmitted message (received without error). Hence, this collection of four codewords does not allow single error correction. In some sense, to detect or correct errors, the valid codewords need to be "far apart". How to formalize this is discussed in the next section.

10.10 Hamming Distance

The reason the example above fails to be a good code (i.e., doesn't allow even single error detection) is that the valid codewords are not "spread out" enough. Because they are too "close" to each other, a single error can take us from one valid codeword to another valid codeword, thereby preventing any type of error detection.

Formalizing the notion of "distance" between codewords will let us see exactly what condition the valid codewords need to satisfy to allow single error detection. It will also let us address higher levels of protection, both detection and correction.

Definition (Hamming Distance). Given two strings of equal length, the *Hamming distance* between them is defined to be the number of positions on which the two strings disagree.

The reason this is the appropriate notion for us is that each error changes one bit, so it moves a string by Hamming distance 1. Two errors result in a string that's Hamming distance 2 away from the original, and so on.

If two valid codewords are Hamming distance 1 apart, then a single error in the appropriate bit will take us from one valid codeword to another valid codeword, making detection of the error impossible. We see that to allow single error detection, there must be a distance of at least 2 between *any* valid codewords. Note the emphasis on "any". Even if there are just two valid codewords with distance 1 between them with all the others far apart, we still cannot claim the code will detect single errors. Remember, to make this claim we need to detect *every* single error, no matter which codeword was originally sent. This leads to the following definition.

Definition (Distance of a Code). Given a code (i.e., a set of valid codewords of equal length), the *distance of the code* is defined to be the minimum Hamming distance between any two valid codewords.

To allow single error detection, we see that the distance of the code must be 2, i.e., all valid codewords must be at least distance 2 apart.

Now, let's think about what is needed in order to correct errors. Remember that we assume a valid codeword is sent from the outset. If we receive an invalid string, we know that an error must have occurred. But, how should we go about trying to correct the error?

What “correcting” the error means is that given the observed invalid string, we find the original valid codeword that was sent. The obvious, and only reasonable, guess is to choose that valid codeword that is closest to the observation, assuming there is a unique closest valid codeword. We'll be able to recover the original codeword this way as long as after the errors, the result is still closer to the original than to any other valid codeword.

For example, if the distance of the code is 3 then all valid codewords are at least distance 3 apart. In this case, even after a single error, the result will still be closer to the original codeword than to any other valid codeword. Hence, we will be able to correct the error. However, if we use the code to correct single errors then we will not be able to detect double errors with a distance of three. If two valid codewords are a distance 3 apart, then certain double errors will cause the received bits to end up closer to a different valid codeword than the original. In this case, we'll think there was a single error and improperly “correct” this. This is depicted in Figure 10.7. However, if we are only interested in detecting errors but not correcting them, then a code with distance 3 allows detecting 2 errors.

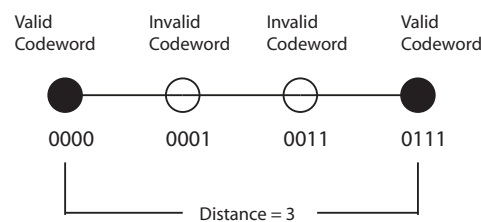


Figure 10.7: Distance 3 code.

To detect double errors *and* correct single errors, we will need a distance of 4. In this case, single errors will get corrected as before. A double error will leave us equally distant (distance 2) between two valid codewords, and so we will know there was a double error, although we won't know which was the message transmitted. This is depicted in Figure 10.8. But again, if we are only interested in error detection but not correcting single errors, then a distance 4 code will allow detecting 3 errors.

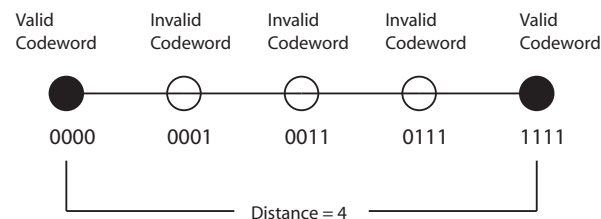


Figure 10.8: Distance 4 code.

A distance of 5 allows double error correction. A distance of 6 allows double error correction and triple error detection. A little thought shows that to correct k errors we need a distance of

$2k + 1$. To correct k errors and detect $k + 1$ errors we need a distance of $2k + 2$. Equivalently, if d is the distance of the code, then we can correct $\lfloor \frac{d-1}{2} \rfloor$ errors and can detect $\lfloor \frac{d}{2} \rfloor$ errors, where $\lfloor x \rfloor$ denotes the greatest integer less than or equal to x . Alternatively, if we are only interested in detecting errors (but not correcting them), then a distance d code allows detecting up to $d - 1$ errors.

$A(n, d)$ denotes the maximum possible number of valid codewords of length n such that the Hamming distance between any two distinct codewords is at least d . Two standard bounds on $A(n, d)$ are the *Hamming bound* and the *Gilbert–Varshamov bound*. The *Hamming bound* is an upper bound. As discussed in the previous paragraph, a code with minimum Hamming distance d can correct $t = \lfloor \frac{d-1}{2} \rfloor$ errors. For each valid codeword c , consider the Hamming ball of radius t , consisting of all binary words whose Hamming distance from c is at most t . In order for the decoder to correct t errors and recover a unique nearest codeword, the Hamming balls of radius t around distinct valid codewords must be disjoint. Each such ball contains $\sum_{i=0}^t \binom{n}{i}$ binary words, while the total number of possible binary words of length n is 2^n . Therefore, the maximum possible number of valid codewords satisfies

$$A(n, d) \leq \left\lfloor \frac{2^n}{\sum_{i=0}^t \binom{n}{i}} \right\rfloor.$$

The *Gilbert–Varshamov bound* is a lower bound. To construct a largest set of valid codewords with a minimum distance of d , we can keep adding codewords until no more can be added without violating the minimum-distance condition. And every binary word outside the set must lie within distance $d - 1$ of at least one codeword in the set; otherwise that word could be added as another codeword. Thus, the Hamming balls of radius $d - 1$ around the chosen codewords cover the entire space of binary words of length n , which implies

$$A(n, d) \geq \left\lceil \frac{2^n}{\sum_{i=0}^{d-1} \binom{n}{i}} \right\rceil.$$

10.11 Channel Coding Theorem

In the previous section, we discussed how we can achieve different levels of protection. Basically, we need to have codewords that are sufficiently far apart to achieve the desired level of protection. That is, the distance of the code (as measured by the minimum distance between any two valid codewords) must be sufficiently large. This can be achieved by taking a long enough block of bits, say a block of length n , but using just small subset of these length n bit strings as codewords.

But how much protection do we need to ensure that we can communicate reliably? Of course, this will depend on the nature of the “channel” we are using to transmit (or store) the data – i.e., on the probabilities of different kinds of errors.

It turns out that a powerful result from information theory characterizes the conditions under which data can be reliably transmitted. We will describe just a very special case of an enormous body of work in this area.

In particular, we’ll consider only the case of what is called the binary symmetric channel (or BSC), which is a very common and simple model for a noisy channel. Both the input and output are either 0 or 1, and the BSC has an associated parameter p that represents the probability that an error will occur (i.e., that the output will *not* be the same as the input). So, with probability $1 - p$ the input will be received without an error. Normally we assume that $p < 1/2$ since if it was larger than $1/2$ whatever symbol was received could simply be flipped by the receiver to mimic $p < 1/2$. Also, we assume that the channel operates independently from one symbol to the next.

The BSC can be represented as shown in Figure 10.9, and is a special case of a broader class of channels called discrete memoryless channels.

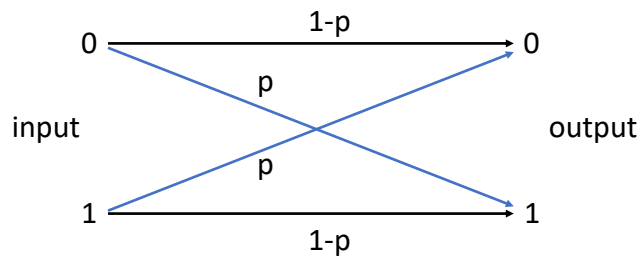


Figure 10.9: Binary Symmetric Channel.

If $p = 0$, then all bits sent through the channel are received without error, so we can transmit one bit per channel use with zero probability of error. If $p = 1/2$ then the received symbol will be 0 with probability $1/2$ and 1 with probability $1/2$ no matter whether the input is 0 or 1. If a transmitter uses this channel many times, the receiver will simply receive a random string of 0's and 1's, so the transmitter is unable to send any message that can be reliably recovered by the receiver. We want to address the question of what can be done if $0 < p < 1/2$?

Suppose we have a BSC with probability of error $p = 0.1$. We can transmit one bit per channel use, but each bit has a probability of 0.1 of being received incorrectly. If we send a message with four bits, then the probability that this 4-bit message will be received correctly is $0.9^4 = 0.651$ and so the probability of error is 0.3439. But what if we use a Hamming code to be able to correct a single error. In this case, the transmitter will send 7 bits and the receiver will receive the 4 message bits without error if there is at most one error in the received 7 bits. This happens with probability

$$0.9^7 + 7(0.1)0.9^6 \approx 0.4783 + 0.3720 = 0.8503$$

Thus, the probability of error is only 0.15. However, we only sent 4 message bits in 7 uses of the channel, for a rate of about 0.57 bits per channel use.

Some natural questions are can we do better, and if so how? And what are the fundamental limits on the data rate at which we can reliably communicate through this kind of channel?

The fundamental limit is determined by a quantity called the channel capacity. For the BSC, the channel capacity C is given by

$$C = 1 - H(p) \tag{10.3}$$

where $H(p)$ is the binary entropy function

$$H(p) = p \log_2 \frac{1}{p} + (1-p) \log_2 \frac{1}{1-p} \tag{10.4}$$

The channel coding theorem states that we can get an error rate as close to zero as we want for a sufficiently long message as long as we transmit at a rate R bits per channel use that is less than the channel capacity C . And, if we try to transmit at a rate R that is larger than C then we cannot get an arbitrarily small error rate. More formally, the following is a statement of the celebrated channel coding theorem.

Channel Coding Theorem for the BSC

Consider a binary symmetric channel and let C be the capacity of the channel as given in Equation (10.3).

- (i) (**Achievability**) For any $\epsilon > 0$ and any transmission rate $R < C$, for a sufficiently large message of length N there exists an encoder and a decoder such that we can transmit at rate R over a binary symmetric channel such that the probability of error is less than ϵ .
- (ii) (**Converse**) If the transmission rate $R > C$ then an arbitrarily small probability of error is not possible.

This result basically says there is a hard cutoff for reliable transmission. At rates less than the capacity C we can ensure that the probability of error is as close to zero as desired (albeit, for a sufficiently long message). And for rates higher than the capacity, the probability of error cannot be made close to zero. While we have stated the result for the BSC, other and more general results exist for other channels. These results guarantee that if $R < C$ suitable encoders and decoders exist, but the theorem does say what those encoders and decoders should be. Finding good codes (i.e., that result in small probability of error and are computationally feasible) is the subject of coding theory.

Chapter 11

The Laplace Transform

As we saw in earlier chapters, the Fourier transform provides a powerful way to analyze signals by decomposing them into complex exponentials. This frequency domain representation gives a powerful way to ... The Fourier transform also is important in the analysis of LTI systems.

However, the Fourier transform has some limitations. One important limitation is that it applies only when the signal is sufficiently well-behaved such as when it is absolutely integrable or square integrable. Many signals of interest do not decay fast enough to meet these conditions. The Laplace transform extends the idea of the Fourier transform by introducing an exponential weighting that enables convergence for signals whose Fourier transforms do not exist. The familiar frequency ω is replaced with a complex parameter $s = \sigma + j\omega$ which is very useful in analyzing the stability of systems. Also, the Fourier transform is not ideal for causal signals and systems. A version of the Laplace transform naturally handles this. In particular the unilateral Laplace transform (as opposed to the bilateral Laplace transform) is very useful in analyzing systems with specified initial conditions.

In this chapter, we will introduce both the bilateral Laplace transform (also called the two-sided Laplace transform) and the unilateral Laplace transform (also called the one-sided Laplace transform).

11.1 The Bilateral Laplace Transform

Recall that if a complex exponential $e^{j\omega t}$ is applied as the input to an LTI system, then the output is simply $H(\omega)e^{j\omega t}$ where

$$H(\omega) = \int_{-\infty}^{\infty} h(t)e^{-j\omega t} dt$$

is the Fourier transform of the impulse response $h(t)$. It turns out that the same calculations go through if the exponent $j\omega$ is replaced with a general complex number $s = \sigma + j\omega$. That is, if a complex exponential e^{st} is applied as the input to an LTI system, then the output is $H(s)e^{st}$ where

$$H(s) = \int_{-\infty}^{\infty} h(t)e^{-st} dt$$

*©2026 by Sanjeev R. Kulkarni. All rights reserved.

†Lecture Notes for ECE201 Information Signals.

This is precisely the definition of the bilateral Laplace transform. For a general signal $x(t)$, the Laplace transform of $x(t)$, denoted $X(s)$ is given by

$$X(s) = \int_{-\infty}^{\infty} x(t)e^{-st} dt \quad (11.1)$$

Similar to the Fourier transform, we call $x(t)$ and $X(s)$ a Laplace transform pair, and write

$$x(t) \longleftrightarrow X(s)$$

Or, to make it clear we are dealing with the Laplace transform, we may write

$$x(t) \xrightarrow{\mathcal{L}} X(s)$$

. We also may write

$$X(s) = \mathcal{L}\{x(t)\}$$

where $\mathcal{L}\{\cdot\}$ denotes the Laplace transform operator.

The bilateral Laplace transform can be seen as a natural generalization of the Fourier transform. In fact, when $s = j\omega$, we have

$$X(j\omega) = X(s)|_{s=j\omega} = \mathcal{F}[x(t)] \quad (11.2)$$

so that the Fourier transform of $x(t)$ is the Laplace transform of $x(t)$ evaluated on the imaginary axis.

For general $s = \sigma + j\omega$, we get

$$X(s) = X(\sigma + j\omega) = \int_{-\infty}^{\infty} x(t)e^{-(\sigma+j\omega)t} dt = \int_{-\infty}^{\infty} [x(t)e^{-\sigma t}] e^{-j\omega t} dt \quad (11.3)$$

so that the Laplace transform is the Fourier transform of $x(t)e^{-\sigma t}$.

11.2 Region of Convergence

Although we previously discussed the Fourier transform of functions such as constants, sinusoids, and $\text{step}(t)$, strictly speaking the Fourier transform of a signal $x(t)$ is well-defined only if $x(t)$ is absolutely integrable, i.e., if $\int_{-\infty}^{\infty} |x(t)| dt < \infty$. On the other hand, because of the extra term $e^{-\sigma t}$, the Laplace transform is well-defined for some signals and for some values of σ even when the Fourier transform does not exist.

extending Fourier analysis to signals that are not absolutely integrable. In effect, the bilateral Laplace transform provides a more flexible and general framework for analyzing signals and linear systems, while reducing to the Fourier transform when $\sigma = 0$.

Example 11.1 ()

Consider the signal $x(t) = e^{-at}\text{step}(t)$. If $a > 0$, then the Fourier transform can be easily computed as follows:

$$\mathcal{F}\{x(t)\} = \int_{-\infty}^{\infty} x(t)e^{-j\omega t} dt = \int_0^{\infty} e^{-at}e^{-j\omega t} dt = \frac{1}{j\omega + a} \quad \text{for } a > 0. \quad (11.4)$$

However, if $a < 0$, then the Fourier transform does not exist.

On the other hand, for any a we can find the Laplace transform for appropriate σ . Specifically,

$$\mathcal{L}\{x(t)\} = \int_{-\infty}^{\infty} x(t)e^{-st} dt \quad (11.5)$$

$$= \int_0^{\infty} e^{-at}e^{-st} dt \quad (11.6)$$

$$= \frac{1}{s+a} \quad \text{for } \sigma > -a. \quad (11.7)$$

Since $s = \sigma + j\omega$, we have

$$e^{-at}\text{step}(t) \xleftrightarrow{\mathcal{L}} \frac{1}{s+a} \quad \text{for } \text{Re}(s) > -a. \quad (11.8)$$

where $\text{Re}(s)$ denotes the real part of s .

Note that for $a > 0$, we can take $\sigma = 0$ in which case the Laplace transform is

$$X(s) = \left. \frac{1}{s+a} \right|_{\sigma=0} = \frac{1}{j\omega + a}$$

so that the Laplace transform on the imaginary axis ($\sigma = 0$) reduces to the Fourier transform as expected. ■

It's important to note that in the example above, the Laplace transform does not converge for all s . In this particular case, the Laplace transform converges only for $\text{Re}(s) > -a$. However, this is not unusual. It is generally the case that the Laplace transform does not converge for all s . The set of values for which the Laplace transform converges is called the *region of convergence* (abbreviated as ROC). To completely specify the Laplace transform of a signal, it's important to provide the ROC as well as the expression of the transform. This can be seen from the previous example together with the following example.

Example 11.2 ()

Consider the signal $x(t) = -e^{-at}\text{step}(-t)$. The Laplace transform is

$$\mathcal{L}\{x(t)\} = \int_{-\infty}^{\infty} x(t)e^{-st} dt \quad (11.9)$$

$$= \int_{-\infty}^0 -e^{-at}e^{-st} dt \quad (11.10)$$

$$= \int_{-\infty}^0 -e^{-t(s+a)} dt = \frac{1}{s+a} \quad \text{for } \sigma < -a. \quad (11.11)$$

The integral converges if and only if $\text{Re}(s) < -a$, so that $\text{Re}(s+a) < 0$. Therefore,

$$-e^{-at}\text{step}(-t) \xleftrightarrow{\mathcal{L}} \frac{1}{s+a} \quad \text{for } \text{Re}(s) < -a. \quad (11.12)$$
■

Notice that the expression for the Laplace transform is the same in both examples, namely $\frac{1}{s+a}$. The only difference in the two cases is the ROC, which shows the importance of specifying the ROC to uniquely define a signal.

For a causal signal $x(t)$ (i.e., a signal for which $x(t) = 0$ for all $t < 0$), the ROC is of the form $\text{Re}(s) > \sigma_0$ for some σ_0 . For an anti-causal signal (one for which $x(t) = 0$ for $t > 0$), the ROC is of the form $\text{Re}(s) < \sigma_0$ for some σ_0 . The ROC's in these cases are half-planes in the s -plane. More generally, the ROC will be of the form $\sigma_1 < \text{Re}(s) < \sigma_2$ for some σ_1, σ_2 . This is a vertical strip in the s -plane. Of course, the ROC could be empty (in which case the Laplace transform does not exist), or it could be the entire s -plane. For example, for a signal that is integrable and has finite-duration, the ROC is the entire s -plane.

11.3 Some Bilateral Laplace Transform Pairs and Properties

In Table 11.3, we provide some basic transform pairs for the bilateral Laplace transform. As mentioned above, in addition to the algebraic expression of the Laplace transform, it's also important to specify the ROC. The transforms shown can be obtained directly from the definition together with basic properties of integration.

The first entry (Laplace transform of $\delta(t)$) follows from the defining property of the delta function. The next two entries are the examples we showed above. The Laplace transform of the step function is a special case of $e^{-at}u(t)$ with $a = 0$, though it can also be easily computed directly:

$$\begin{aligned}\mathcal{L}[u(t)] &= \int_{-\infty}^{\infty} u(t)e^{-st} dt \\ &= \int_0^{\infty} e^{-st} dt = -\frac{1}{s}e^{-st} \Big|_0^{\infty} = \frac{1}{s}\end{aligned}$$

Likewise, using integration by parts, we can compute the Laplace transform of $tu(t)$:

$$\begin{aligned}\mathcal{L}[tu(t)] &= \int_{-\infty}^{\infty} tu(t)e^{-st} dt \\ &= \int_0^{\infty} t e^{-st} dt = -\frac{t}{s}e^{-st} \Big|_0^{\infty} + \int_0^{\infty} \frac{1}{s}e^{-st} dt \\ &= 0 - \left[\frac{1}{s^2}e^{-st} \right]_0^{\infty} = \frac{1}{s^2}\end{aligned}$$

Repeated integration by parts gives the result for $t^n u(t)$.

The Laplace transform shares many properties that are very similar to properties of the Fourier transform. Some of these properties are shown in Table 11.3. As with the Fourier transform, these properties can be derived straightforwardly from the definition. For example, the first property (linearity) follows immediately from the definition of the Laplace transform and linearity of integrals. The time shift property can be obtained from the definition and a simple change of variables.

Signal $x(t)$	Transform $X(s)$	ROC
$\delta(t)$	1	$-\infty < \Re(s) < \infty$
$e^{-at}u(t)$	$\frac{1}{s+a}$	$\Re(s) > -a$
$-e^{-at}u(-t)$	$\frac{1}{s+a}$	$\Re(s) < -a$
$u(t)$	$\frac{1}{s}$	$\Re(s) > 0$
$-u(-t)$	$\frac{1}{s}$	$\Re(s) < 0$
$t^n u(t)$	$\frac{n!}{s^{n+1}}$	$\Re(s) > 0$
$te^{-at}u(t)$	$\frac{1}{(s+a)^2}$	$\Re(s) > -a$
$-te^{-at}u(-t)$	$\frac{1}{(s+a)^2}$	$\Re(s) < -a$
$\cos(\omega t)$	$\frac{s}{s^2 + \omega^2}$	$\Re(s) = 0$
$\sin(\omega t)$	$\frac{\omega}{s^2 + \omega^2}$	$\Re(s) = 0$

Table 11.1: Some Common Bilateral Laplace Transform Pairs

Specifically, with the substitution $\tau = t - t_0$, we get

$$\begin{aligned}
 \mathcal{L}[x(t - t_0)] &= \int_{-\infty}^{\infty} x(t - t_0) e^{-st} dt \\
 &= \int_{-\infty}^{\infty} x(\tau) e^{-s(\tau + t_0)} d\tau \\
 &= e^{-st_0} \int_{-\infty}^{\infty} x(\tau) e^{-s\tau} d\tau \\
 &= e^{-st_0} X(s)
 \end{aligned}$$

Property	Time Domain	Laplace Transform	ROC
	$x(t), x_1(t), x_2(t)$	$X(s), X_1(s), X_2(s)$	R, R_1, R_2
Linearity	$ax_1(t) + bx_2(t)$	$aX_1(s) + bX_2(s)$	At least $R_1 \cap R_2$
Time Shift	$x(t - t_0)$	$e^{-st_0} X(s)$	R
s -Shift	$e^{-at} x(t)$	$X(s + a)$	$R - a$
Scaling	$x(at)$	$\frac{1}{ a } X\left(\frac{s}{a}\right)$	aR
Differentiation	$\frac{d^n x(t)}{dt^n}$	$s^n X(s)$	At least R
Integration	$\int_{-\infty}^t x(\tau) d\tau$	$\frac{1}{s} X(s)$	At least $R \cap \{\operatorname{Re}(s) > 0\}$
Multiply by t^n	$t^n x(t)$	$(-1)^n \frac{d^n X(s)}{ds^n}$	R
Convolution	$x_1(t) * x_2(t)$	$X_1(s) X_2(s)$	At least $R_1 \cap R_2$

Table 11.2: Bilateral Laplace Transform Properties

11.4 Inverse Laplace Transform

The definition of the Laplace transform in Equation (11.1) is the forward transform. A natural question is whether there is an equation for an inverse transform as there is for the Fourier transform. Recall Equation (11.3) that writes the Laplace transform of $x(t)$ as the Fourier transform of the exponentially weighted signal $x(t)e^{-\sigma t}$. We assume σ is chosen in the ROC (i.e., so that $\sigma + j\omega$ is in the ROC for all ω).

Using the formula for the inverse Fourier transform, we get

$$x(t)e^{-\sigma t} = \mathcal{F}^{-1}[X(\sigma + j\omega)] = \frac{1}{2\pi} \int_{-\infty}^{\infty} X(\sigma + j\omega) e^{j\omega t} d\omega \quad (11.13)$$

so that

$$x(t) = \frac{1}{2\pi} \int_{-\infty}^{\infty} X(\sigma + j\omega) e^{(\sigma + j\omega)t} d\omega \quad (11.14)$$

In the expression above, we can change the variable of integration from ω to $s = \sigma + j\omega$. Since σ is a constant, $ds = j d\omega$ and so

$$x(t) = \frac{1}{2\pi j} \int_{\sigma - j\infty}^{\sigma + j\infty} X(s) e^{st} ds \quad (11.15)$$

The equation for the inverse transform is an integral in the complex s -plane along the vertical line $s = \sigma$, where σ is any fixed constant such that the vertical line is in the ROC. In principle, the inverse transform expression could be used to recover $x(t)$ given $X(s)$ and a σ in the ROC. However, In practice the inverse is usually found using tables of known transforms, partial fraction decomposition, or manipulation using other known transform pairs.

11.5 The (Unilateral) Laplace Transform

The bilateral Laplace transform deals with signals defined for all t – i.e., for $-\infty < t < \infty$. However, in some applications, the signals of interest start at a specific time, usually assumed to be $t = 0$. Also, the systems of interest are often subject to initial conditions in addition to the input to the system. In these cases, the *unilateral* Laplace transform is a more appropriate tool. The unilateral Laplace transform is closely related to the bilateral Laplace transform and is defined as follows:

$$X(s) = \int_{0^-}^{\infty} x(t) e^{-st} dt \quad (11.16)$$

Before continuing, it's important to make some comments on this definition and notation. Note that the lower limit is 0^- rather than just say 0 or 0^+ . The reason for 0^- is to account for initial conditions and for possible impulses at $t = 0$. The lower limit of 0^- means

$$\lim_{\epsilon \rightarrow 0^+} \int_{-\epsilon}^{\infty} x(t) e^{-st} dt \quad (11.17)$$

so that the integral includes 0. This ensures that initial conditions and impulses at $t = 0$ are accounted for in the definition of the unilateral Laplace transform.

Also, note that we use the same notation $X(s)$ for the unilateral Laplace transform that we used for the bilateral Laplace transform. This is quite standard and the meaning is usually clear from the context. If it's not clear from the context or if there is a need for both uses, we can write $X_b(s)$ and $X_u(s)$ to make the distinction.

Likewise, for the Laplace transform operator we can write $\mathcal{L}_b$ and/or $\mathcal{L}_u$ instead of simply $\mathcal{L}$ if it's not clear from the context. Often, especially in systems and control, $\mathcal{L}$ with no subscript refers to the unilateral Laplace transform.